UNIVERSIDAD TECNOLÓGICA DE XICOTEPEC DE JUÁREZ

UNIVERSIDAD TECNOLÓGICA DE XICOTEPEC DE JUAREZ

INGENIERÍA EN DESARROLLO Y GESTIÓN DE SOFTWARE

DESARROLLO MÓVIL INTEGRAL

MANUAL DE INSTALACION Y CONFIGURACION DEL ENTORNO DE DESARROLLO PARA APLICACIONES MÓVILES MULTIPLATAFORMA

VERSIÓN 1

Integrantes:
ANGEL DE JESUS BAÑOS TELLEZ
JESUS ALEJANDRO ARTIAGA MORALES
FRANCISCO GARCIA GARCIA

Fecha de Elaboración: 14/SEPTIEMBRE/2026

Listado de Tecnologías Utilizadas

Flutter · FastAPI / Node.js-Express · PostgreSQL · MongoDB · Keycloak · Docker · Git · GitHub

2. Control de versiones del documento

Versión	Fecha	Autor	Cambio realizado
1.0	14/09/2026	Angel de Jesus Baños Tellez	Versión inicial
1.1	18/09/2026	Jesus Alejandro Artiaga Morales	
1.2	22/09/2026	Francisco Garcia Garcia	

.

3. Introducción

En la actualidad, la creación de aplicaciones móviles demanda soluciones eficientes, escalables y adaptables a distintos sistemas operativos sin duplicar esfuerzos de ingeniería. Para comprender el enfoque de este proyecto, es fundamental analizar los paradigmas de desarrollo, la modularidad de los sistemas y las herramientas modernas de infraestructura.

3.1. ¿Qué es el desarrollo móvil multiplataforma?

El desarrollo móvil multiplataforma consiste en la creación de aplicaciones de software orientadas a diferentes sistemas operativos móviles (principalmente Android e iOS) utilizando una única base de código fuente. A diferencia de los enfoques tradicionales, este modelo permite optimizar recursos y acelerar el tiempo de entrega al reutilizar la lógica de negocio y la interfaz de usuario en las distintas plataformas.

3.2. Diferencias entre desarrollo nativo y multiplataforma

- **Desarrollo Nativo:** Se construye de forma específica para un sistema operativo utilizando sus lenguajes y herramientas oficiales (por ejemplo, Kotlin/Java para Android o Swift/Objective-C para iOS). Ofrece el máximo rendimiento y acceso directo a las características de hardware del dispositivo, pero requiere mantener códigos separados para cada plataforma, incrementando costos y tiempos de desarrollo.
- **Desarrollo Multiplataforma:** Utiliza frameworks que compilan o renderizan componentes para múltiples sistemas operativos a partir de un lenguaje común. Aunque tradicionalmente existía una brecha en términos de rendimiento frente al desarrollo nativo, las tecnologías actuales logran una experiencia de usuario fluida y una alta cercanía al rendimiento nativo, reduciendo drásticamente la duplicidad de código.

3.3. Importancia de la separación de responsabilidades

Para garantizar que una aplicación sea mantenible, escalable y fácil de probar, es una buena práctica arquitectónica separar claramente los componentes del sistema:

- **Frontend:** Capa encargada de la interfaz gráfica y de la interacción directa con el usuario final en su dispositivo móvil.
- **Backend:** Núcleo de la lógica de negocio y procesamiento de datos que centraliza las reglas de la aplicación.
- **Autenticación:** Módulo centralizado encargado de verificar la identidad de los usuarios y gestionar los permisos de acceso de forma segura.
- **Almacenamiento:** Subsistema responsable de la persistencia de los datos, garantizando su integridad y disponibilidad.

- **Infraestructura:** Capa de soporte físico o virtualizada donde se despliegan y ejecutan los servicios que sostienen la aplicación.

3.4. Uso de APIs como mecanismo de comunicación

Las Interfaces de Programación de Aplicaciones (APIs), particularmente bajo el estilo arquitectónico REST, fungen como el contrato estándar de comunicación entre el cliente (la aplicación móvil) y el servidor. Permiten desacoplar el frontend del backend, asegurando que cualquier cambio o evolución en la interfaz o en la lógica de procesamiento no comprometa la estabilidad del sistema completo, siempre y cuando se respeten los formatos y protocolos de intercambio de datos.

3.5. Importancia de los entornos reproducibles

Garantizar entornos de desarrollo, prueba y producción idénticos es vital para evitar el problema clásico de *"en mi máquina sí funciona"*. Contar con entornos reproducibles asegura que el software se comporte exactamente igual sin importar dónde se ejecute, minimizando errores imprevistos durante los despliegues y facilitando la colaboración entre los integrantes del equipo técnico.

3.6. Arquitectura general del sistema

El flujo general de operación de la plataforma integra los conceptos anteriores mediante una arquitectura por capas y microservicios:

$$\text{Flutter} \longrightarrow \text{API REST} \longrightarrow \text{Backend} \longrightarrow \text{Bases de Datos}$$
Este flujo principal se encuentra complementado y respaldado por componentes transversales clave:

- **Keycloak:** Proveedor de gestión de identidad y acceso que centraliza los flujos de autenticación y autorización en todo el ecosistema.
- **Docker / Docker Compose:** Herramientas de contenedorización que aseguran la portabilidad y la creación de **entornos reproducibles**, empaquetando cada servicio con sus respectivas dependencias.

4. Objetivo del manual

Objetivo general

Instalar, configurar y validar un entorno de desarrollo que permita construir aplicaciones móviles multiplataforma con Flutter, consumir APIs REST, implementar servicios backend, gestionar bases de datos relacionales y NoSQL, incorporar mecanismos de autenticación y utilizar herramientas modernas de control de versiones y contenerización.

Objetivos específicos

El estudiante será capaz de:

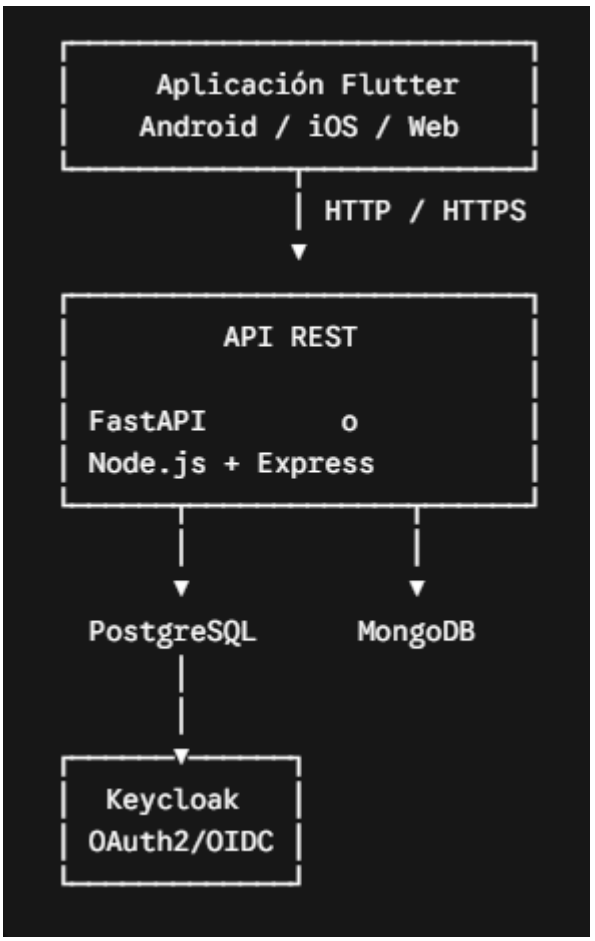
- Configurar Flutter y sus dependencias.
- Crear aplicaciones móviles.
- Instalar y administrar PostgreSQL.
- Instalar y utilizar MongoDB.
- Implementar APIs REST.
- Documentar APIs mediante OpenAPI/Swagger.
- Probar servicios mediante Postman.
- Configurar Keycloak.
- Utilizar Docker y Docker Compose.
- Gestionar proyectos mediante Git y GitHub.
- Integrar una aplicación Flutter con una API.

5. Arquitectura tecnológica propuesta

Esta sección es importante porque conecta la teoría con lo que posteriormente se instalará en el entorno de desarrollo. Proporciona una visión global de cómo interactúan las distintas tecnologías seleccionadas para construir un sistema moderno, escalable y seguro.

5.1 Arquitectura general

El siguiente diagrama ilustra el flujo de comunicación de la aplicación, desde la interfaz de usuario hasta la persistencia de datos y la gestión de identidades:



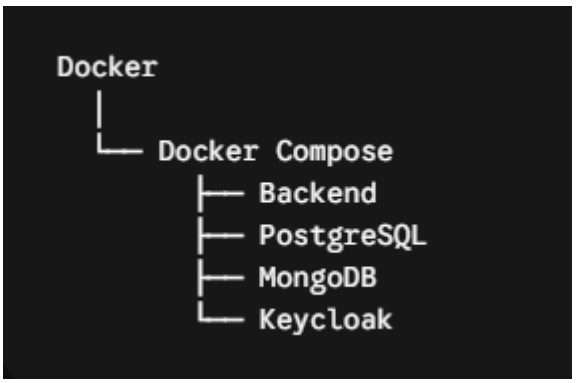
Justificación teórica de la arquitectura:

- **Capa de Presentación (Flutter):** Actúa como el cliente multiplataforma. Realiza peticiones HTTP/HTTPS hacia el servidor para consumir o enviar datos, ofreciendo una experiencia de usuario fluida.
- **Capa Lógica (API REST):** Desarrollada con FastAPI o Node.js + Express, sirve como intermediario. Recibe las peticiones del cliente, procesa la lógica de negocio y determina a qué base de datos consultar.

- **Capa de Datos:** Se propone un modelo híbrido. **PostgreSQL** (base de datos relacional) maneja datos altamente estructurados y transaccionales, mientras que **MongoDB** (base de datos NoSQL) ofrece flexibilidad para almacenar datos no estructurados o documentos dinámicos.
- **Capa de Seguridad (Keycloak):** Implementa los protocolos OAuth2 y OpenID Connect (OIDC) para gestionar la autenticación y autorización. Notablemente, Keycloak se apoya en PostgreSQL para almacenar de forma segura las credenciales y configuraciones de los usuarios.

5.2 Infraestructura

Para garantizar que el entorno sea reproducible y evitar conflictos con el sistema operativo local, se implementa la siguiente estructura de contenerización:



Justificación teórica de la infraestructura: El uso de **Docker Compose** permite orquestar múltiples servicios simultáneamente. A través de un único archivo de configuración (docker-compose.yml), se levantan de manera aislada el backend, ambas bases de datos y el servidor de identidades (Keycloak), asegurando que todos operen bajo la misma red virtual y se comuniquen entre sí de forma transparente.

5.3 Desarrollo colaborativo

El control del código fuente y la gestión de versiones se administrará de la siguiente manera:

Git → GitHub

Justificación teórica del control de versiones: Git funcionará de manera local para rastrear los cambios en el código (commits, ramas), mientras que GitHub actuará como el repositorio remoto. Esto no solo protege el código ante pérdida de datos, sino que facilita la colaboración, revisión y el despliegue del proyecto.

6. Descripción del stack tecnológico

A continuación, se presenta una tabla introductoria que detalla las tecnologías que componen el entorno de desarrollo, clasificadas por su categoría y la función específica que desempeñan en el proyecto:

Tecnología	**Categoría**	**Función**
Flutter	Framework	Desarrollo móvil multiplataforma.
Dart	Lenguaje	Lenguaje utilizado por Flutter.
FastAPI	Backend	Desarrollo de APIs con Python.
Node.js	Runtime	Ejecución JavaScript en servidor.
Express	Framework	Desarrollo de APIs con Node.
PostgreSQL	BD relacional	Persistencia estructurada.
MongoDB	BD NoSQL	Persistencia documental.
Keycloak	IAM	Autenticación y autorización.
Swagger/OpenAPI	Documentación	Documentación y prueba de APIs.
Postman	Testing	Consumo y pruebas de APIs.

Docker	Contenedores	Aislamiento de servicios.
Docker Compose	Orquestación	Ejecución coordinada de contenedores.
Git	SCM	Control de versiones.
GitHub	Plataforma	Desarrollo colaborativo.

7. Requisitos de hardware

Para garantizar un rendimiento óptimo durante el desarrollo, es necesario cumplir con ciertos requerimientos físicos en el equipo de cómputo. Estos se dividen en mínimos (para ejecutar las herramientas de forma básica) y recomendados (para una experiencia fluida y sin cuellos de botella).

7.1. Configuración mínima

- **Procesador:** Arquitectura de 64 bits.
- **Memoria RAM:** 8 GB.
- **Almacenamiento:** 20 a 30 GB de espacio libre.
- **Características especiales:** Virtualización de hardware habilitada.
- **Red:** Conexión a Internet.

7.2. Configuración recomendada

- **Procesador:** Intel Core i5 / AMD Ryzen 5 o superior.
- **Memoria RAM:** 16 GB o superior.
- **Almacenamiento:** Unidad de Estado Sólido (SSD) con 50 GB o más de espacio disponible.
- **Características especiales:** Virtualización de hardware habilitada.

> **Nota importante sobre la Memoria RAM:** La capacidad de memoria es un factor crítico en este entorno de trabajo. Contar con 16 GB o más es altamente recomendado debido a que durante el ciclo de desarrollo se ejecutarán simultáneamente herramientas de alto consumo: el Android Emulator, el motor de Docker (alojando Keycloak, PostgreSQL y MongoDB) y el propio Entorno de Desarrollo Integrado (IDE).

8. Software requerido

A continuación se presenta la lista de comprobación de todas las herramientas y tecnologías que se instalarán y configurarán a lo largo de este manual:

- [ ] Visual Studio Code
- [ ] Git
- [ ] Flutter SDK
- [ ] Dart SDK
- [ ] Android Studio
- [ ] Android SDK
- [ ] Emulador Android
- [ ] Docker Desktop
- [ ] PostgreSQL (Vía Docker)
- [ ] MongoDB (Vía Docker)
- [ ] Postman
- [ ] Python

- [ ] FastAPI
- [ ] Node.js
- [ ] Express
- [ ] Keycloak (Vía Docker)

9. Preparación del sistema operativo

Antes de proceder con la descarga e instalación de las herramientas de software, es indispensable preparar el sistema operativo base para evitar conflictos de compatibilidad.

9.1. Actualizaciones del sistema

Es fundamental verificar que el sistema operativo cuente con las últimas actualizaciones de seguridad y parches de estabilidad instalados a través de su gestor oficial (como Windows Update o el actualizador del sistema de macOS/Linux).

9.2. Configuración de virtualización

Para que herramientas como Docker Desktop y el Emulador de Android funcionen de manera nativa y rápida, es obligatorio habilitar las extensiones de virtualización por hardware directamente en la BIOS o UEFI de la computadora. Dependiendo del fabricante del procesador, esta tecnología se denomina:

- **Intel VT-x (Virtualization Technology):** Implementación de Intel para ejecutar máquinas virtuales con acceso directo a los recursos de hardware.
- **AMD-V (AMD Virtualization):** Contraparte de los procesadores AMD que cumple la misma función de aislamiento y aceleración por hardware.

9.3. Variables de entorno

Las variables de entorno son valores dinámicos que el sistema operativo utiliza para determinar el comportamiento de los procesos. Las más importantes a configurar en este entorno son:

- **`PATH`:** Define la lista de directorios donde el sistema operativo buscará automáticamente los archivos ejecutables cuando se ingresa un comando en la terminal, evitando tener que escribir la ruta completa.
- **`JAVA_HOME`:** Apunta al directorio de instalación del Java Development Kit (JDK), un requisito base de Android Studio.
- **`ANDROID_HOME`:** Indica la ruta exacta donde se encuentran las herramientas del SDK de Android.

9.4. Terminal y PowerShell

El manejo de la interfaz de línea de comandos (CLI) es esencial para configurar el entorno. Para interactuar fluidamente, se deben dominar los siguientes comandos básicos:

- **`cd` (Change Directory):** Sirve para navegar y cambiar el directorio de trabajo actual.
- **`dir` (o `ls`):** Lista los archivos y carpetas contenidos en un directorio.
- **`mkdir` (Make Directory):** Crea una o varias carpetas nuevas.
- **`cls` (o `clear`):** Limpia el texto de la pantalla de la terminal.

Conceptos de navegación clave:

- **Ruta absoluta:** Es la ruta completa y exacta hacia un archivo o directorio, comenzando desde la raíz del sistema de archivos (ej. `C:\Usuarios\Desarrollo\Proyecto`).
- **Ruta relativa:** Es la ruta hacia un archivo que parte de la ubicación actual en la que se encuentra el usuario en la terminal (ej. `.\Proyecto`).
- **Variable de entorno (uso):** En la consola, se pueden invocar utilizando sintaxis específica (como `%ANDROID_HOME%` en Windows o `$ANDROID_HOME` en sistemas basados en Unix) para acceder rápidamente a rutas del sistema.

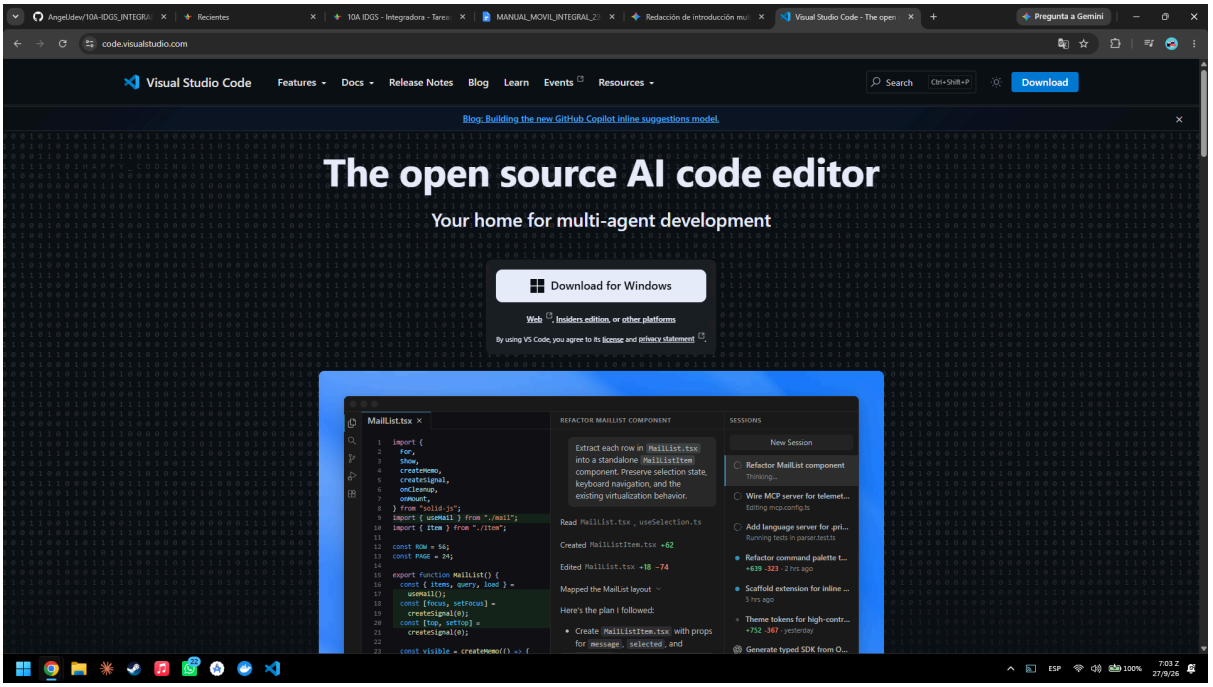
10. Instalación de Visual Studio Code

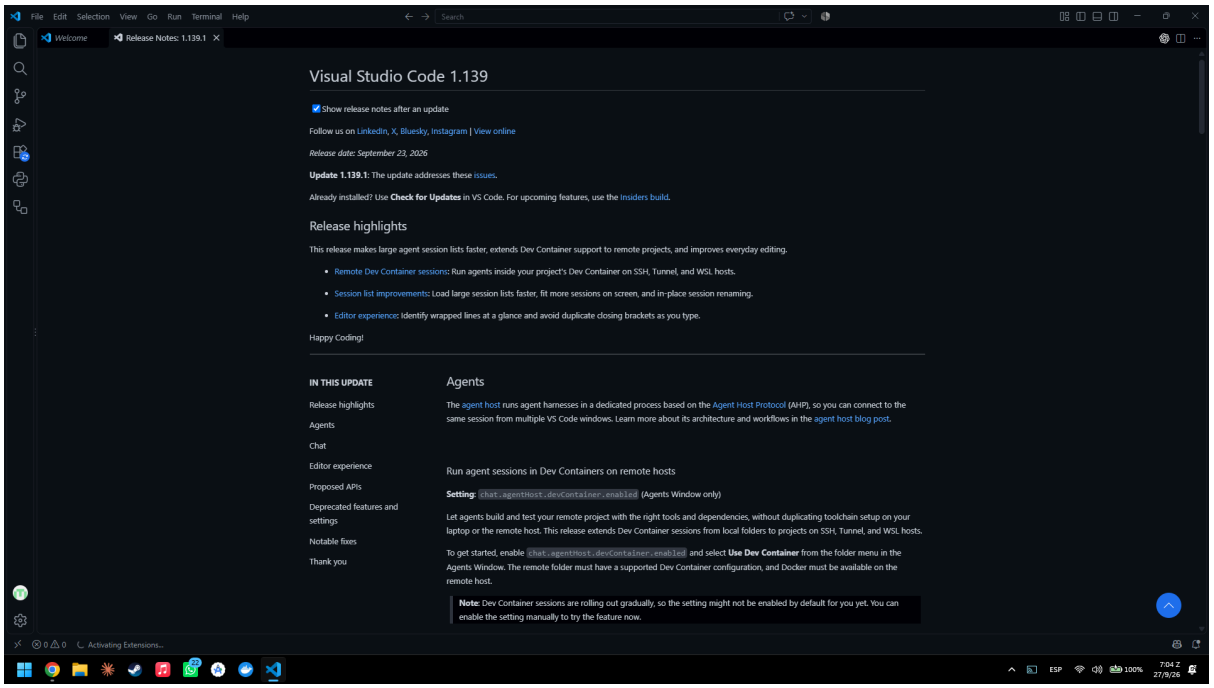
10.1 ¿Qué es un IDE/editor?

- **Editor de código:** Es un software ligero diseñado específicamente para escribir y editar código fuente, ofreciendo resaltado de sintaxis y autocompletado básico.
- **IDE (Entorno de Desarrollo Integrado):** Es una suite más pesada y robusta que incluye editor, compilador, depurador y herramientas de construcción (como Android Studio).
- **Visual Studio Code (VS Code):** Se posiciona en un punto intermedio. Es un editor de código altamente extensible que, mediante complementos, puede alcanzar las capacidades de un IDE completo manteniendo un rendimiento excepcional.

10.2 Instalación

1. Navegar a la página oficial de Visual Studio Code.
2. Descargar el instalador correspondiente al sistema operativo (Windows, macOS o Linux).
3. Ejecutar el instalador y seguir el asistente, asegurándose de marcar las opciones "Agregar al PATH" y "Abrir con Code" en el menú contextual.





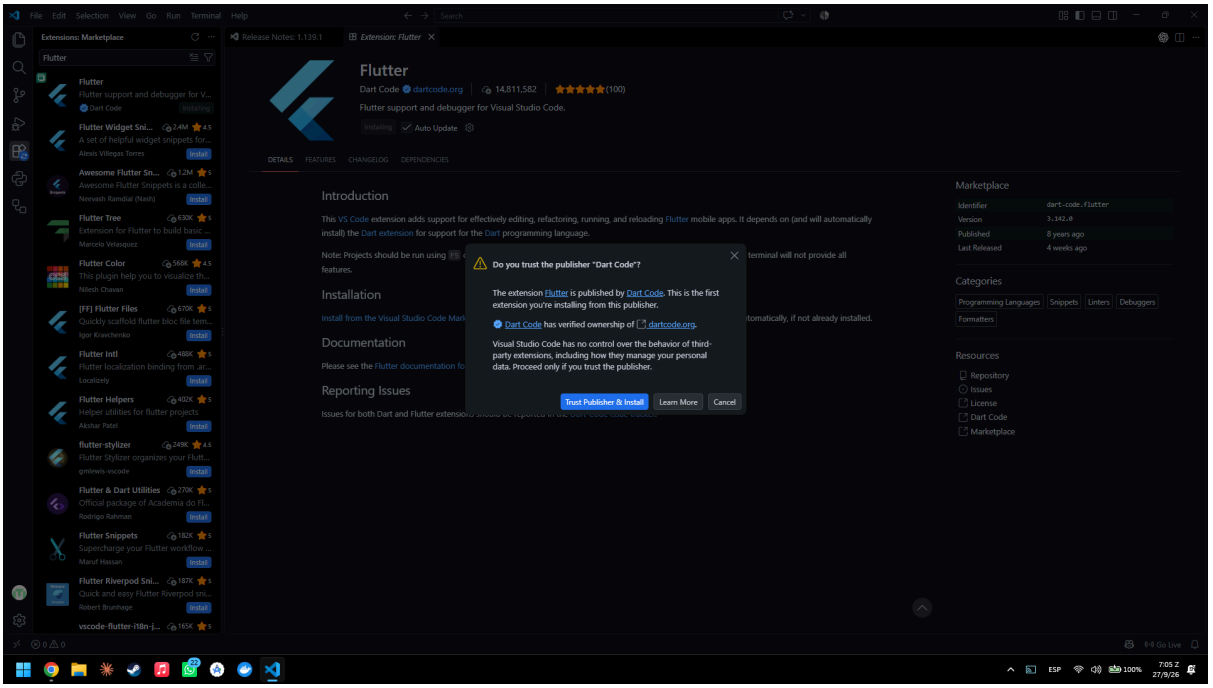
10.3 Extensiones recomendadas

Para adaptar VS Code a nuestro stack tecnológico, se recomienda instalar las siguientes extensiones desde el panel lateral (Ctrl+Shift+X):

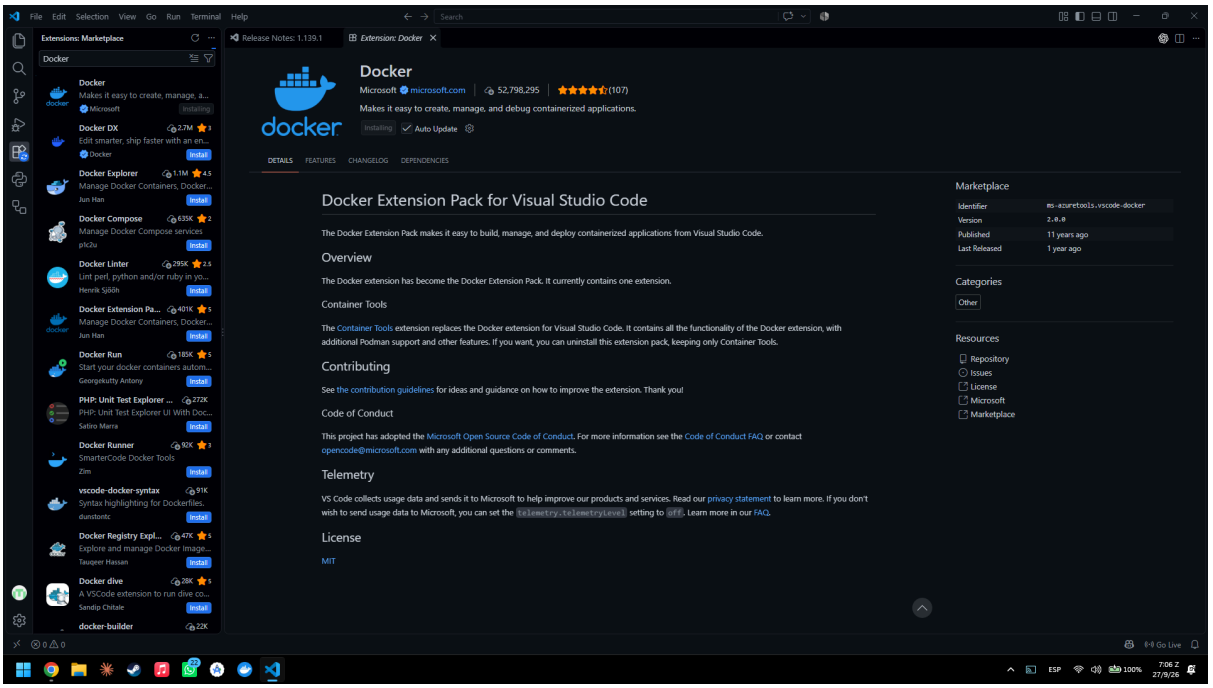
- **Flutter:** Soporte oficial para compilar, depurar y utilizar las herramientas del framework.
- **Dart:** Lenguaje base de Flutter (se instala automáticamente junto con la extensión de Flutter).
- **Python:** Para el desarrollo del backend, incluyendo soporte para el entorno de FastAPI.
- **Docker:** Facilita la gestión de contenedores, imágenes y archivos `docker-compose.yml` directamente desde el editor.
- **GitHub Pull Requests and Issues:** Para gestionar repositorios, revisiones y ramas sin salir del editor.
- **REST Client:** Permite enviar peticiones HTTP y ver las respuestas directamente en un archivo de texto plano.
- **Thunder Client (Opcional):** Una alternativa ligera a Postman integrada nativamente en VS Code.
- **PostgreSQL:** Para conectarse, explorar tablas y ejecutar consultas SQL en la base de datos relacional.
- **MongoDB for VS Code:** Para navegar y consultar las colecciones de la base de datos NoSQL.

UNIVERSIDAD TECNOLÓGICA DE XICOTEPEC DE JUÁREZ

Flutter

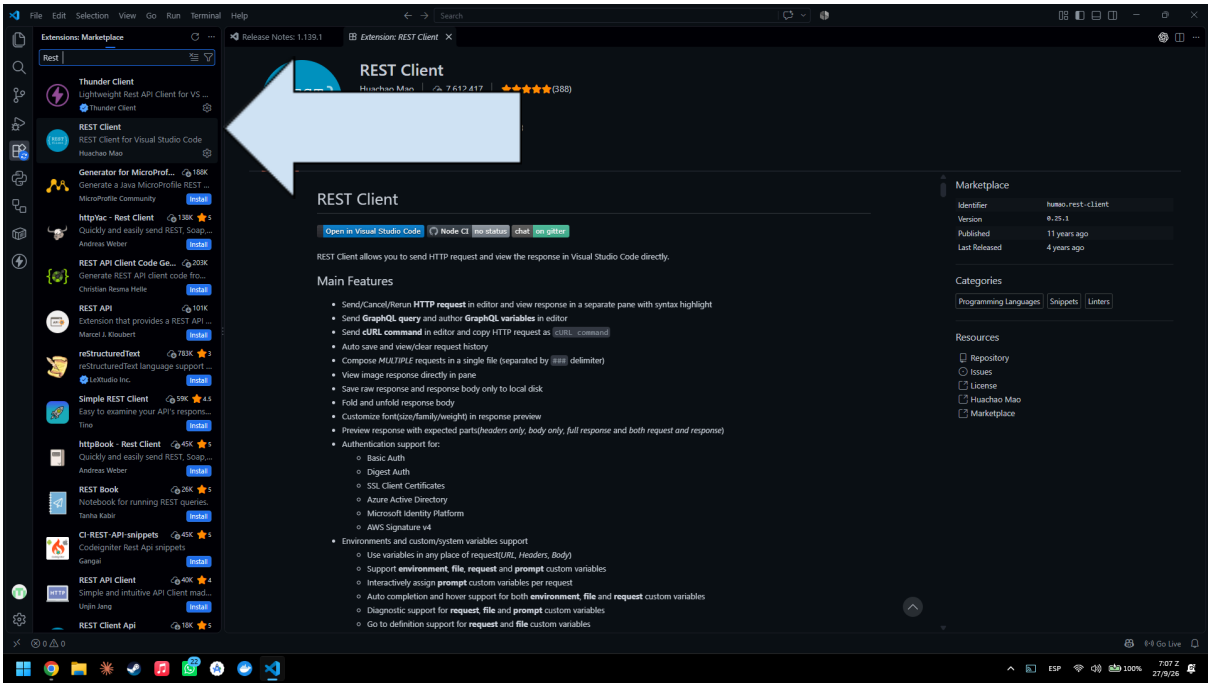


Docker

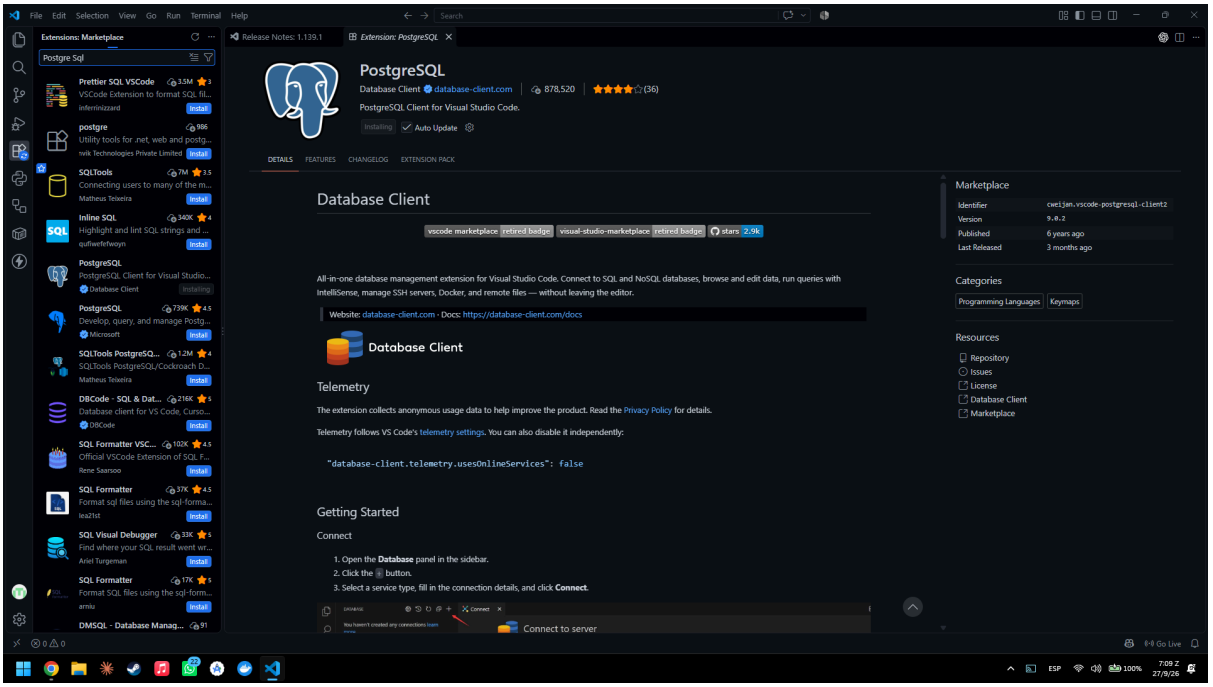


UNIVERSIDAD TECNOLÓGICA DE XICOTEPEC DE JUÁREZ

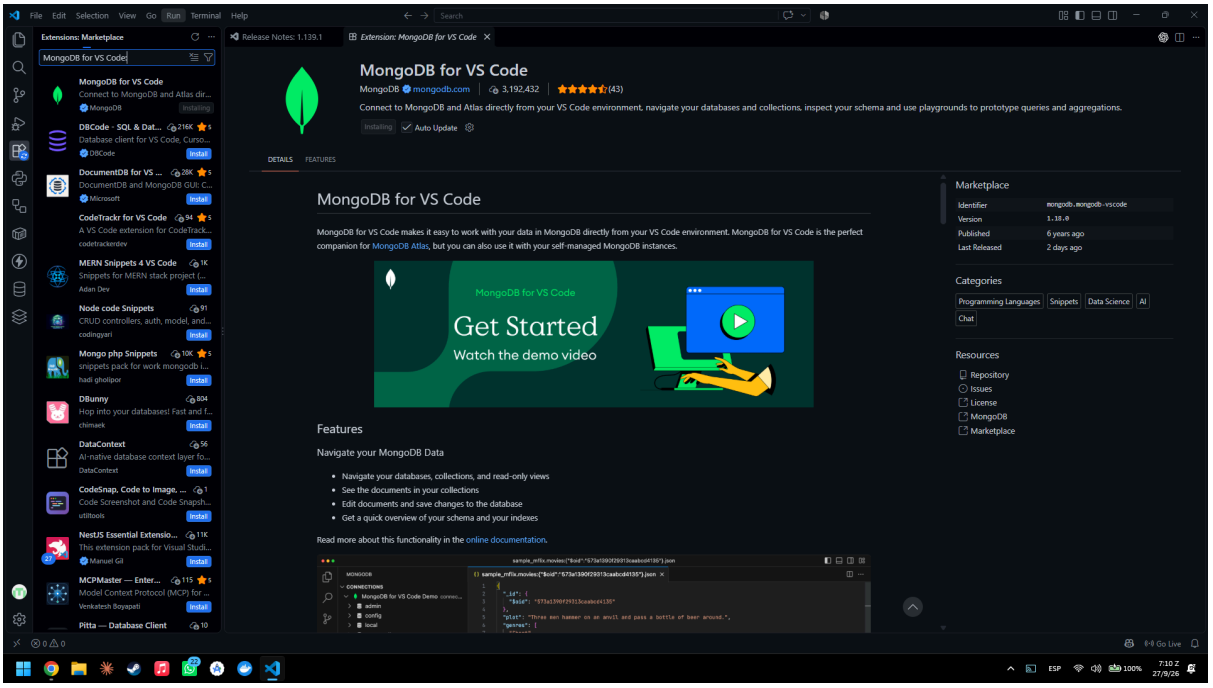
Rest y Thunder Client



PostgreSQL



MongoDB for VS CODE

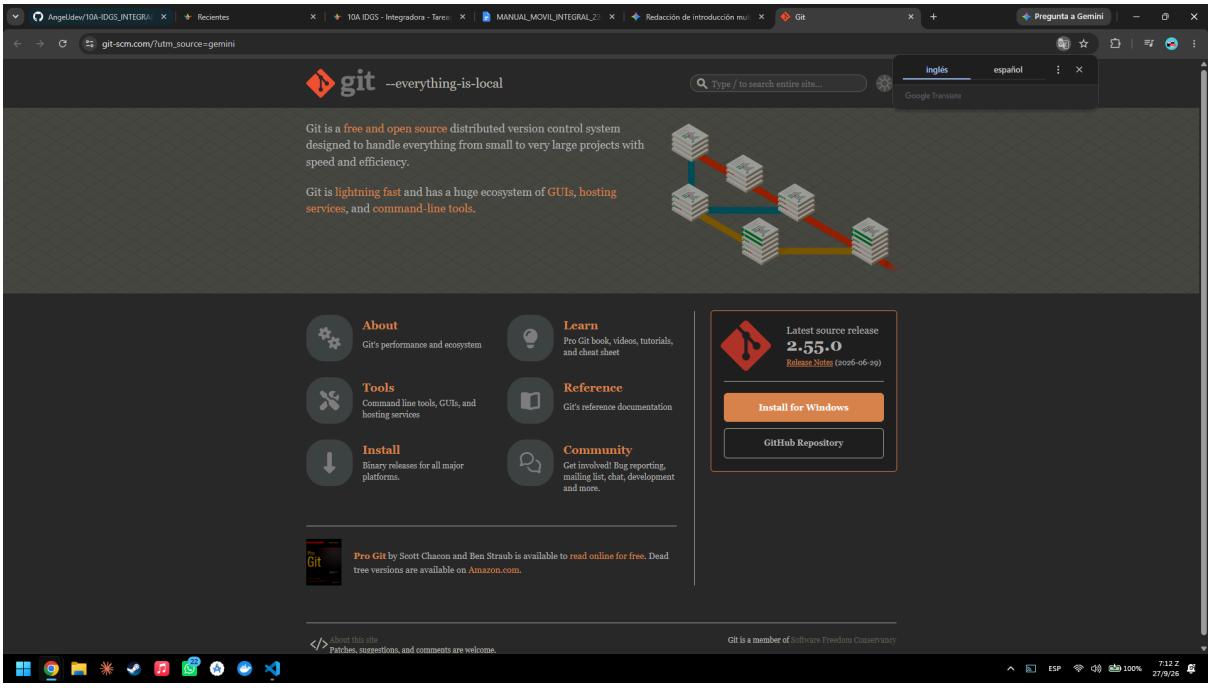


11.1 Conceptos fundamentales

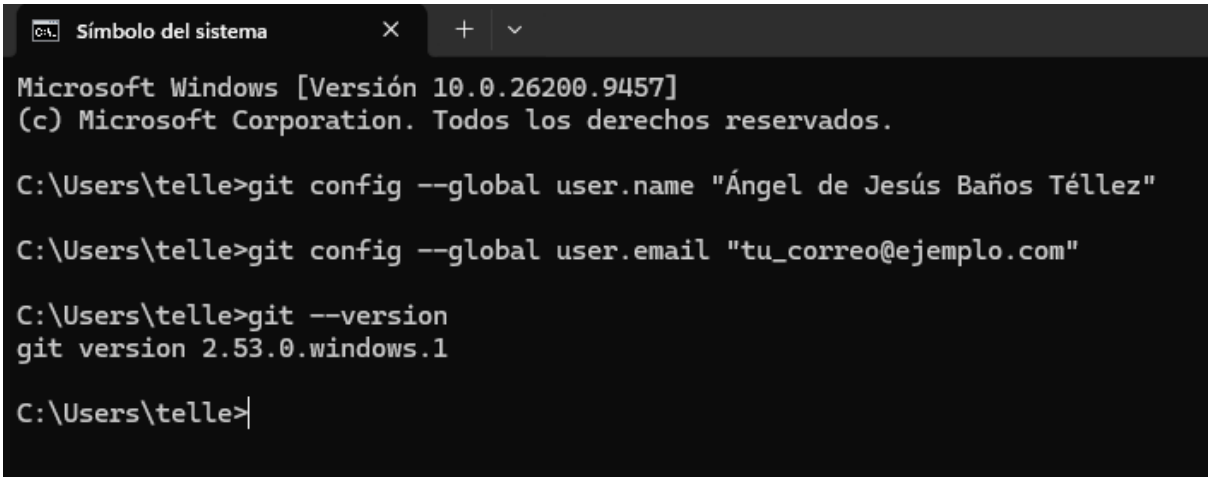
- Repositorio (Repo): Un directorio donde Git rastrea todos los cambios de los archivos que contiene.
- Commit: Una "fotografía" o punto de control permanente en el historial del repositorio que registra los cambios realizados.
- Branch (Rama): Una línea de desarrollo independiente. Permite trabajar en nuevas características sin afectar el código principal (usualmente la rama `main` o `master`).
- Merge: El proceso de fusionar los cambios de una rama secundaria hacia otra rama.
- Remote: Una versión del repositorio alojada en internet o en una red (como GitHub), que permite sincronizar el trabajo local.
- Push: La acción de enviar los *commits* locales al repositorio remoto.
- Pull: La acción de descargar los cambios del repositorio remoto e integrarlos al directorio de trabajo local.
- Clone: Crear una copia exacta de un repositorio remoto en la máquina local.

11.2 Instalación

1. Descargar el instalador desde el sitio oficial [git-scm.com](git-scm.com).
2. Ejecutar la instalación dejando la configuración recomendada por defecto (especialmente la que añade Git al PATH del sistema).



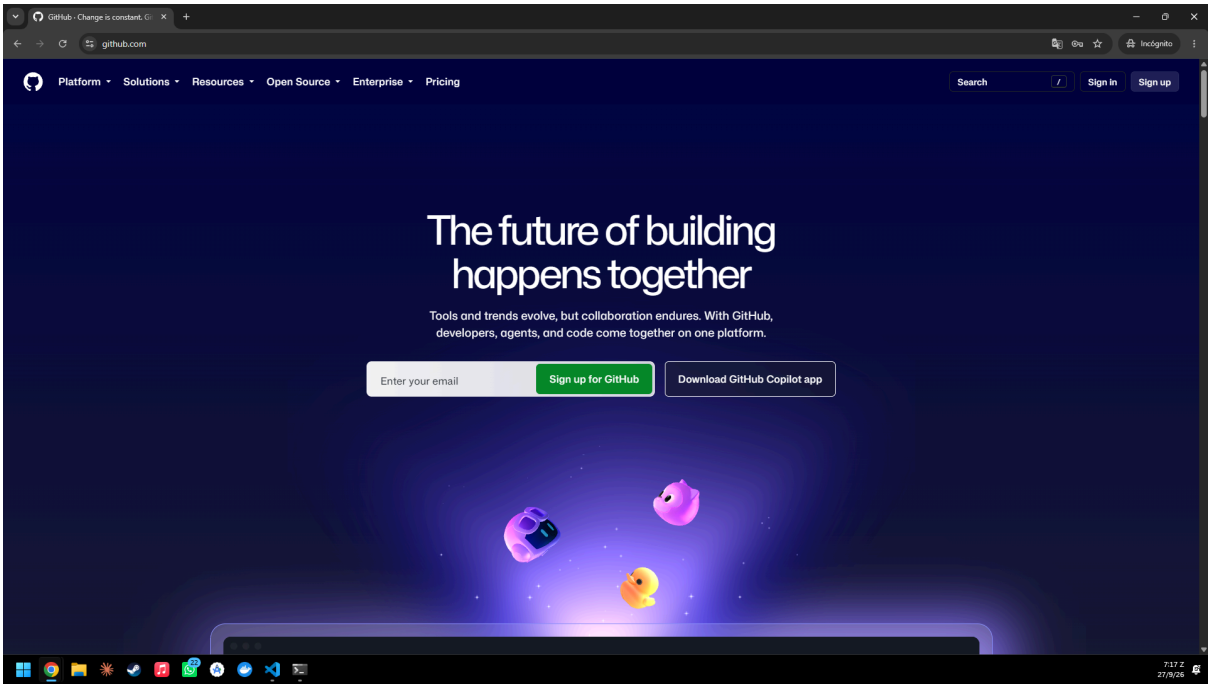
11.3 Configuración inicial



12. Configuración de GitHub

12.1 Creación de cuenta

Acceder a github.com y registrarse utilizando un correo electrónico, preferentemente institucional o profesional, y configurar un nombre de usuario identificable.



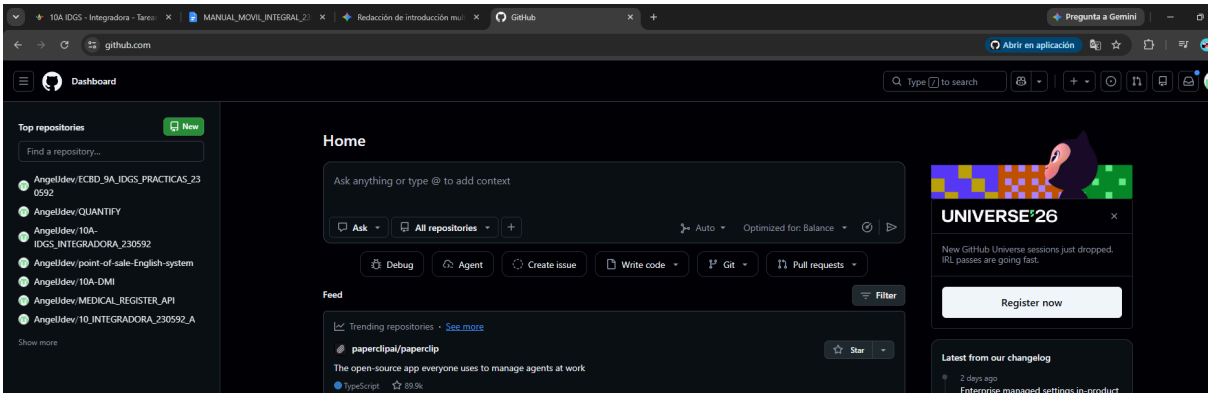
12.2 Configuración de autenticación

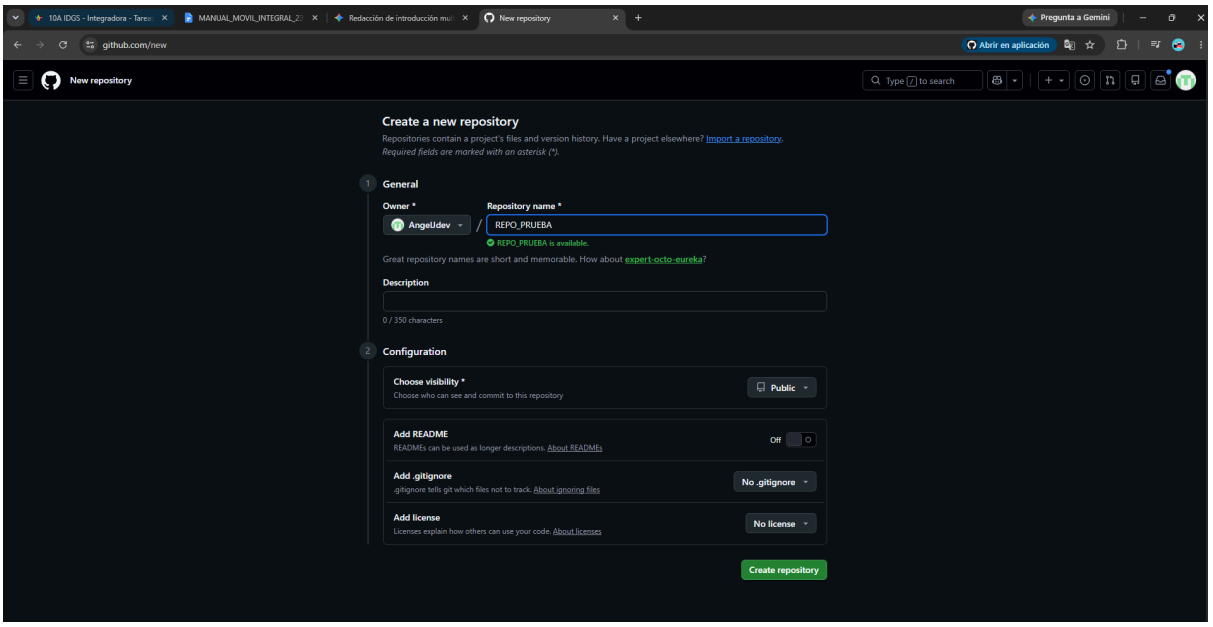
Para que Git local se comunique con GitHub, existen dos vías principales:

- **HTTPS:** Utiliza un Personal Access Token (PAT) en lugar de una contraseña tradicional. Es fácil de configurar, pero requiere autenticarse o cachear las credenciales.
- **SSH:** Utiliza un par de claves criptográficas (pública y privada). La clave pública se guarda en GitHub y la privada en la computadora. Es más seguro y evita ingresar contraseñas en cada acción.

12.3 Creación de repositorio

En la plataforma de GitHub, navegar al botón "New repository", asignar un nombre al proyecto, seleccionar si será Público o Privado, y (opcionalmente) inicializarlo con un archivo





12.4 Clonado

Para vincular el repositorio remoto con el equipo local:

```
PS C:\Users\telle> git clone https://github.com/AngelJdev/REPO_PRUEBA.git
Cloning into 'REPO_PRUEBA'...
warning: You appear to have cloned an empty repository.
PS C:\Users\telle> ^C
PS C:\Users\telle> cd .\REPO_PRUEBA\
PS C:\Users\telle\REPO_PRUEBA> mkdir Readme.md
```

12.5 Flujo básico de trabajo en Git

El ciclo de vida de los archivos en Git sigue un flujo de tres estados principales antes de llegar a la nube:

```
PS C:\Users\telle\REPO_PRUEBA> git add README.md
>> git commit -m "Initial commit"
[main (root-commit) 5ef903f] Initial commit
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 README.md
PS C:\Users\telle\REPO_PRUEBA> git branch -M main
>> git push -u origin main
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Writing objects: 100% (3/3), 269 bytes | 269.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/AngelJdev/REPO_PRUEBA.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

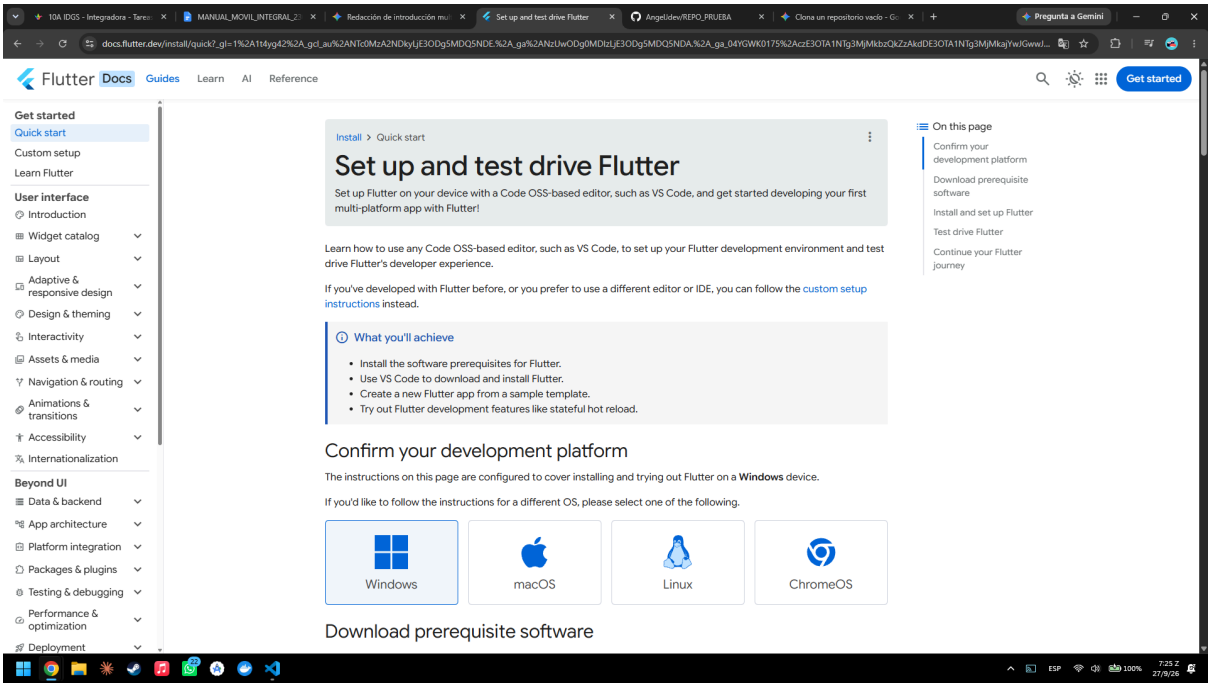
13. Instalación de Flutter

13.1 ¿Qué es Flutter?

- **Framework de UI:** Es un kit de desarrollo de interfaz de usuario de código abierto creado por Google.
- **Multiplataforma:** Permite compilar aplicaciones nativas para móviles (iOS/Android), web y escritorio desde una única base de código.
- **Widgets:** En Flutter, absolutamente todo en la pantalla es un *Widget* (desde un botón hasta el diseño estructural), los cuales se anidan para construir interfaces complejas.
- **Dart:** Es el lenguaje de programación orientado a objetos en el que se basa Flutter, optimizado para interfaces gráficas rápidas.
- **Hot Reload:** Su característica estrella; permite inyectar cambios de código fuente en la máquina virtual en ejecución, reflejando el diseño en el emulador en milisegundos sin perder el estado de la aplicación.

13.2 Instalación de Flutter SDK

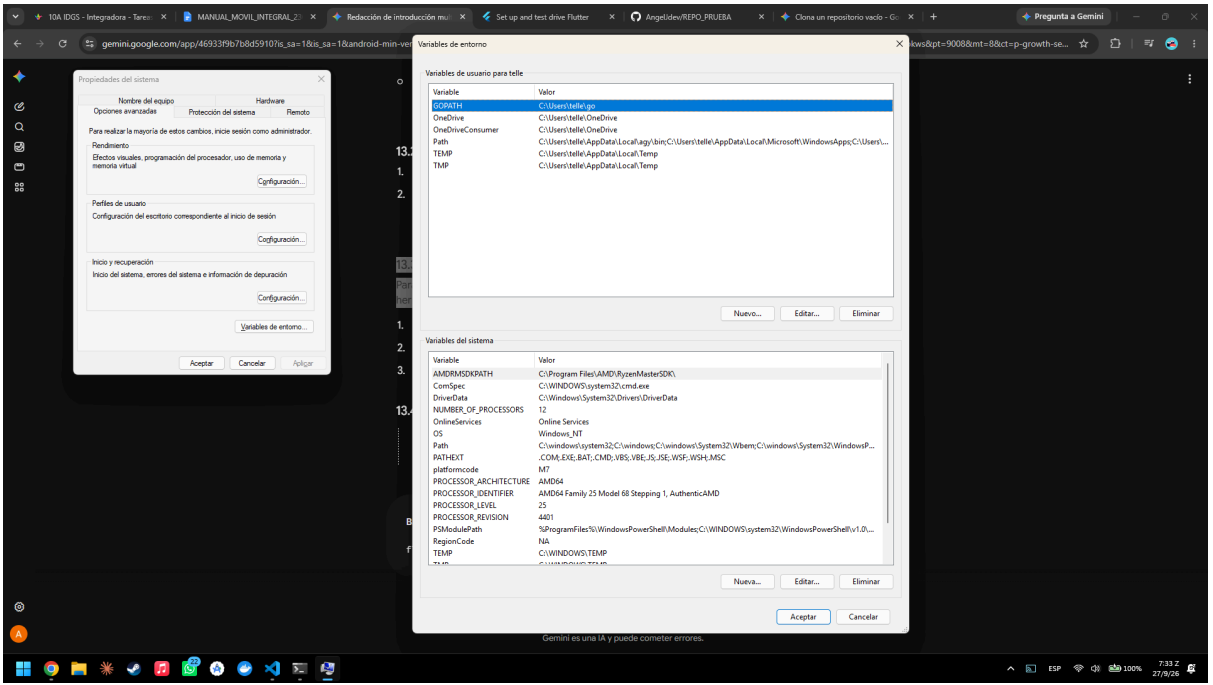
1. Descargar el archivo comprimido del SDK de Flutter desde flutter.dev.
2. Extraer el archivo en una ubicación de fácil acceso en el disco duro, por ejemplo: `C:\src\flutter`. *(Nota: Evitar instalarlo en carpetas con privilegios especiales como "Archivos de programa").*



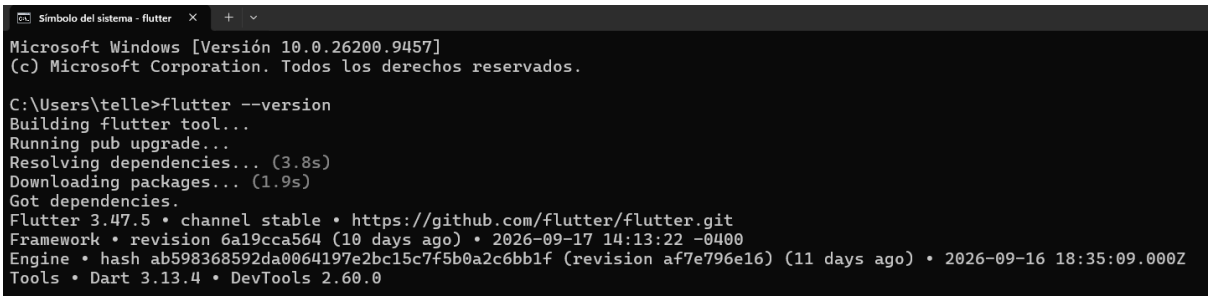
13.3 Configuración del PATH

Para ejecutar comandos de Flutter desde cualquier terminal, es necesario agregar la ruta de sus herramientas a las variables de entorno:

1. Abrir la configuración de variables de entorno del sistema operativo.
2. Editar la variable `PATH`.
3. Añadir la ruta completa a la carpeta `bin` de Flutter (ejemplo: `C:\src\flutter\bin`).



13.4 Diagnóstico



14. Instalación de Android Studio y Android SDK

Android Studio es el Entorno de Desarrollo Integrado (IDE) oficial para Android. Aunque programaremos en VS Code, instalar Android Studio es obligatorio para obtener las herramientas de compilación que Flutter necesita para generar las aplicaciones móviles.

Durante la instalación de Android Studio, se descargarán los siguientes componentes esenciales:

14.1 Android SDK

El Kit de Desarrollo de Software (SDK) contiene las bibliotecas y herramientas base necesarias para construir, probar y depurar aplicaciones en la plataforma Android.

14.2 SDK Platform

Son los archivos específicos de una versión concreta del sistema operativo Android (por ejemplo, Android 13 o API 33) contra la cual se compilará la aplicación.

14.3 SDK Build Tools

Herramientas encargadas de empaquetar el código fuente, los recursos y las bibliotecas en el formato ejecutable final (`.apk` o `.aab`).

14.4 Platform Tools

Incluye utilidades que interactúan con el dispositivo, destacando ADB (Android Debug Bridge), el cual permite instalar la aplicación en el emulador o en un teléfono físico para depuración.

14.5 Android Emulator

Simula un dispositivo Android en la computadora, permitiendo probar la aplicación en diferentes tamaños de pantalla y versiones de sistema operativo sin necesidad de hardware físico.

15. Creación de un dispositivo virtual Android

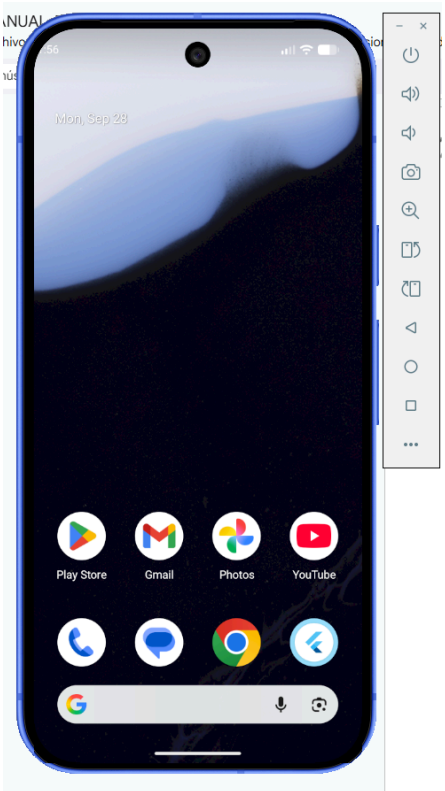
Para ejecutar la aplicación, es necesario configurar un emulador a través del Administrador de Dispositivos de Android Studio.

- **AVD (Android Virtual Device):** Es la configuración que define las características de hardware y software del emulador.
- **Imagen Android**: Es el sistema operativo (ROM) que ejecutará el AVD. Se recomienda descargar imágenes con las APIs de Google incluidas.
- **Arquitectura:** Define el tipo de procesador del emulador. Para un rendimiento óptimo, debe coincidir con la arquitectura de la computadora (x86_64 para Intel/AMD, o ARM64 si se usa hardware específico).
- **RAM:** Cantidad de memoria asignada al emulador. Se recomienda un mínimo de 2 GB (2048 MB) para una ejecución fluida.
- **Almacenamiento:** Espacio en disco reservado para instalar aplicaciones y guardar datos de usuario en el entorno virtual.

```
C:\Users\telle\OneDrive\Escritorio\10A-DMI>flutter devices
Found 4 connected devices:
  sdk gphone16k x86 64 (mobile) • emulator-5554 • android-x64    • Android 17 (API 37) (emulator)
  Windows (desktop)             • windows       • windows-x64    • Microsoft Windows [Versión 10.0.26200.9457]
  Chrome (web)                  • chrome        • web-javascript • Google Chrome 153.0.8010.53
  Edge (web)                    • edge          • web-javascript • Microsoft Edge 153.0.4234.32

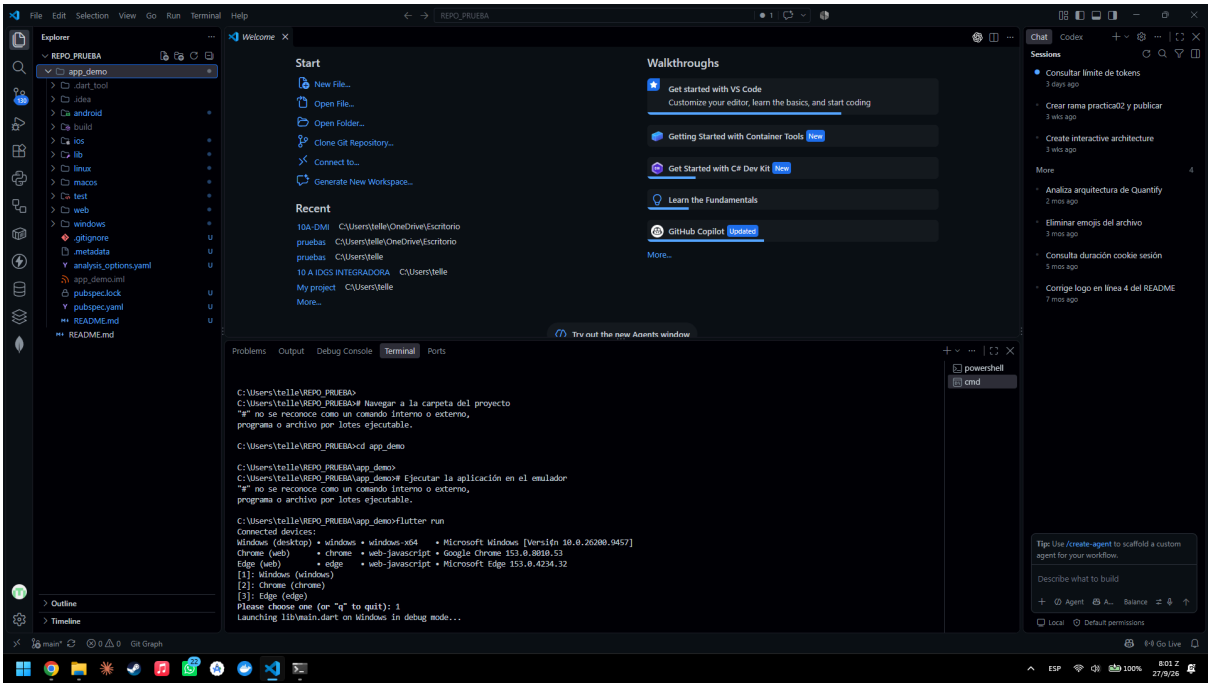
Run "flutter emulators" to list and start any available device emulators.

If you expected another device to be detected, please run "flutter doctor" to diagnose potential issues. You may also try increasing the time to wait for
connected devices with the "--device-timeout" flag. Visit https://flutter.dev/setup/ for troubleshooting tips.
```



16. Primer proyecto Flutter

Con el entorno configurado, procedemos a generar el proyecto base.



Estructura base del proyecto

El comando `create` genera múltiples directorios, siendo los más relevantes:

- `android/` e `ios/`: Contienen el código nativo y configuraciones específicas para cada plataforma. Rara vez se modifican directamente.
- `lib/`: Es el corazón del proyecto. Aquí reside todo el código fuente en lenguaje Dart. El archivo `main.dart` es el punto de entrada de la aplicación.
- `test/`: Directorio destinado a las pruebas unitarias y de integración.
- `pubspec.yaml`: Archivo de configuración central donde se declaran las dependencias (paquetes de terceros), fuentes e imágenes del proyecto.
- `README.md`: Documentación principal del proyecto en formato Markdown.

17. Fundamentos de APIs REST

Antes de implementar el backend, es fundamental comprender cómo se comunicará nuestra aplicación Flutter con el servidor.

17.1 Cliente-servidor

Es la arquitectura donde un cliente (la app móvil) solicita recursos o servicios, y un servidor (el backend) procesa la solicitud y devuelve una respuesta.

17.2 HTTP

El Protocolo de Transferencia de Hipertexto (HTTP) es el estándar de comunicación en la web. Define cómo se formatean y transmiten los mensajes entre cliente y servidor.

17.3 Métodos HTTP

Definen la intención de la petición. En una API REST, se mapean directamente a las operaciones CRUD (Crear, Leer, Actualizar, Eliminar).

Método	Operación	Función
GET	**Consultar**	**Solicita la representación de un recurso.**
POST	**Crear**	**Envía datos al servidor para crear un nuevo recurso.**
PUT	**Actualizar**	**Reemplaza completamente un recurso existente.**
PATCH	**Modificar**	**Aplica modificaciones parciales a un recurso.**
DELETE	**Eliminar**	**Borra un recurso específico.**

17.4 Códigos de estado HTTP

Indican el resultado de la petición:

- **200 OK: Petición exitosa.**
- **201 Created: Recurso creado exitosamente (típico en POST).**
- **400 Bad Request: Petición malformada por parte del cliente.**
- **401 Unauthorized: Se requiere autenticación (ej. token faltante).**
- **403 Forbidden: Autenticado, pero sin permisos para acceder al recurso.**
- **404 Not Found: El recurso solicitado no existe.**
- **500 Internal Server Error: Falla inesperada en el servidor.**

17.5 JSON

JavaScript Object Notation (JSON) es el formato de texto ligero estándar para el intercambio de datos entre la API y la aplicación móvil.

JSON

{

"id": 1,

"nombre": "Usuario"

}

18. Instalación y configuración de Postman

18.1 ¿Qué es un cliente API?

Es una herramienta gráfica que permite construir peticiones HTTP, enviarlas a un servidor y analizar la respuesta sin necesidad de programar un frontend.

18.2 Collections (Colecciones)

Agrupan peticiones relacionadas, permitiendo organizar las pruebas de la API por módulos y guardarlas para su ejecución posterior.

18.3 Environments (Entornos)

Permiten gestionar configuraciones dependiendo del contexto de ejecución (Desarrollo, Pruebas, Producción), alterando el comportamiento de las peticiones sin modificarlas manualmente.

18.4 Variables

Se utilizan para almacenar valores dinámicos y reutilizables dentro de las colecciones, inyectándose en las URLs, cabeceras o cuerpo de la petición mediante llaves dobles. Ejemplo:

- {{base_url}}: Puede apuntar a http://localhost:8000 en desarrollo y a un dominio real en producción.
- {{token}}: Almacena temporalmente la credencial de autenticación del usuario.

18.5 Primera petición

Una vez levantado un servidor local, la prueba básica desde Postman consiste en configurar un método GET apuntando a:

Plaintext

GET http://localhost:8000/

19. Backend — Ruta A: FastAPI

19.1 ¿Qué es FastAPI?

Es un framework web moderno y de alto rendimiento para construir APIs con Python, basado en la validación de tipos estándar. Es ideal para microservicios debido a su velocidad y generación automática de documentación.

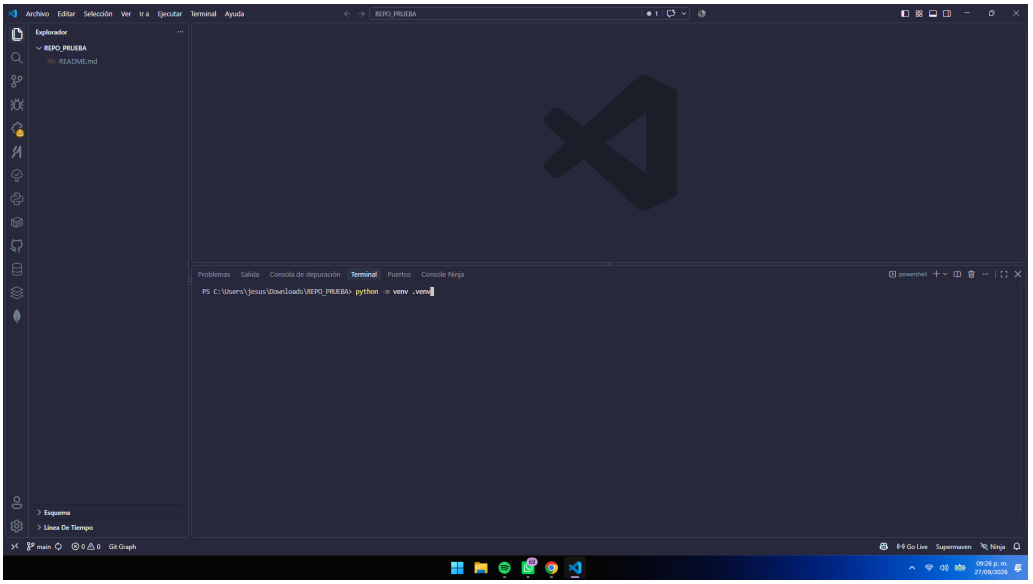
19.2 Instalación de Python

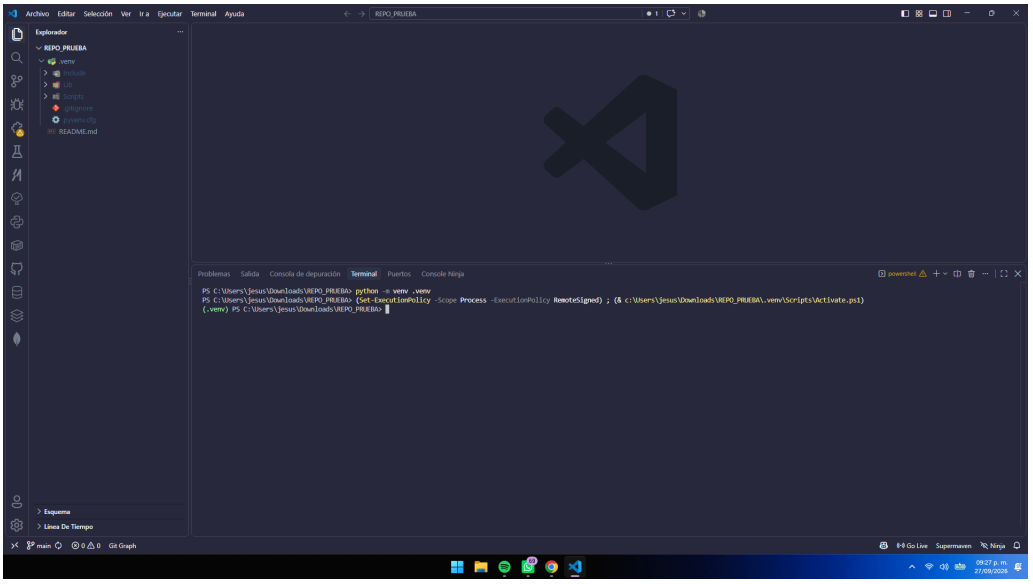
Descargar el instalador desde python.org y asegurarse de marcar la opción "Add Python to PATH" durante la instalación.

19.3 Entorno virtual

Aísla las dependencias del proyecto para evitar conflictos con otras aplicaciones Python.

python -m venv .venv





19.4 Activación

Fundamento teórico: Crear la carpeta del entorno con `venv` no basta; es necesario "activarlo" para que la terminal redirija las rutas de ejecución de Python y `pip` hacia esa carpeta aislada y no al sistema global.

Procedimiento técnico:

- En Windows (PowerShell):

  .venv\Scripts\Activate.ps1

- En macOS / Linux:

  source .venv/bin/activate

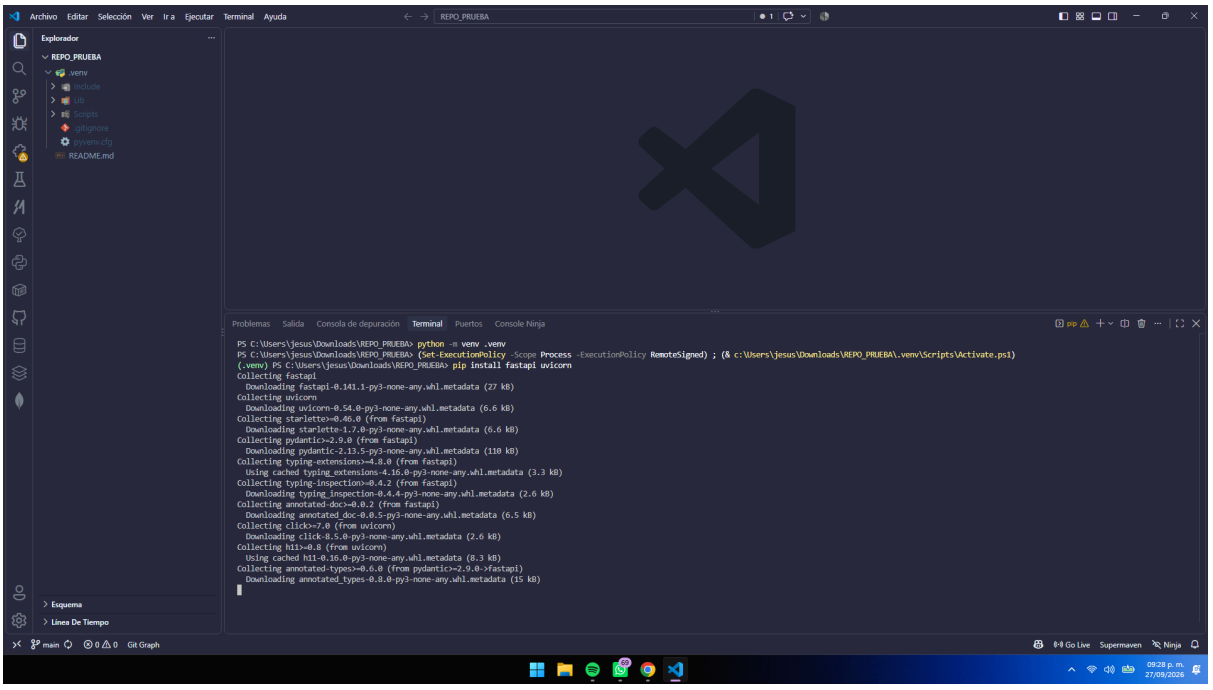
```
(.venv) PS C:\Users\jesus\Downloads\REPO_PRUEBA> .venv\Scripts\Activate.ps1
(.venv) PS C:\Users\jesus\Downloads\REPO_PRUEBA>
```

19.5 Instalación de dependencias

Fundamento teórico: FastAPI requiere un servidor ASGI (*Asynchronous Server Gateway Interface*) para procesar peticiones concurrentes y asíncronas de manera eficiente. Uvicorn cumple esta función.

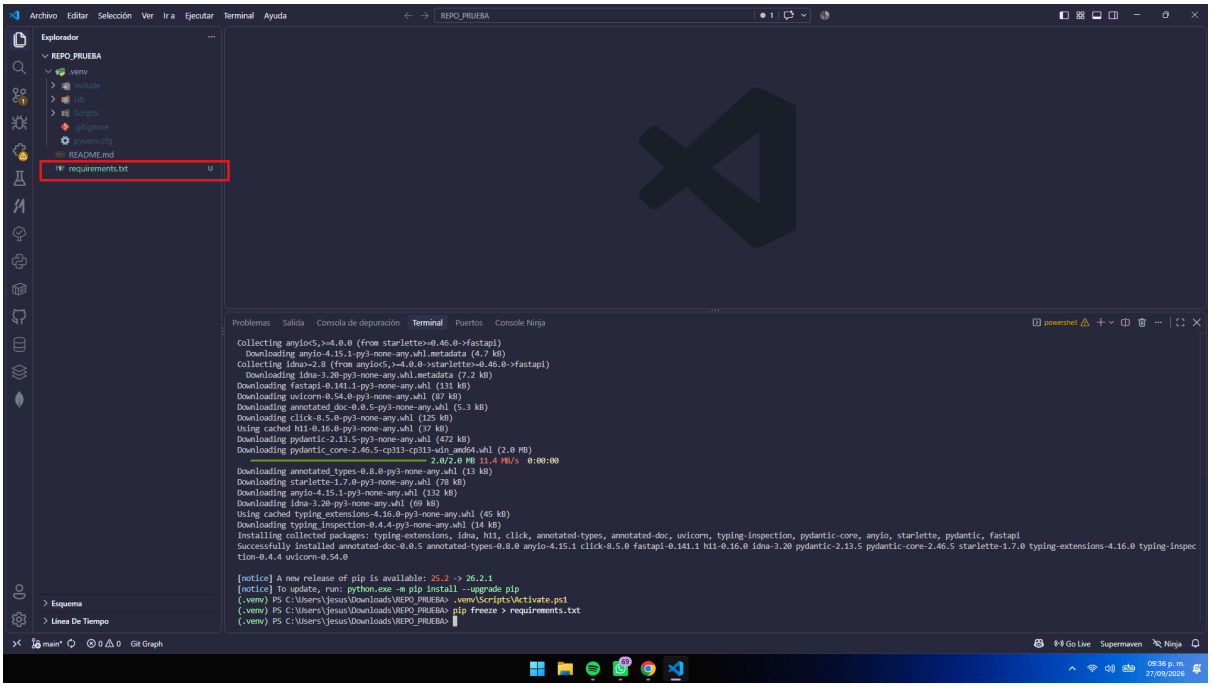
Procedimiento técnico:

pip install fastapi uvicorn



Para asegurar la reproducibilidad del entorno, se genera el archivo de bloqueo:

pip freeze > requirements.txt



Validación: Verificar que el archivo `requirements.txt` contenga las versiones exactas instaladas.

19.6 API básica y arquitectura por capas

- **Fundamento teórico:** En entornos profesionales no se coloca toda la lógica dentro de los endpoints. Se sigue un patrón multicapa:
  - **Endpoint / Controller:** Recibe la petición HTTP, valida el esquema de entrada y devuelve la respuesta.
  - **Service:** Contiene las reglas de negocio (validaciones, cálculos).
  - **Repository / Data Access:** Realiza las consultas hacia la base de datos.
- **Procedimiento técnico:** Crear el archivo `main.py`:

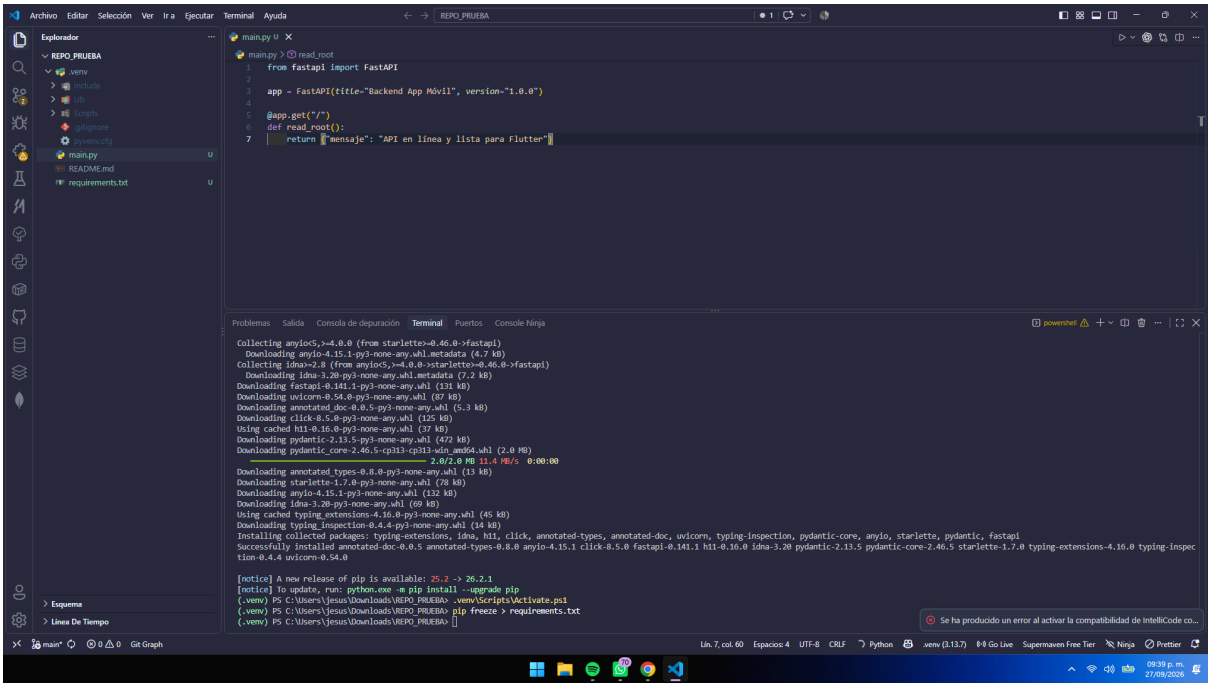
```
from fastapi import FastAPI

app = FastAPI(title="Backend App Móvil", version="1.0.0")

@app.get("/")

def read_root():

    return {"mensaje": "API en línea y lista para Flutter"}
```

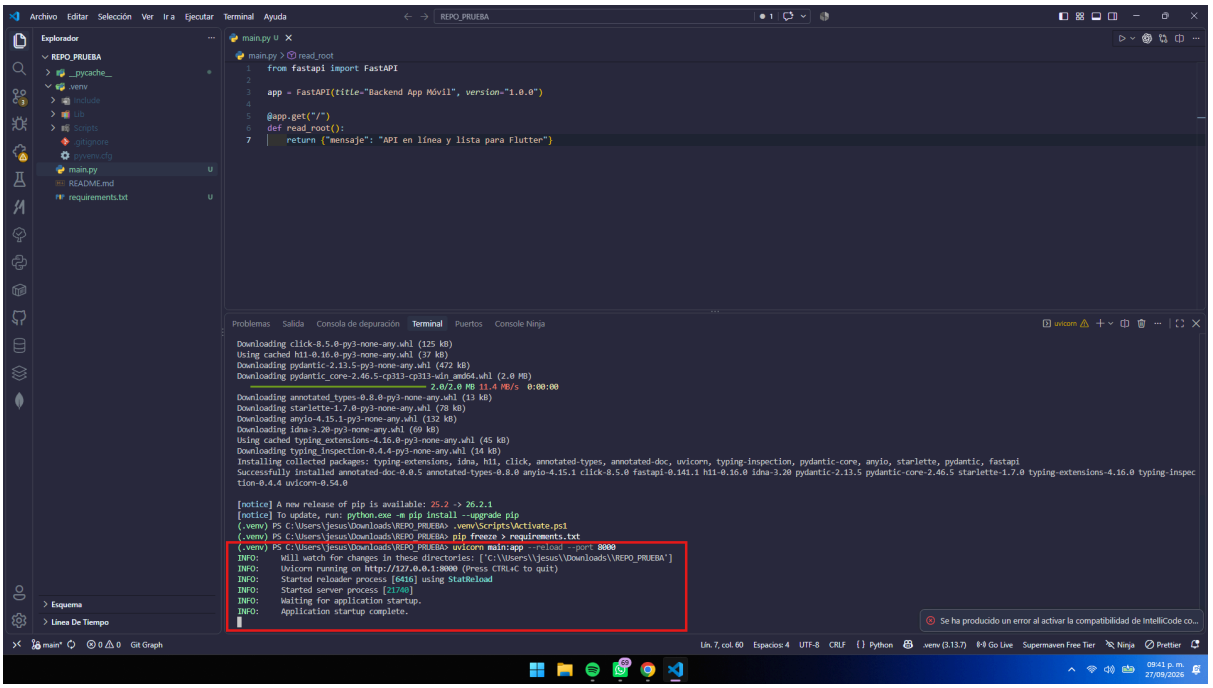


19.7 Ejecución del servidor

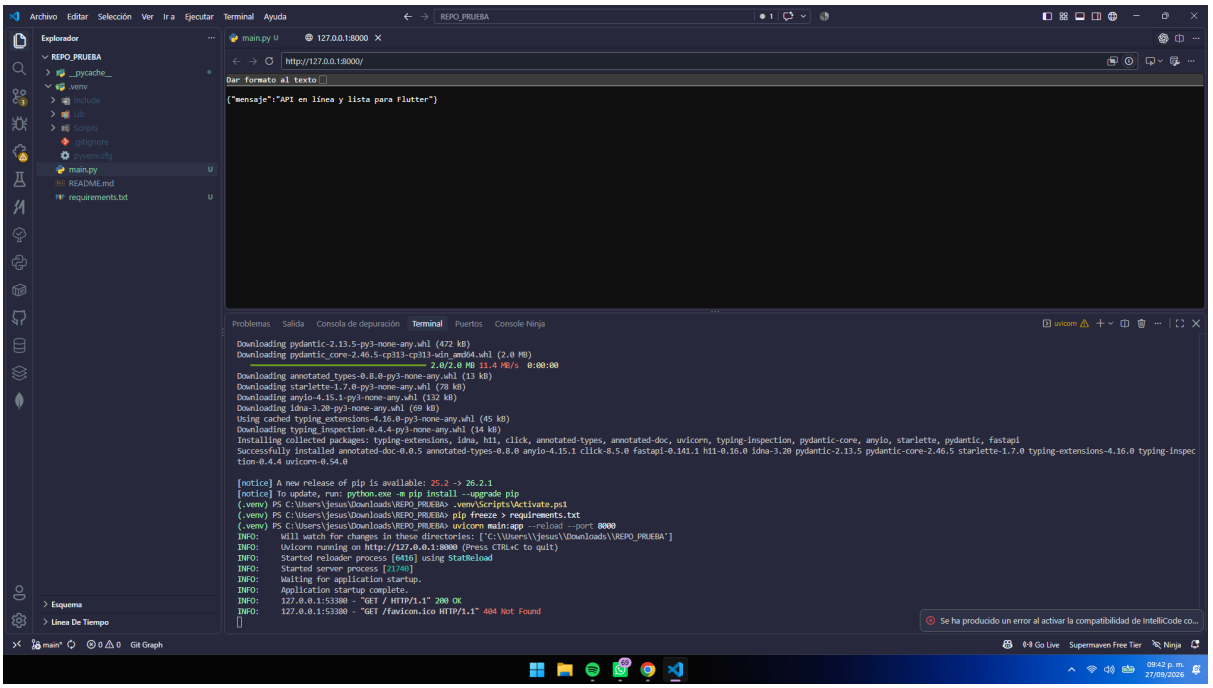
Fundamento técnico: El flag `--reload` habilita el *Hot Reload*, reiniciando el proceso del servidor cada vez que detecta cambios en el código fuente (indispensable para desarrollo local).

Procedimiento técnico:

uvicorn main:app --reload --port 8000



Validación: Abrir el navegador en http://127.0.0.1:8000 y comprobar la respuesta JSON.



20. Swagger y OpenAPI

20.1 ¿Qué es OpenAPI?

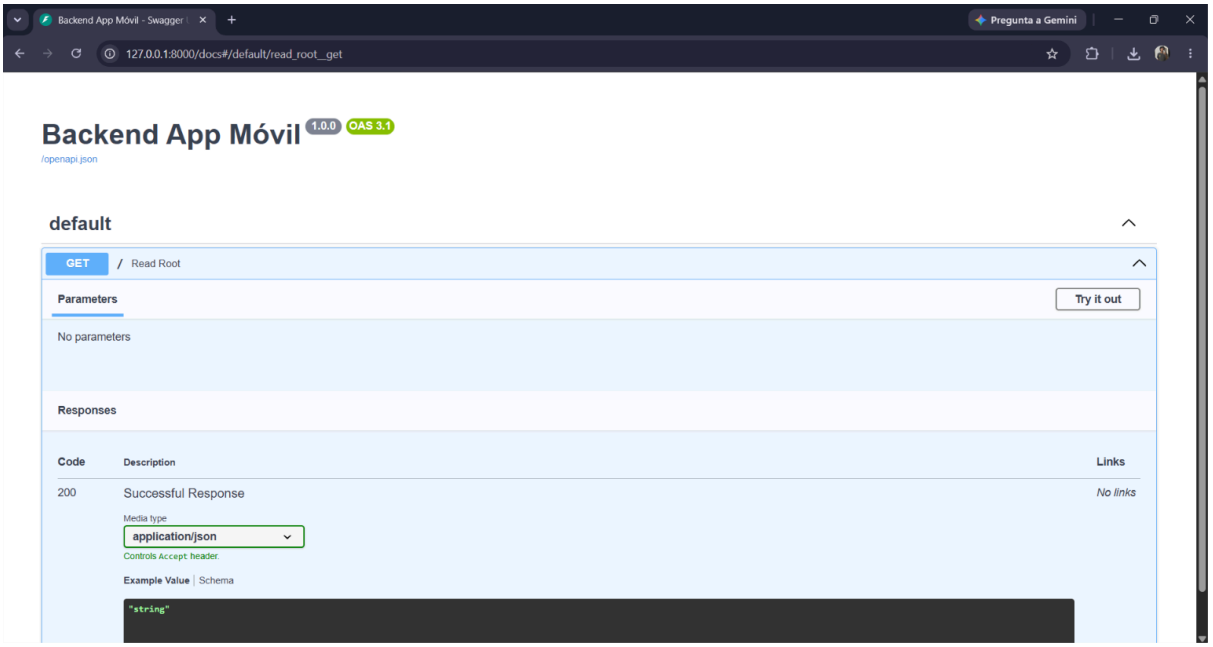
- **Explicación formativa:** Es una especificación estándar de la industria (independiente del lenguaje) para describir interfaces REST mediante un archivo JSON o YAML, detallando rutas, parámetros, códigos de estado y esquemas de datos.

20.2 ¿Qué es Swagger UI?

- **Explicación formativa:** Es la herramienta visual que interpreta la especificación OpenAPI y genera una interfaz gráfica interactiva en la web para consultar y probar la API sin herramientas adicionales.

20.3 Documentación automática en FastAPI

- **Explicación formativa:** FastAPI genera la especificación de manera nativa sin configuraciones externas basándose en los modelos y tipos de Python.
- **Prueba:** Navegar a http://127.0.0.1:8000/docs (Swagger UI) o a http://127.0.0.1:8000/redoc (ReDoc).



20.4 Prueba interactiva: Swagger vs. Postman

- **Cuadro comparativo formativo:**
  - **Swagger UI:** Ideal para documentación viva, pruebas rápidas de contrato y visualización inmediata para el desarrollador frontend.
  - **Postman:** Ideal para automatización de pruebas, cadenas de peticiones con variables dinámicas, mocks y pruebas de regresión en equipo.

21. Backend — Ruta B: Node.js + Express

21.1 Introducción a Node.js

- **Fundamento teórico:** Node.js es un entorno de ejecución (*runtime*) basado en el motor V8 de Chrome que permite ejecutar JavaScript en el servidor mediante un modelo asíncrono y no bloqueante impulsado por eventos (*Event Loop*). `npm` actúa como su gestor de paquetes.

21.2 Instalación de Node.js

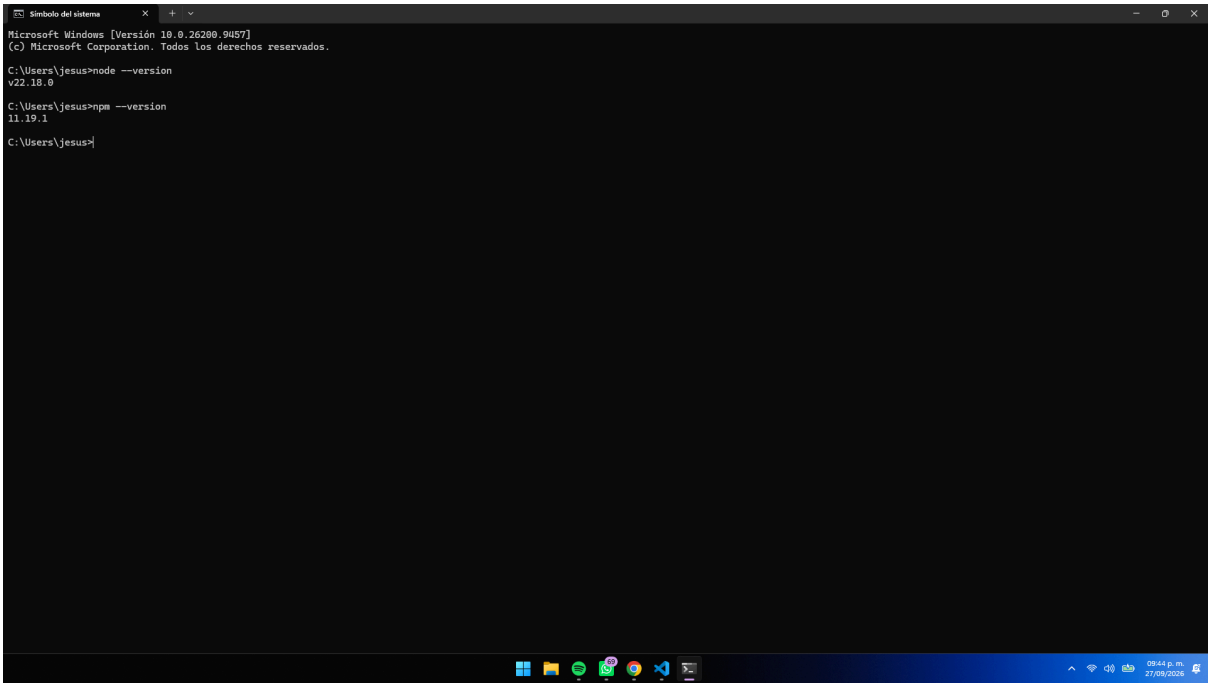
- **Instrucciones:** Descargar la versión **LTS** (*Long Term Support*) desde el sitio oficial `nodejs.org` para asegurar estabilidad y compatibilidad con librerías empresariales.

21.3 Verificación de la instalación

- **Comandos:**

  node --version

  npm --version

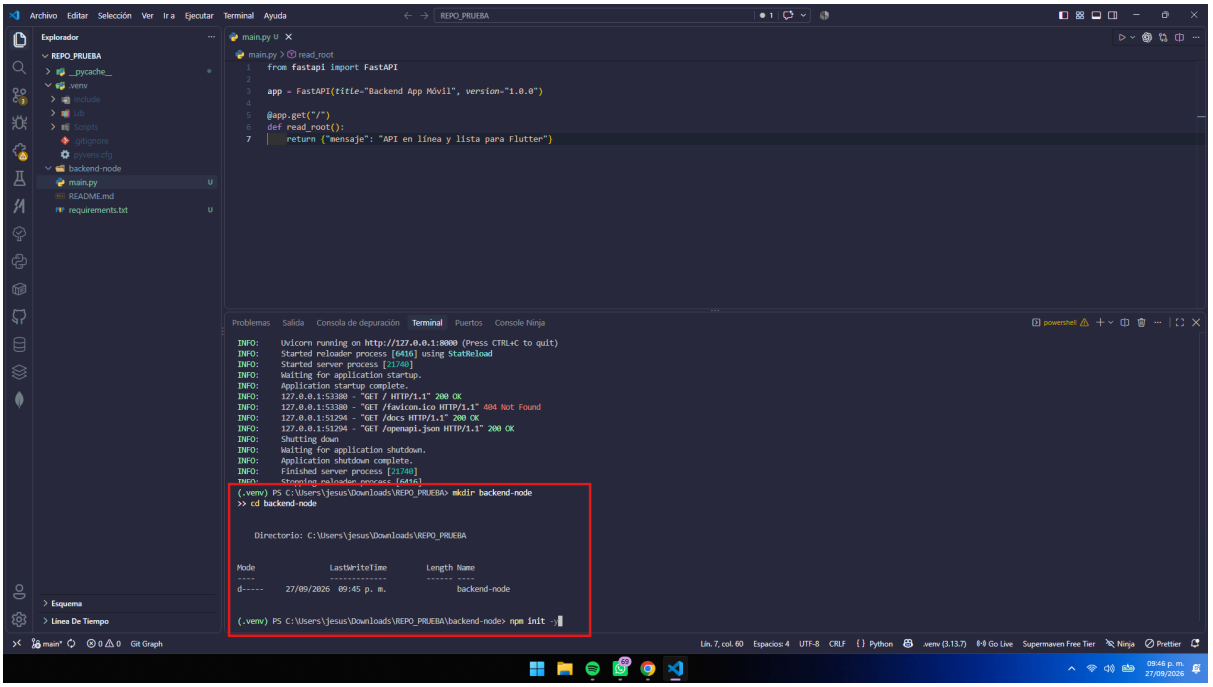


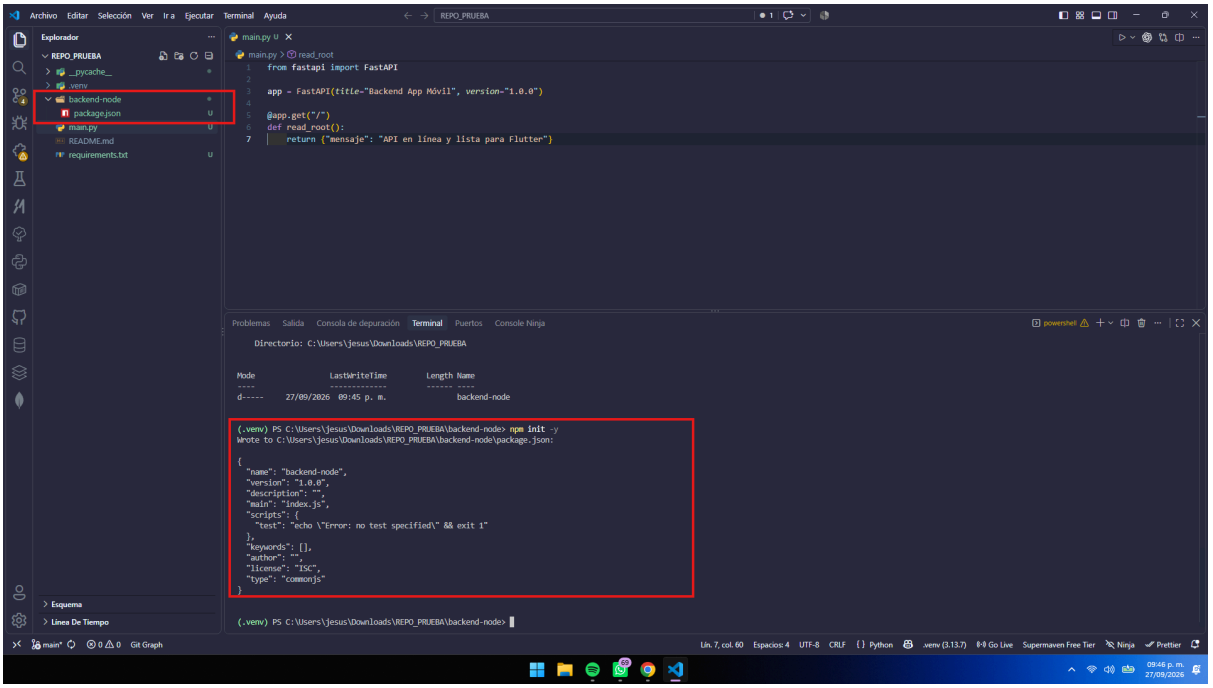
21.4 Inicialización del proyecto

- **Procedimiento:** Crear la carpeta del proyecto y generar el manifiesto:

  mkdir backend-node && cd backend-node

  npm init -y



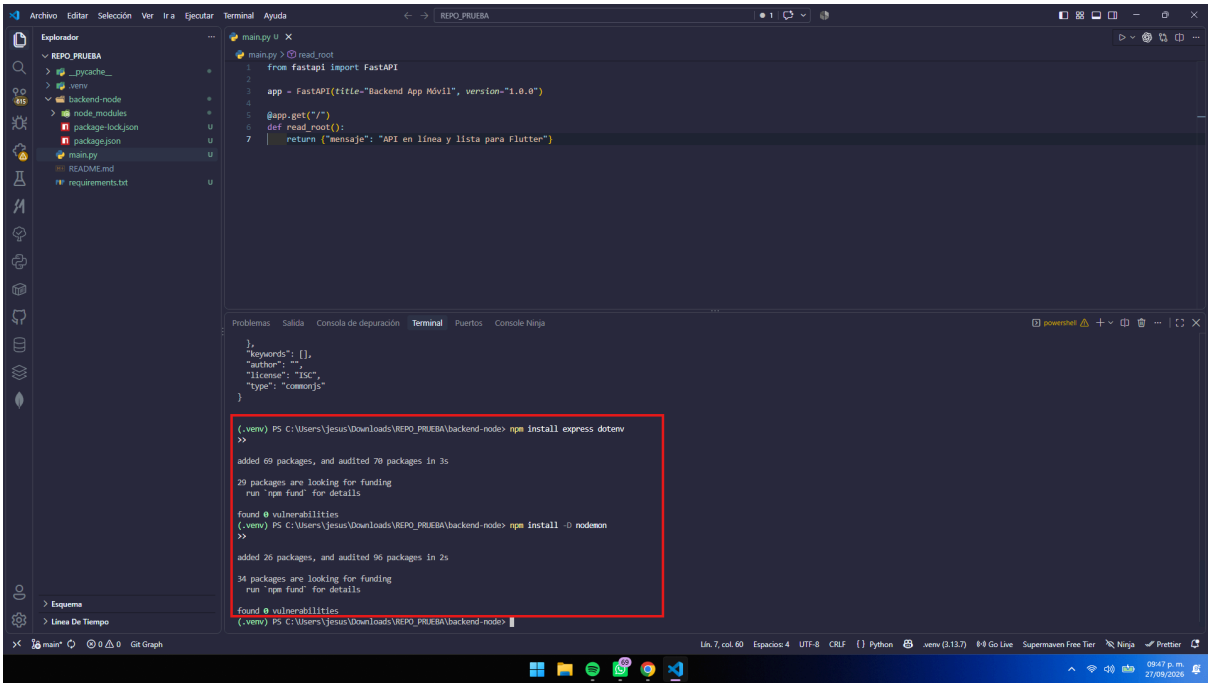


21.5 Instalación de Express

- **Procedimiento:**

npm install express dotenv

npm install -D nodemon



21.6 Arquitectura sugerida del proyecto

- **Estructura de carpetas recomendada:**

src/

├── config/ # Variables de entorno y conexiones a bases de datos

├── routes/ # Definición de rutas y endpoints

├── controllers/ # Controladores que manejan req y res

├── services/ # Lógica de negocio pura

├── models/ # Esquemas/modelos de datos (Mongoose o Sequelize)

├── middlewares/ # Interceptores de autenticación y validación

└── app.js # Entrada principal y montaje de middlewares

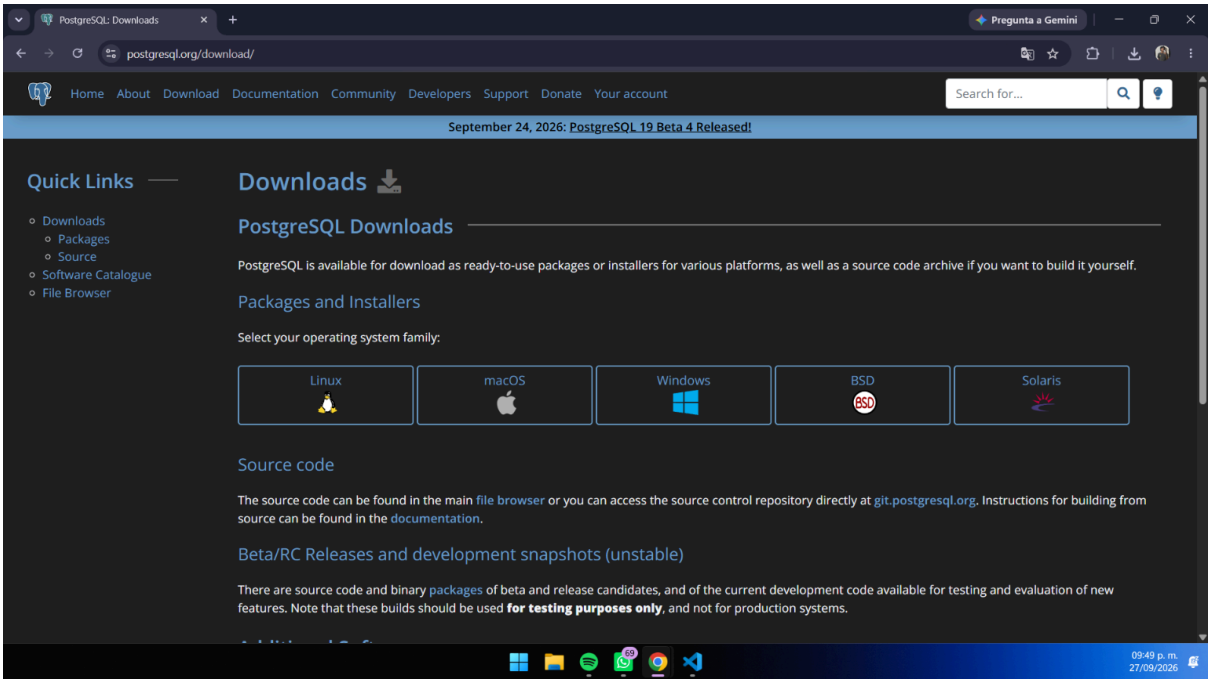
22. PostgreSQL

22.1 Bases de datos relacionales (RDBMS)

- **Fundamento teórico:** Se basan en el modelo relacional y garantizan propiedades **ACID** (Atomicidad, Consistencia, Aislamiento y Durabilidad). Son la opción estándar cuando la integridad de los datos, las transacciones financieras y las relaciones complejas entre entidades son críticas.

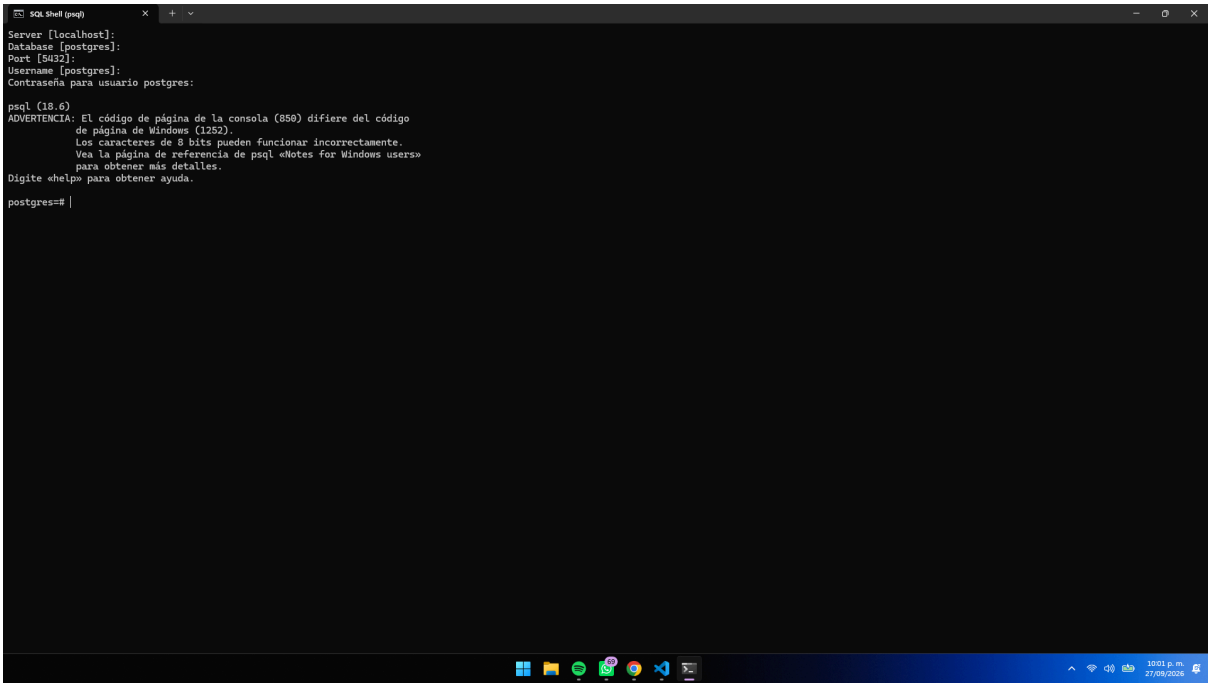
22.2 Instalación

- **Procedimiento:** Instalar PostgreSQL localmente mediante el instalador oficial o, preferentemente, mediante contenedor Docker (introduciendo la portabilidad).



22.3 y 22.4 Configuración, usuarios y roles

- **Concepto:** Por defecto se crea el superusuario `postgres`. En entornos profesionales se debe crear un usuario con privilegios mínimos y una contraseña segura asignada específicamente a la aplicación móvil.



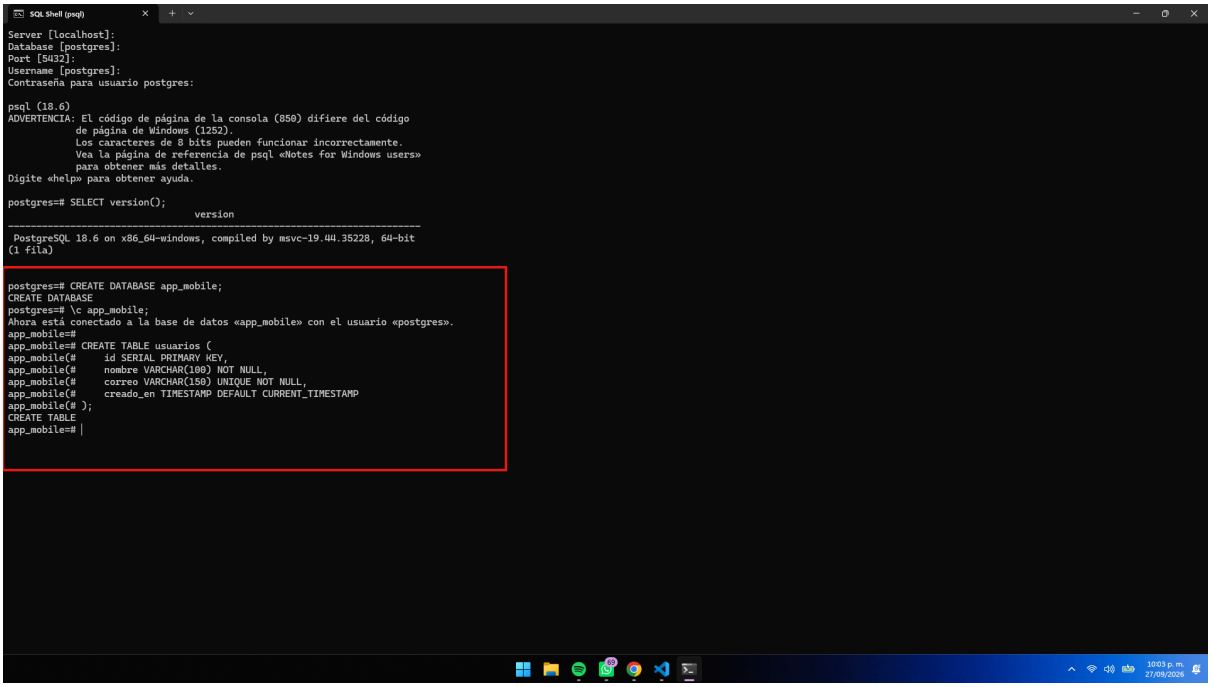
22.5 y 22.6 Creación de base de datos y tablas

- **Script SQL inicial:**

```
CREATE DATABASE app_mobile;

\c app_mobile;

CREATE TABLE usuarios (
    id SERIAL PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    correo VARCHAR(150) UNIQUE NOT NULL,
    creado_en TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```



22.7 Conexión desde el Backend

- **Flujo de conexión:** Explicar cómo la aplicación Flutter no se conecta directamente al puerto 5432 de PostgreSQL por razones de seguridad, sino que la API actúa como intermediaria utilizando un driver o un ORM (como SQLAlchemy en FastAPI o Prisma/TypeORM en Node.js).

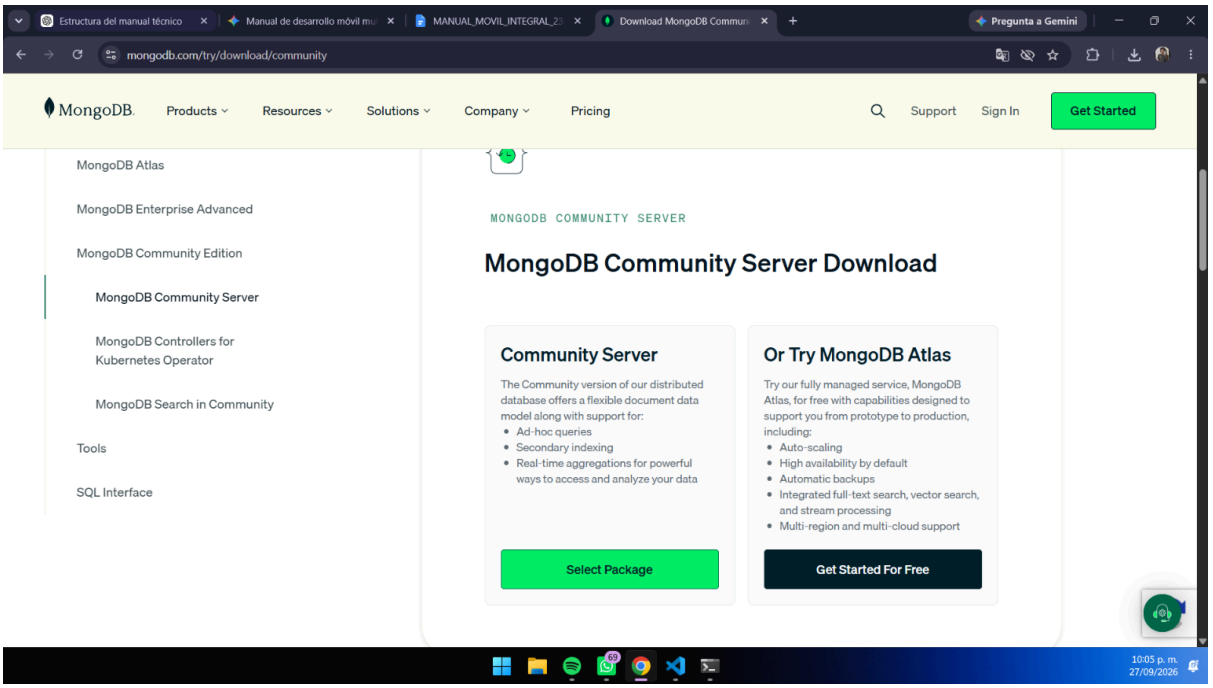
23. MongoDB

23.1 Introducción a NoSQL y modelo documental

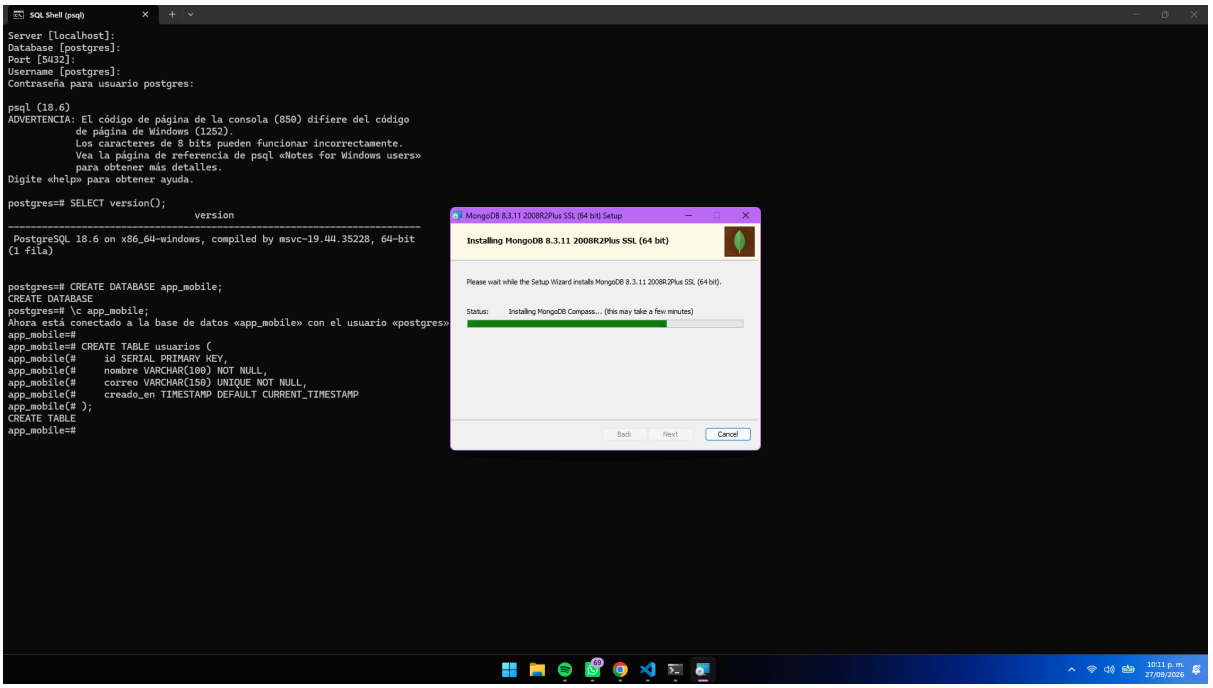
- **Fundamento teórico:** NoSQL abandona la rigidez de esquemas tabulares. MongoDB utiliza documentos con formato **BSON** (representación binaria de JSON), permitiendo estructuras flexibles donde cada documento puede variar sus campos según la necesidad.
- **Mapeo terminológico:**
  - Base de datos → Base de datos
  - Tabla → **Colección**
  - Fila / Registro → **Documento**
  - Columna → **Campo**
  - JOIN → **Embedding** (embebido) o $lookup

23.2 Instalación y 23.3 MongoDB Compass

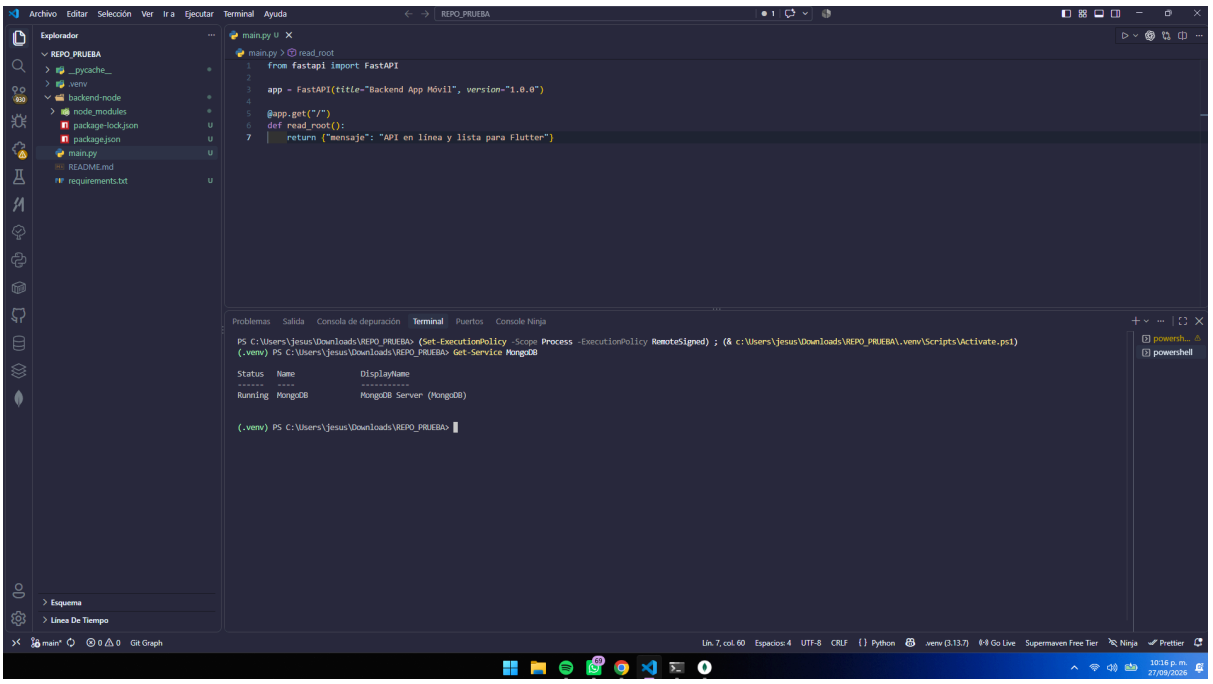
- **Instalación:** Levantar el servicio MongoDB Community Server o contenedor Docker en el puerto 27017.

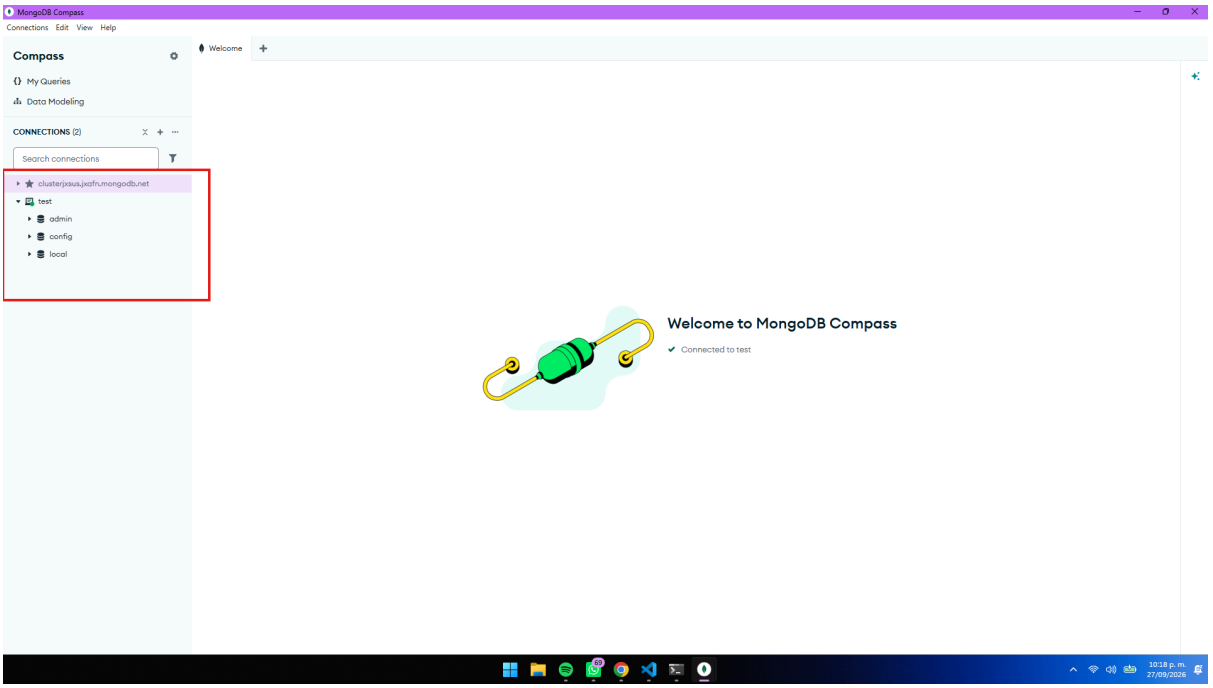


- **Compass:** Instalar MongoDB Compass como la interfaz gráfica (GUI) oficial para visualizar colecciones, crear índices y analizar consultas en tiempo real.



UNIVERSIDAD TECNOLÓGICA DE XICOTEPEC DE JUÁREZ





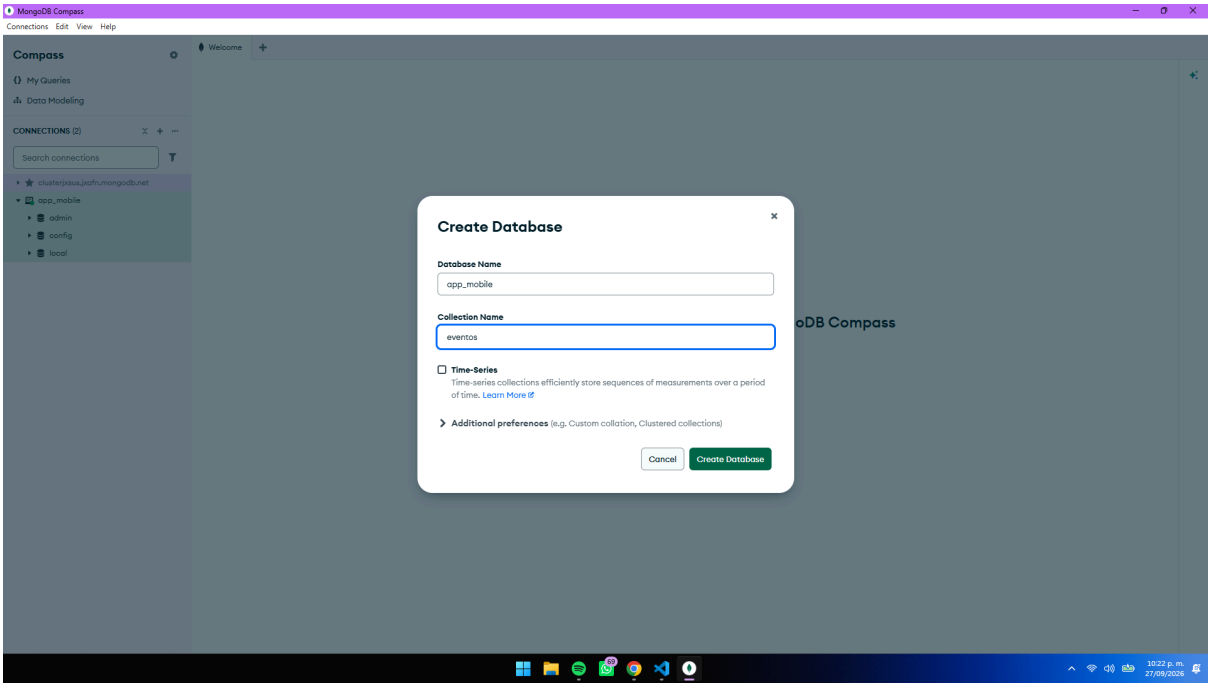
23.4 Creación de la Base de Datos (`app_mobile`) y 23.5 Colección (`eventos`)

A. Fundamento teórico-formativo

- **Base de datos:** Contenedor lógico de nivel superior donde se almacenan colecciones relacionadas con el proyecto.
- **Colección:** Es el equivalente a una tabla en el mundo relacional. Agrupa documentos que comparten un propósito común (en este caso, registrar auditoría, sesiones y telemetría de la app móvil).
- **Diferencia clave con SQL:** En PostgreSQL primero definiste la base y luego diseñaste columnas y tipos fijos en una tabla. En MongoDB la colección se crea sin declarar qué campos contendrá; los esquemas son flexibles y dinámicos.

B. Procedimiento técnico en MongoDB Compass

1. Abre **MongoDB Compass** y asegúrate de estar conectado a `mongodb://localhost:27017`.
2. En la barra lateral izquierda (o en la pestaña central *Databases*), haz clic en el botón verde con el signo más: **`+ Create database`**.
3. Se abrirá una ventana emergente solicitando dos campos obligatorios:
    - **Database Name:** Escribe `app_mobile`.
    - **Collection Name:** Escribe `eventos`.
4. Deja las opciones avanzadas (*Capped Collection*, *Clustered Index*, etc.) sin marcar por defecto.
5. Haz clic en el botón verde **Create Database**.
6. En el panel izquierdo verás aparecer `app_mobile` y, al desplegarla, la colección `eventos`.



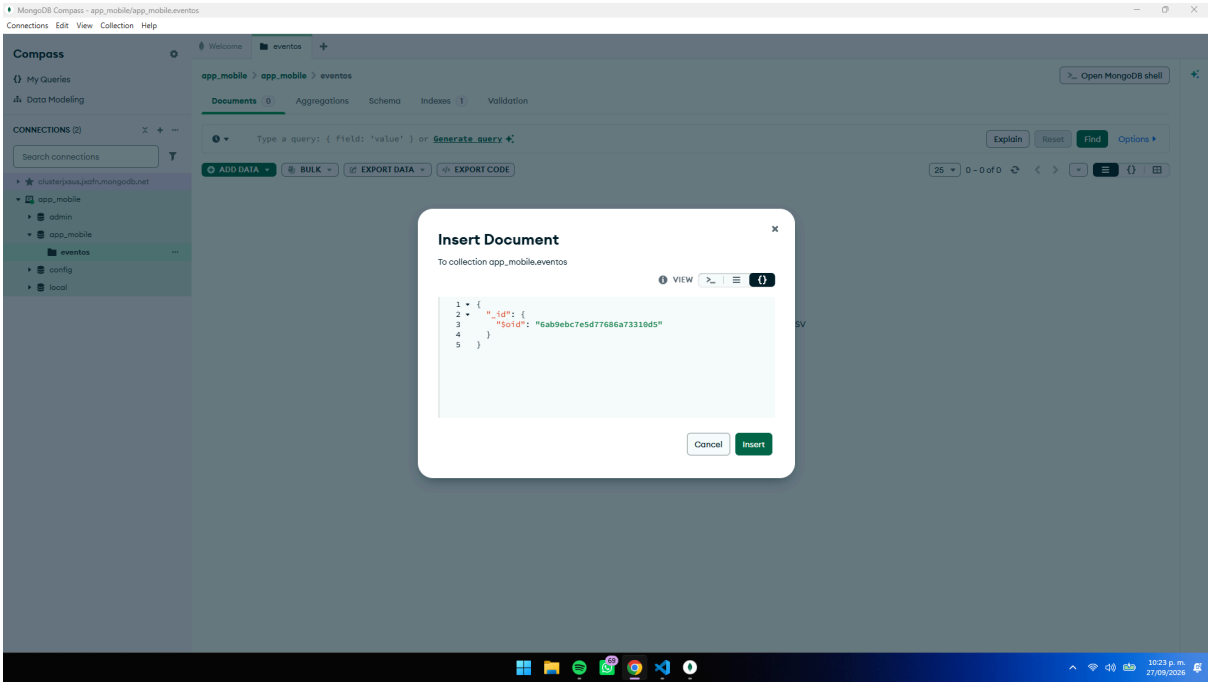
23.6 Inserción de un Documento JSON

A. Fundamento teórico-formativo

- **Documento (BSON/JSON):** Es el registro individual dentro de la colección. Utiliza pares `clave: valor`.
- **Identificador único (`_id`):** Si no defines un campo `_id`, MongoDB genera automáticamente un `ObjectId` único de 12 bytes compuesto por marcas de tiempo, identificador de máquina y contador incremental.
- **Caso de uso móvil:** Las colecciones como `eventos` o `logs` son ideales para MongoDB porque cada tipo de evento en el teléfono puede enviar datos diferentes (errores, clics, tiempos de carga) sin obligarnos a alterar una estructura rígida de base de datos.

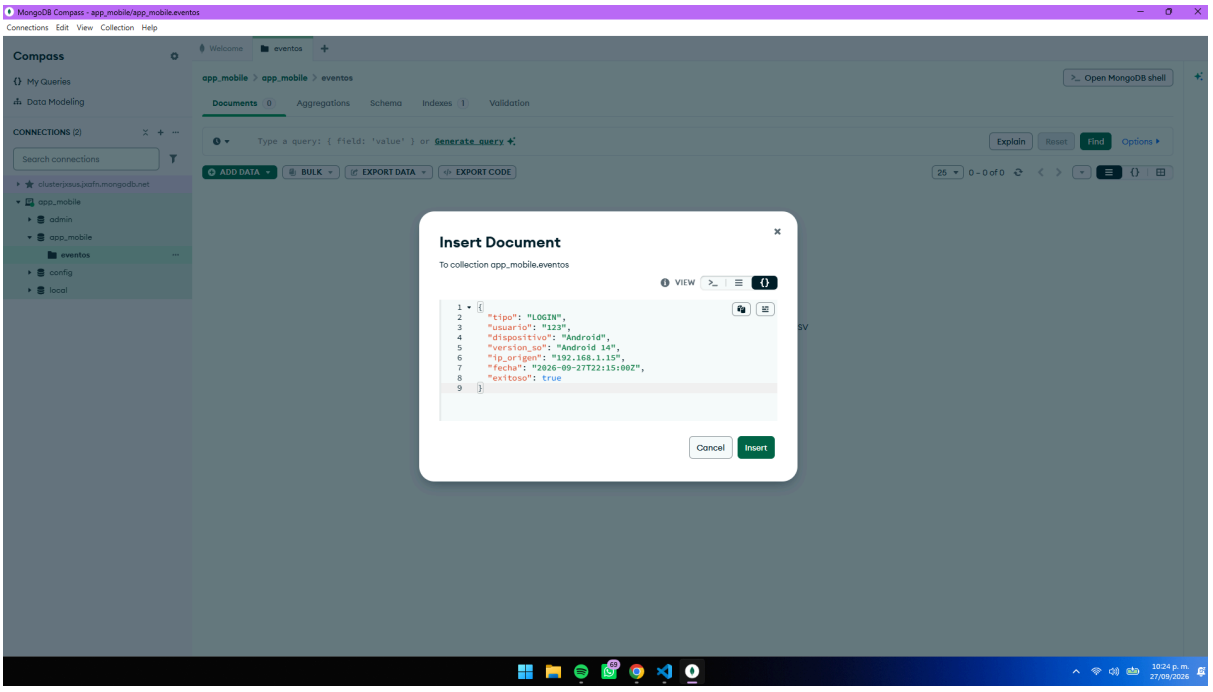
B. Procedimiento técnico en MongoDB Compass

1. En la lista de bases de datos, haz clic sobre `app_mobile` y luego entra a la colección **`eventos`**.
2. En la barra superior de la colección, haz clic en el botón verde desplegable **Add Data** y selecciona **Insert document**.
3. Cambia la vista a modo código haciendo clic en el icono de llaves `{}` (esquina superior derecha del cuadro modal).



4. Borra el contenido por defecto (o respeta el `_id` si ya lo sugiere) y pega la siguiente estructura:

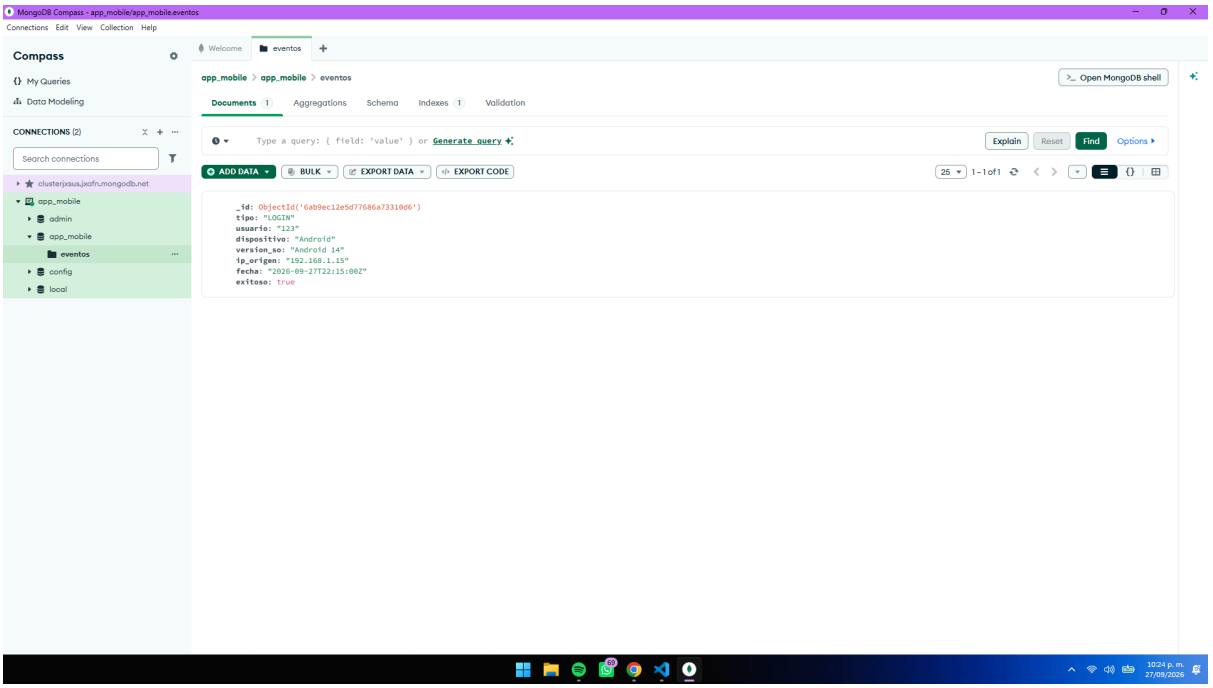

```
{
  "tipo": "LOGIN",
  "usuario": "123",
  "dispositivo": "Android",
  "version_so": "Android 14",
  "ip_origen": "192.168.1.15",
  "fecha": "2026-09-27T22:15:00Z",
  "exitoso": true
}
```

Haz clic en el botón verde **Insert**.

El documento se renderizará en pantalla con su identificador autogenerado:

"_id": ObjectId("...")



24. ¿Cuándo usar PostgreSQL y MongoDB? (Arquitectura Híbrida)

- **Fundamento formativo:** Ninguna tecnología es una solución universal. En aplicaciones móviles modernas se utiliza con frecuencia una **persistencia políglota**:
    - **PostgreSQL:** Identidad del usuario, pagos, suscripciones, inventario y relaciones 1:N o N:M críticas.
    - **MongoDB:** Historial de notificaciones push, registro de actividad del dispositivo (*analytics* locales), borradores no estructurados y catálogos con atributos dinámicos.

25. Keycloak (Gestión de Identidad y Acceso - IAM)

25.1 Conceptos fundamentales de identidad

- **Diferenciación clave:**
    - **Autenticación (AuthN):** Validar *quién es* el usuario (ej. usuario y contraseña, biometría).
    - **Autorización (AuthZ):** Determinar *qué permisos tiene* el usuario una vez autenticado (roles, políticas).

25.2 OAuth 2.0 y 25.3 OpenID Connect (OIDC)

- **Explicación formativa:**
  - **OAuth 2.0:** Protocolo de autorización que permite a una aplicación acceder a recursos en nombre de un usuario sin conocer sus credenciales mediante tokens de acceso (*Access Token*).
  - **OpenID Connect (OIDC):** Capa de identidad construida sobre OAuth 2.0 que añade autenticación federada y entrega un *ID Token* con el perfil del usuario.

25.4 JSON Web Tokens (JWT)

- **Estructura del token:** Desglosar la anatomía del JWT (`Header.Payload.Signature`):
  - **Header:** Algoritmo de firma y tipo de token.
  - **Payload (Claims):** Datos del usuario (`sub`, `email`, roles asignados, fecha de expiración `exp`).
  - **Signature:** Firma criptográfica calculada con la llave privada de Keycloak para evitar manipulación en tránsito.

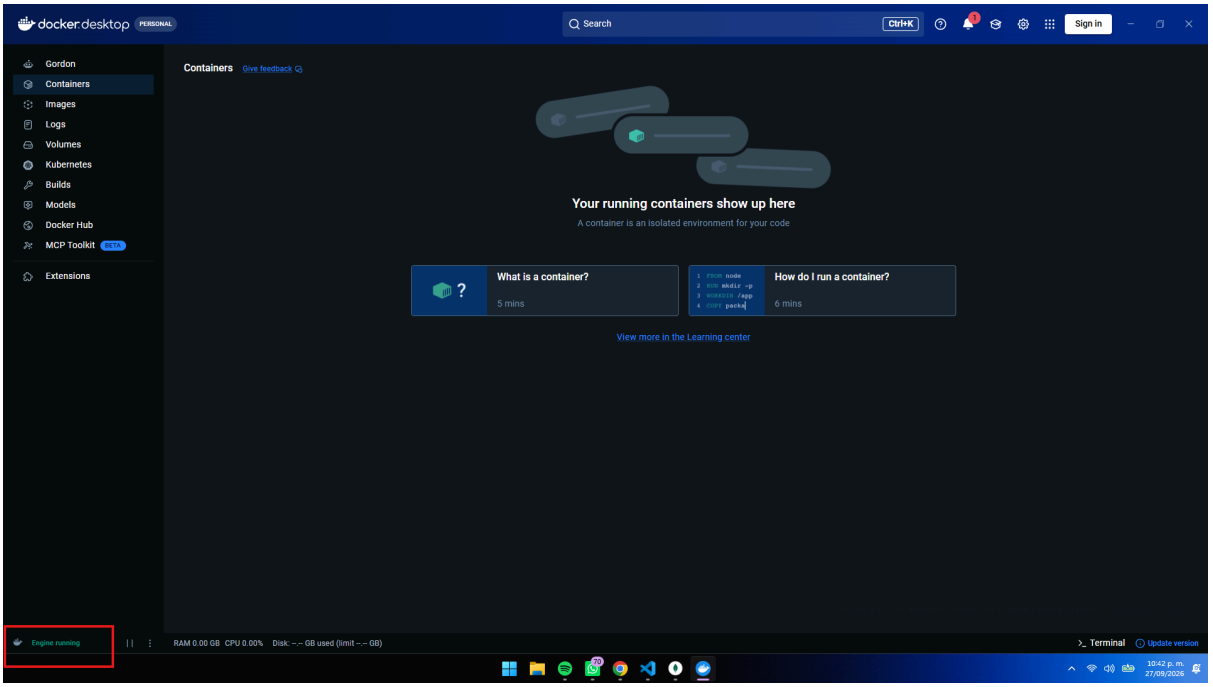
25.5 Despliegue de Keycloak

A. Fundamento teórico-formativo

- **¿Por qué desplegarlo con Docker?**
  Keycloak es un servidor basado en Java (Quarkus) que requiere configuraciones de runtime, dependencias del sistema y bases de datos dedicadas. Instalarlo manualmente directo en Windows es propenso a fallos por versiones de Java o variables de entorno. Desplegarlo en un contenedor Docker garantiza un entorno aislado, reproducible e idéntico para todos los estudiantes.
- **Modo desarrollo (`start-dev`):**
  Por defecto, Keycloak exige certificados HTTPS/TLS válidos para iniciar. Al pasarle la instrucción `start-dev`, Keycloak relaja estos requisitos de producción, permitiendo pruebas locales mediante HTTP sin certificados autofirmados complejos y utilizando una base de datos interna en memoria (H2).

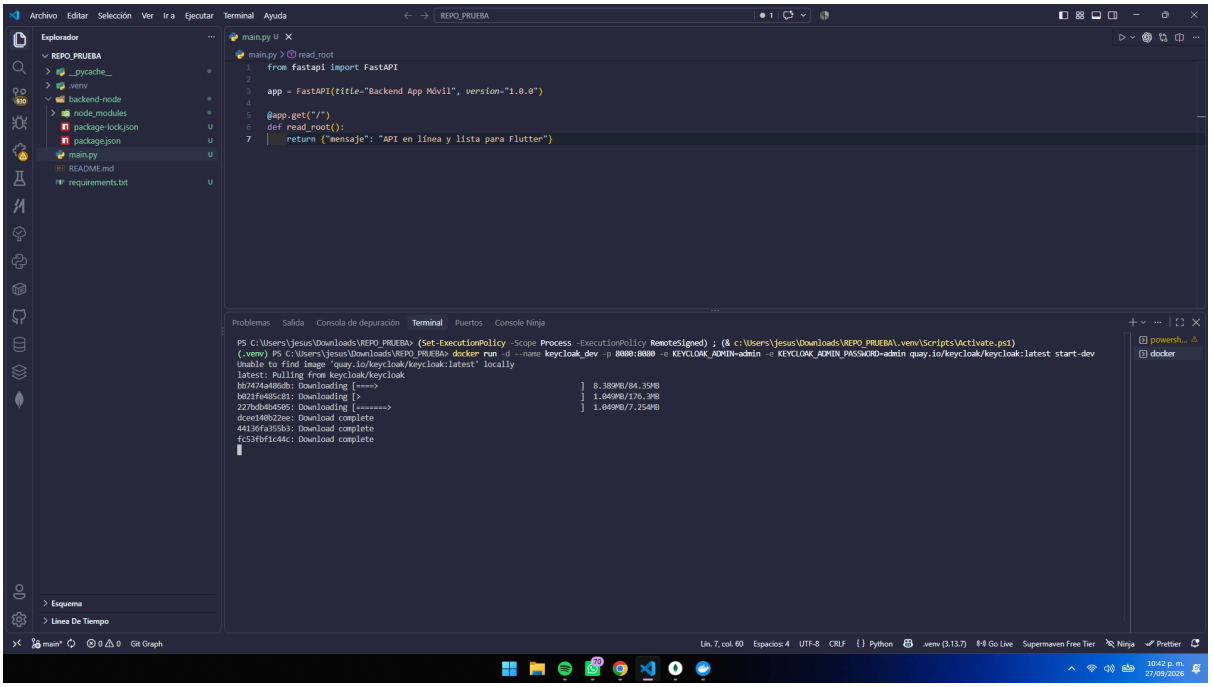
B. Procedimiento técnico paso a paso

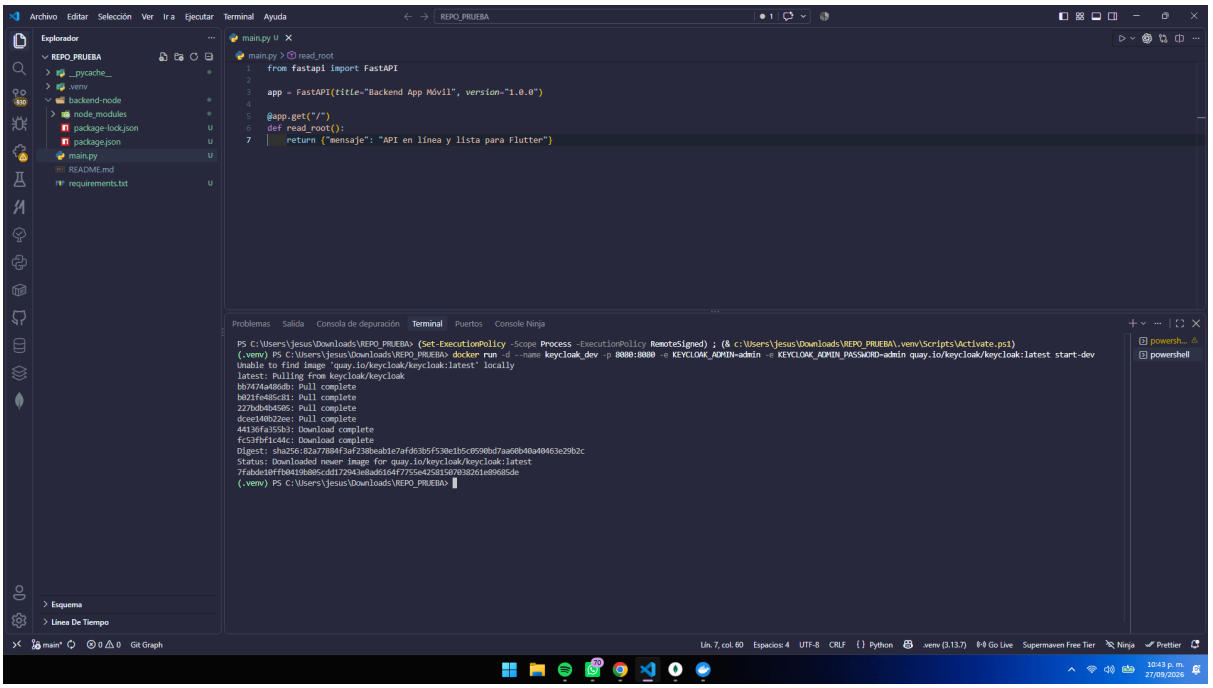
1. **Asegurar que Docker esté activo:**
   Abre la aplicación **Docker Desktop** en Windows y espera a que el icono en la esquina inferior izquierda esté en color verde (*Engine running*).



2. **Ejecutar el contenedor desde la terminal:**
   Abre una terminal de PowerShell (puede ser la de VS Code o la consola de Windows) y ejecuta el siguiente comando en una sola línea:

```
docker run -d --name keycloak_dev -p 8080:8080 -e KEYCLOAK_ADMIN=admin -e KEYCLOAK_ADMIN_PASSWORD=admin quay.io/keycloak/keycloak:latest start-dev
```



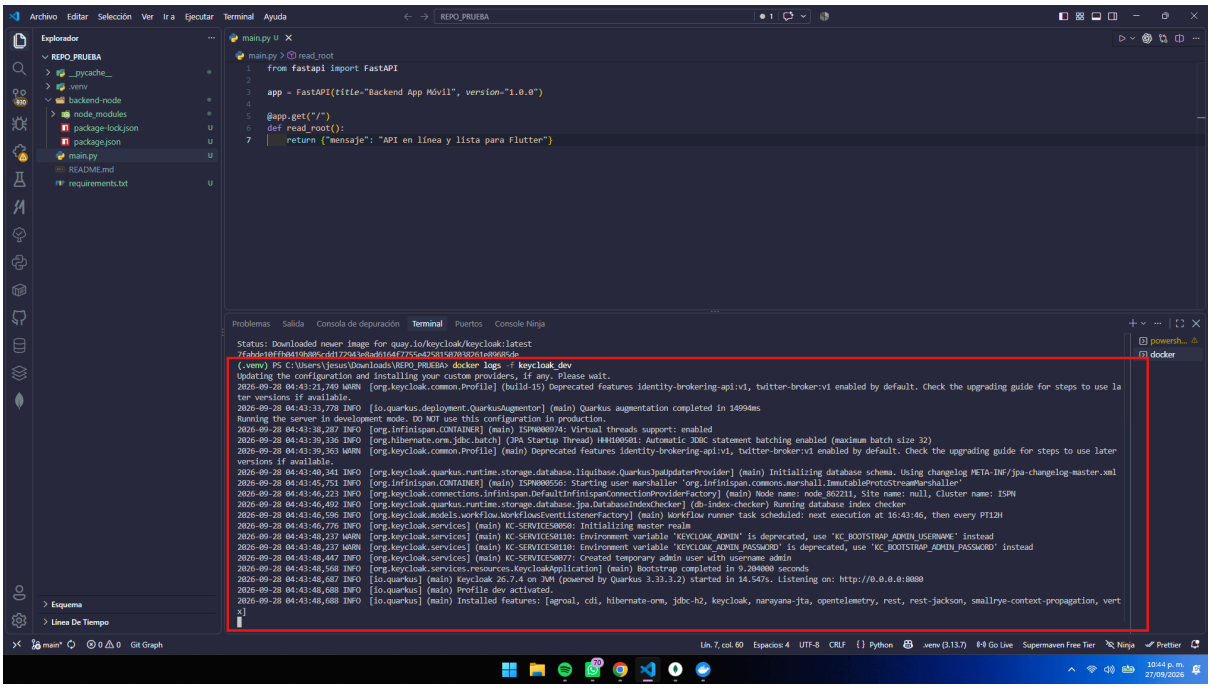


Explicación pedagógica de los parámetros para el manual:

- `-d` (*detached*): Ejecuta el contenedor en segundo plano liberando la terminal.
- `--name keycloak_dev`: Asigna un nombre claro y legible al contenedor.
- `-p 8080:8080`: Mapea el puerto `8080` de tu máquina física al puerto `8080` interno del contenedor.
- `-e KEYCLOAK_ADMIN=admin`: Define la variable de entorno para el usuario maestro inicial.
- `-e KEYCLOAK_ADMIN_PASSWORD=admin`: Establece la contraseña maestra inicial (en producción debe ser una clave robusta).
- `quay.io/keycloak/keycloak:latest`: Descarga la imagen oficial optimizada.
- `start-dev`: Inicia el servidor en modo desarrollo.

Verificación de logs de inicio:
Espera unos 20 a 40 segundos mientras el servidor levanta los servicios internos. Para verificar el arranque, escribe:

docker logs -f keycloak_dev



Verás pasar varias líneas de arranque. Cuando aparezca el mensaje:

Listening on: http://0.0.0.0:8080

Running the server in development mode.

presiona `Ctrl + C` para salir de la vista de logs.

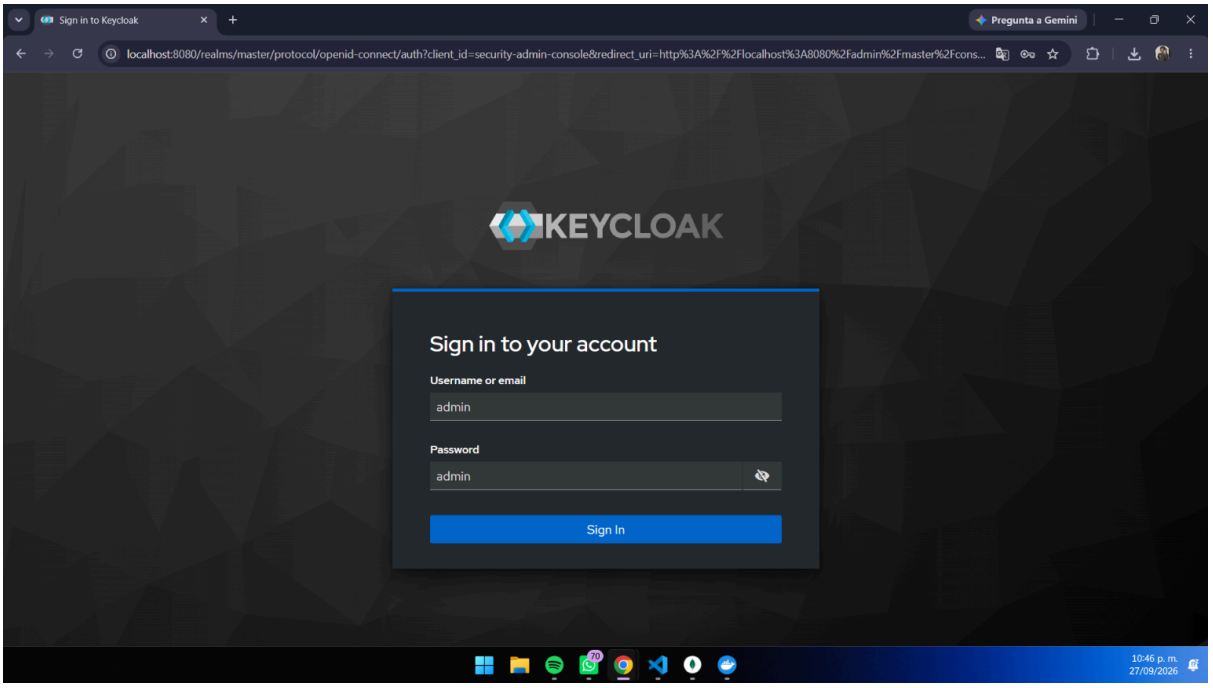
Acceso a la interfaz gráfica:

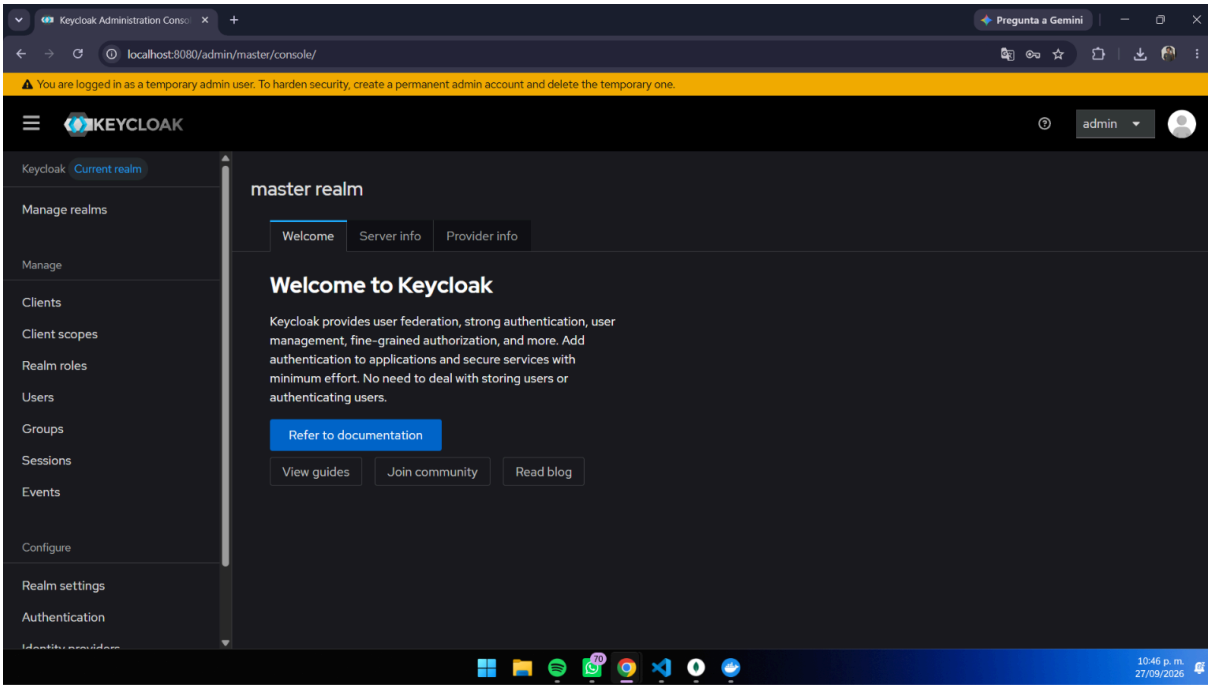
Abre tu navegador web e ingresa a:

http://localhost:8080

Haz clic en **Administration Console** e inicia sesión con las credenciales que pasamos en el comando:

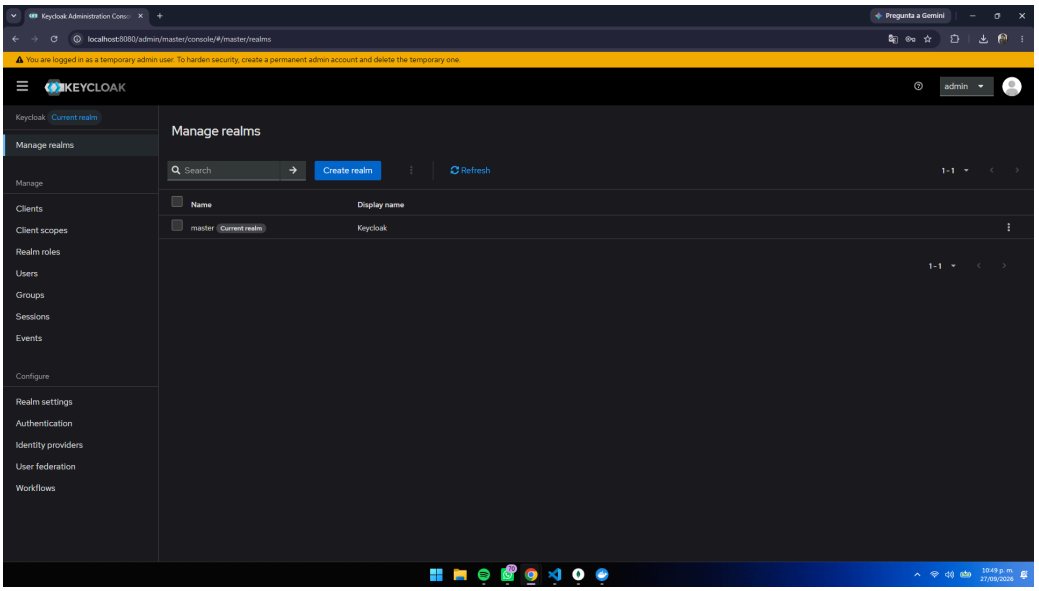
- **Username:** `admin`
- **Password:** `admin`



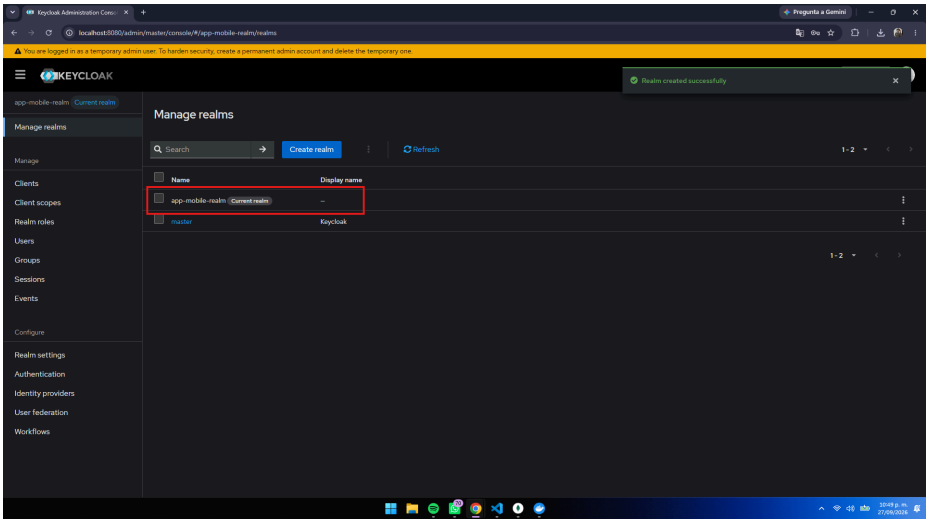


25.6 Creación del Realm

- **Fundamento teórico:** Un **Realm** (reino o dominio) es un espacio aislado de seguridad dentro de Keycloak que gestiona su propio conjunto de usuarios, credenciales, roles y aplicaciones cliente. Nunca debe usarse el realm `master` para las aplicaciones de desarrollo o producción, ya que este está reservado exclusivamente para la administración interna de la plataforma.
- **Procedimiento técnico:**
    1. En la esquina superior izquierda del panel de administración, haz clic en el menú desplegable que dice **master**.
    2. Haz clic en el botón **Create Realm**.

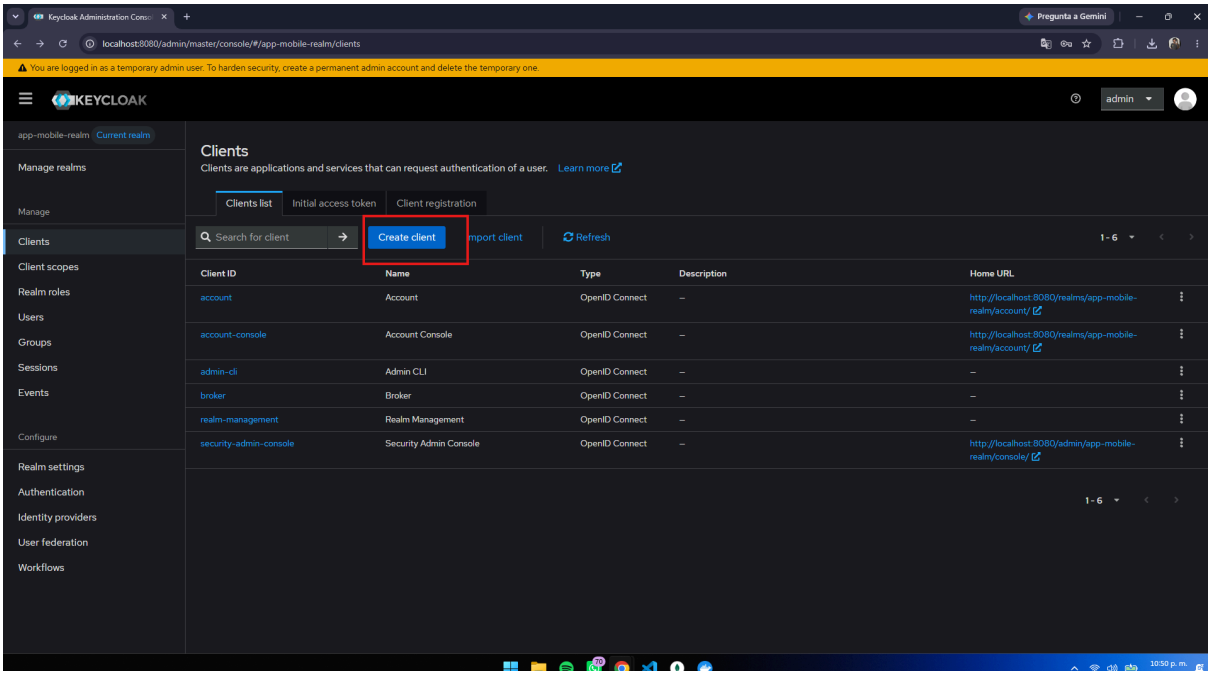


    3. En el campo **Realm name**, escribe: `app-mobile-realm`.
    4. Asegúrate de que el interruptor **Enabled** esté activado (`ON`).
    5. Haz clic en **Create**.
- **Validación:** El selector superior izquierdo ahora debe mostrar activo el nombre `app-mobile-realm`.

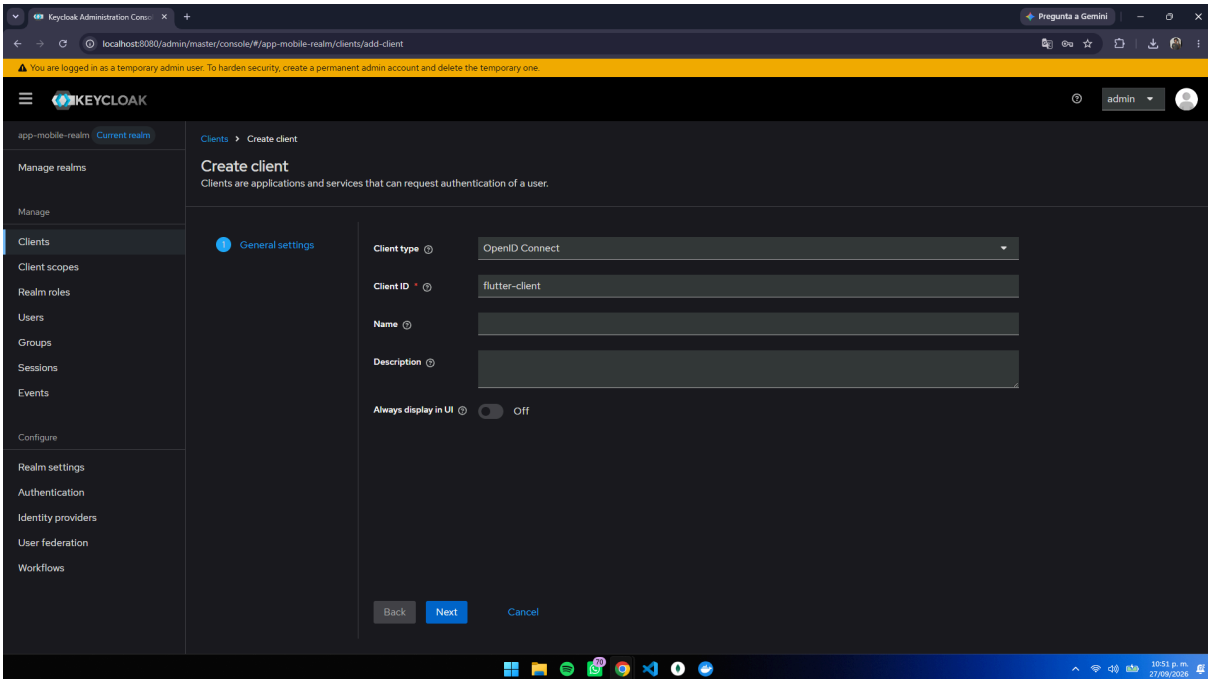


25.7 Creación y configuración de Clients

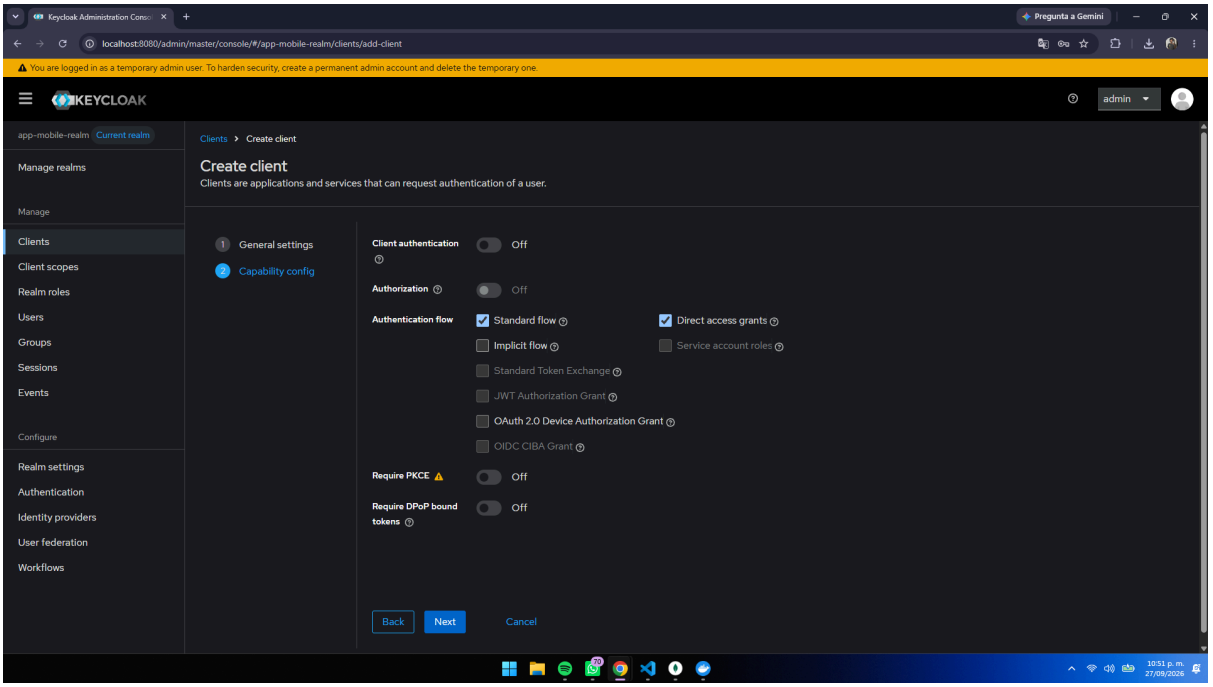
- **Fundamento teórico:** Un **Client** representa una entidad o aplicación que solicita a Keycloak autenticar a un usuario o verificar un token. En una arquitectura móvil se configuran comúnmente dos tipos:
    1.
    2. **Cliente Público (Frontend Flutter):** No puede mantener en secreto una contraseña o clave de cliente (*client secret*) porque el código binario de la aplicación móvil puede ser descompilado. Usa flujos basados en PKCE (*Proof Key for Code Exchange*).
    3. **Cliente Bearer-Only / Confidencial (Backend API):** Valida los tokens JWT que llegan desde Flutter y puede mantener credenciales seguras.
- **Procedimiento técnico (Cliente Flutter):**
    1. En el menú lateral izquierdo, selecciona **Clients** y luego haz clic en **Create client**.



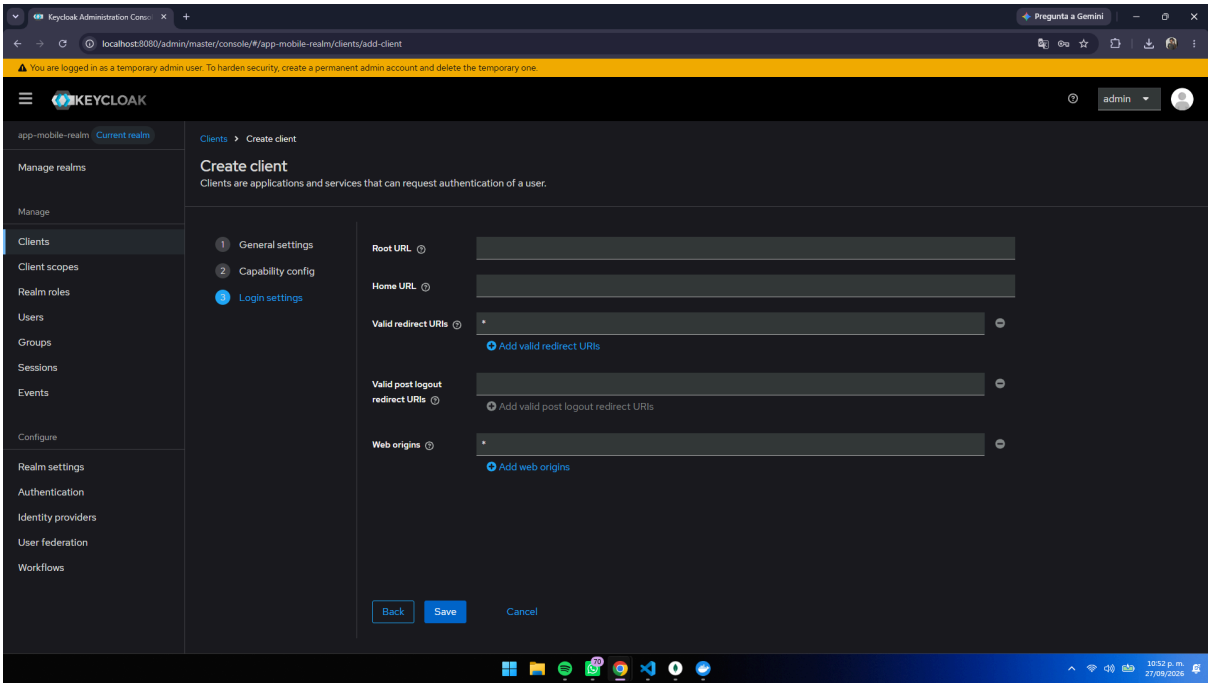
2. **Paso 1 (General Settings):**
    - **Client type:** `OpenID Connect`.
    - **Client ID:** `flutter-client`.
    - Haz clic en **Next**.

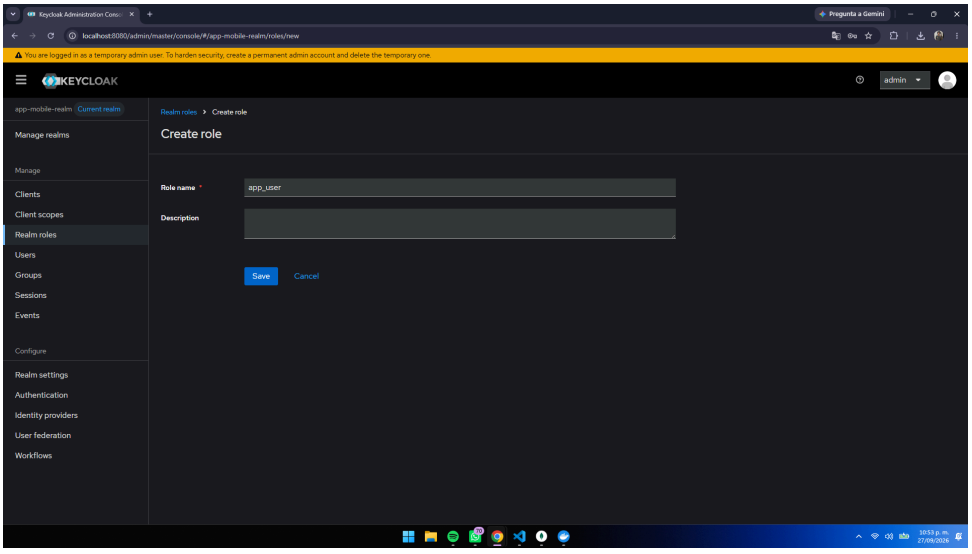


3. **Paso 2 (Capability config):**
    - **Client authentication:** Desactivado (OFF) para que opere como cliente público.
    - **Authentication flow:** Deja marcadas las casillas **Standard flow** y **Direct access grants**.
    - Haz clic en **Next**.



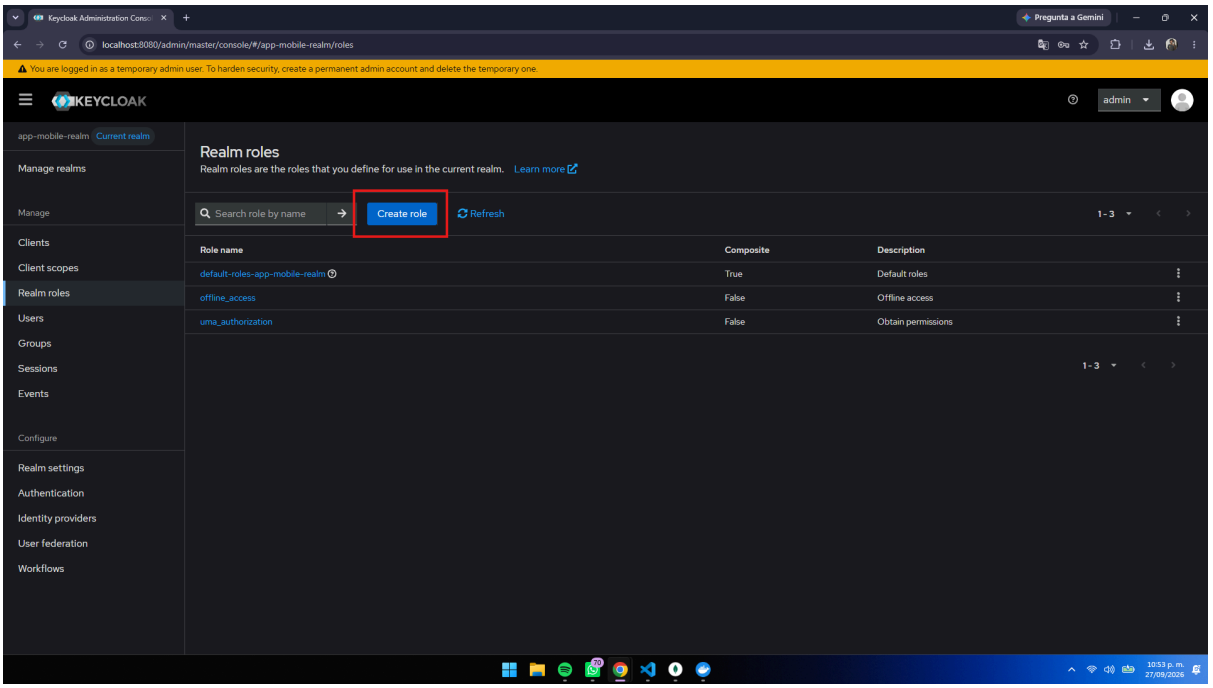
4. **Paso 3 (Login settings):**
    - **Valid redirect URIs:** Escribe * (para pruebas locales o esquemas personalizados de Flutter como `myapp://login-callback`).
    - **Web origins:** Escribe *.
5. Haz clic en **Save**.





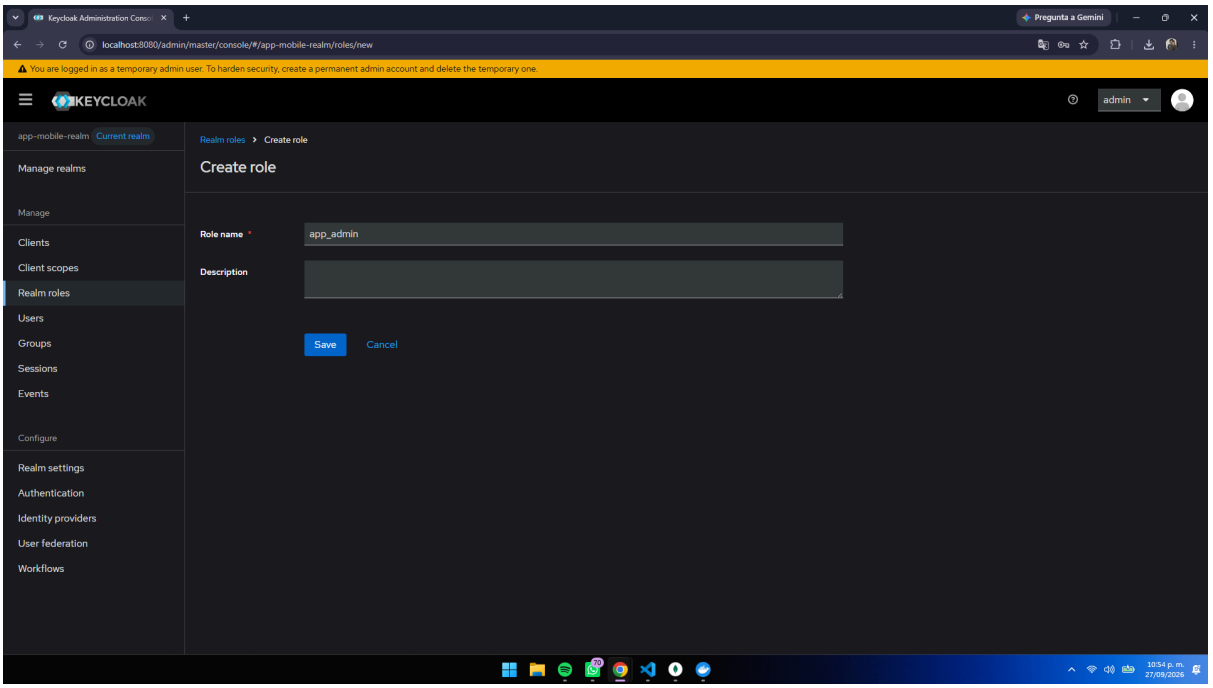
25.8 Definición de Roles

- **Fundamento teórico:** Los **Roles** permiten implementar el modelo RBAC (*Role-Based Access Control*). En lugar de otorgar permisos individuales a cada cuenta, se asignan permisos a roles globales o específicos del cliente y luego se vinculan los usuarios a ellos.
- **Procedimiento técnico:**
    1. En el menú lateral izquierdo, ve a la sección **Realm roles**.
    2. Haz clic en **Create role**.



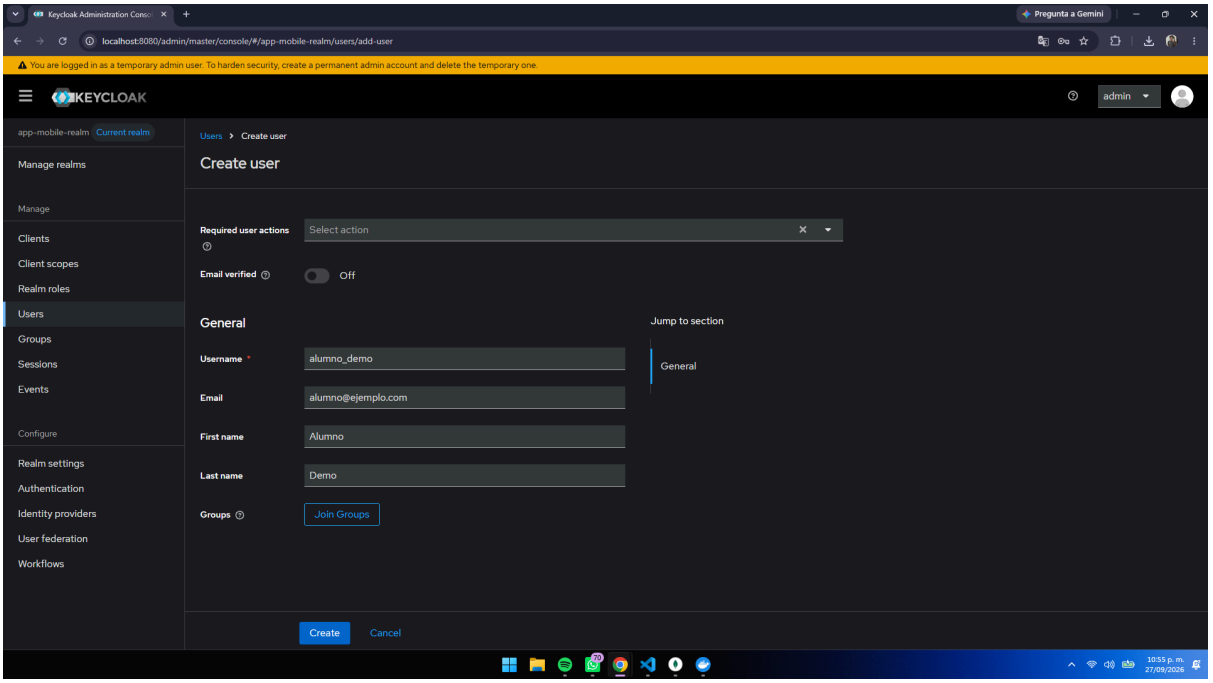
3. En **Role name**, escribe: `app_user` (rol estándar de usuario final).
4. (Opcional) En **Description**, agrega: *Acceso a los endpoints estándar de la app móvil*.
5. Haz clic en **Save**.

6. Repite el proceso para crear el rol de administrador: `app_admin`.

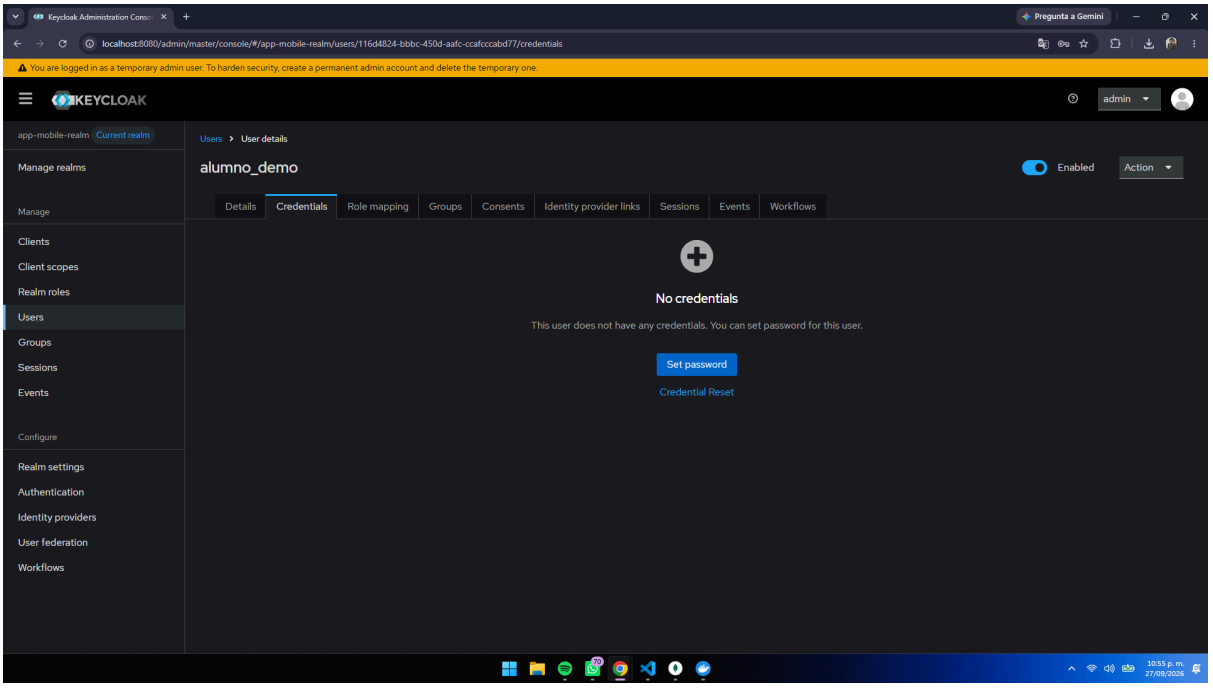


25.9 Creación de Usuarios y Asignación de Credenciales

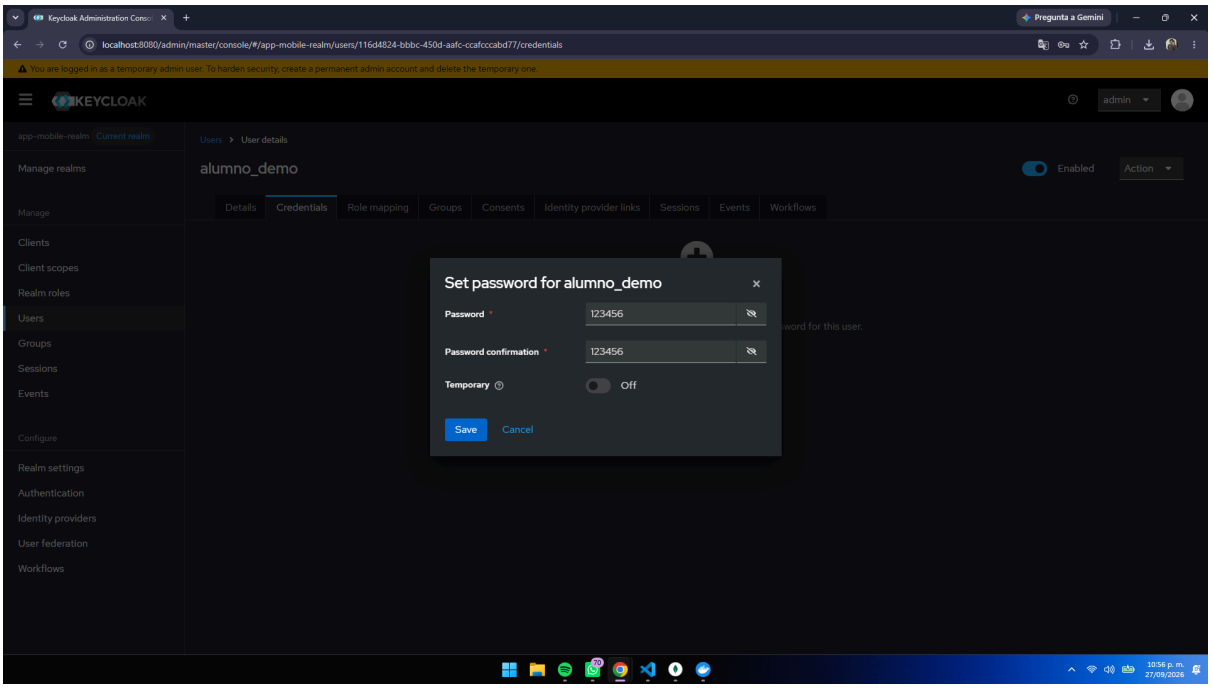
- **Fundamento teórico:** Los usuarios son las identidades finales almacenadas en el directorio de Keycloak. Para validar el ciclo completo se requiere una cuenta de pruebas con credenciales permanentes (no temporales) y roles asignados.
- **Procedimiento técnico:**
    1. En el menú lateral izquierdo, haz clic en **Users** y luego en **Add user**.
    2. Completa los campos básicos:
        - **Username:** `alumno_demo`
        - **Email:** `alumno@ejemplo.com`
        - **First name:** `Alumno`
        - **Last name:** `Demo`
        - Activa el interruptor **Email verified** (`ON`).
    3. Haz clic en **Create**.



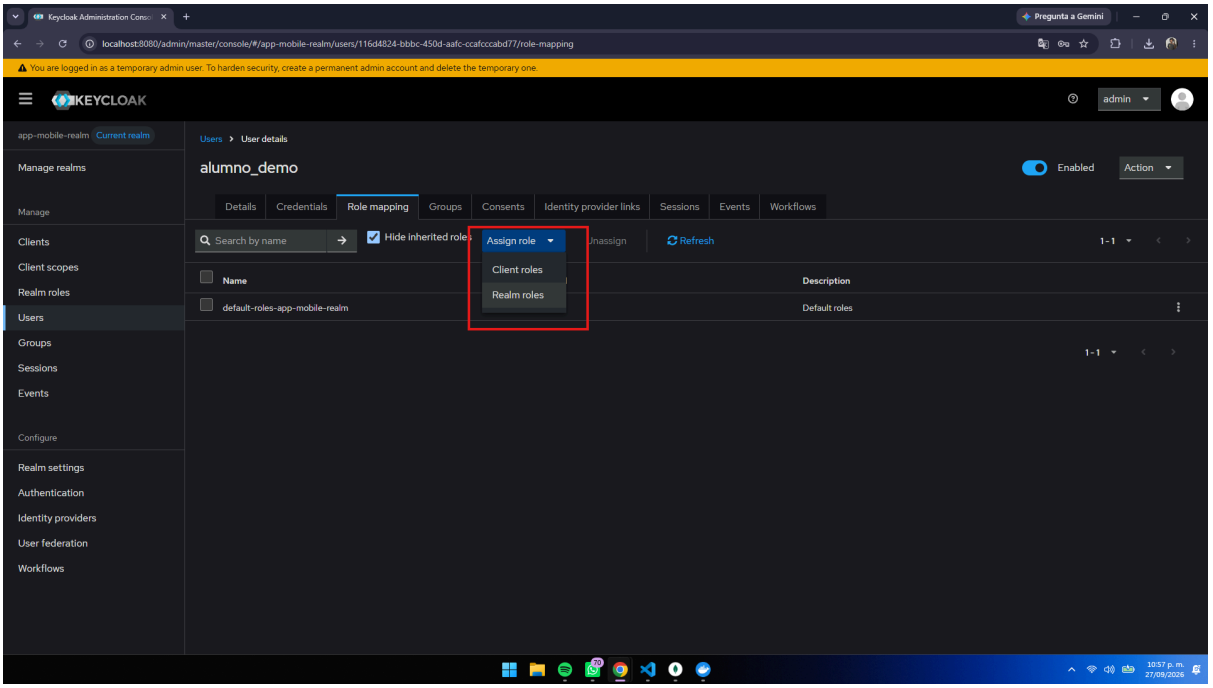
4. **Establecer Contraseña:**
   - En la parte superior de la ficha del usuario, ve a la pestaña **Credentials**.
   - Haz clic en **Set password**.

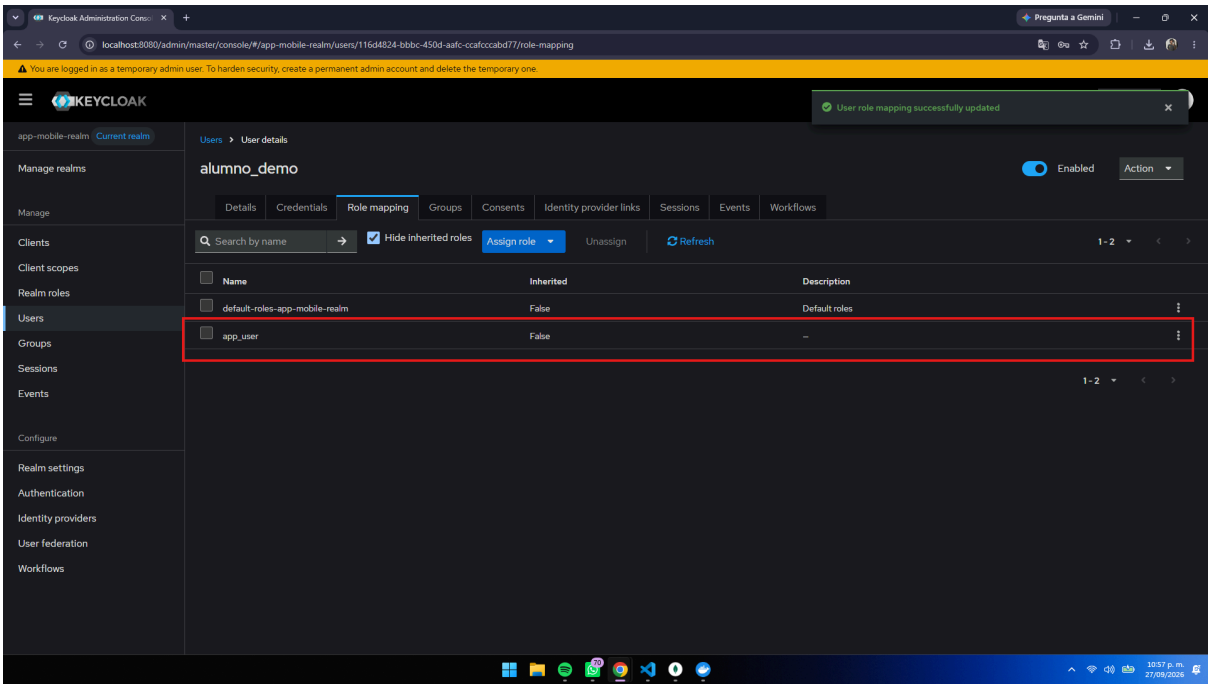


   - Escribe una contraseña de prueba (por ejemplo: 123456).
   - Desactiva el interruptor **Temporary** (OFF) para que el sistema no obligue a cambiar la contraseña en el primer inicio de sesión.
   - Haz clic en **Save** y confirma en **Save password**.



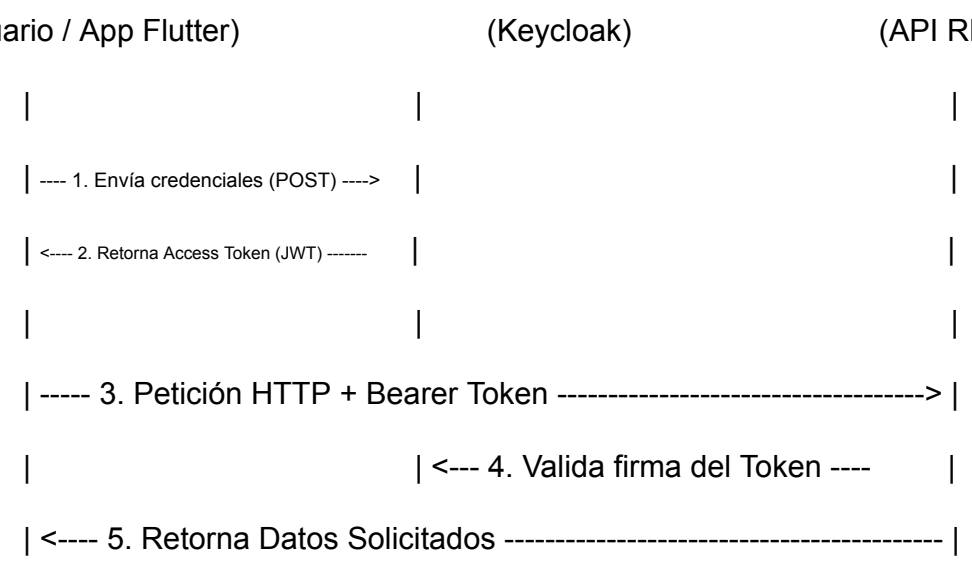
5. **Vincular el Rol:**
   - Ve a la pestaña **Role mapping** del usuario.
   - Haz clic en **Assign role**.
   - Selecciona la casilla de `app_user` y haz clic en **Assign**.





26. Flujo de autenticación

El flujo de autenticación garantiza que únicamente los usuarios validados puedan acceder a la información protegida. El proceso con Keycloak se describe de la siguiente forma:



1. El usuario abre la aplicación en Flutter e ingresa sus credenciales.
2. Flutter envía estas credenciales (vía protocolo OIDC/OAuth2) a Keycloak.
3. Keycloak valida los datos y, si son correctos, devuelve un Access Token (JWT).
4. Flutter almacena este token de manera segura en el dispositivo.
5. Cada vez que Flutter necesita consumir datos de la API REST (Backend), envía el Access Token en la cabecera (Bearer Token) de la petición HTTP.
6. La API REST intercepta la petición y valida la firma del JWT (puede usar Keycloak o validarlo localmente con su llave pública). Si es válido, se procesa la solicitud; si no, devuelve un error 401 Unauthorized.
7. Docker

27. Docker

27.1 ¿Qué es un contenedor?

un contenedor se puede representar como una caja o un paquete que guarda los recursos necesarios y esenciales para que un programa funcione sin problemas, guarda codigo, archivos, configuraciones y complementos, a diferencia de una máquina virtual que simula una computadora entera con todo su hardware y su propio sistema operativo, un contenedor

es algo mucho más ligero, además de acabar con el clásico error de “es que en mi maquina si funciona”

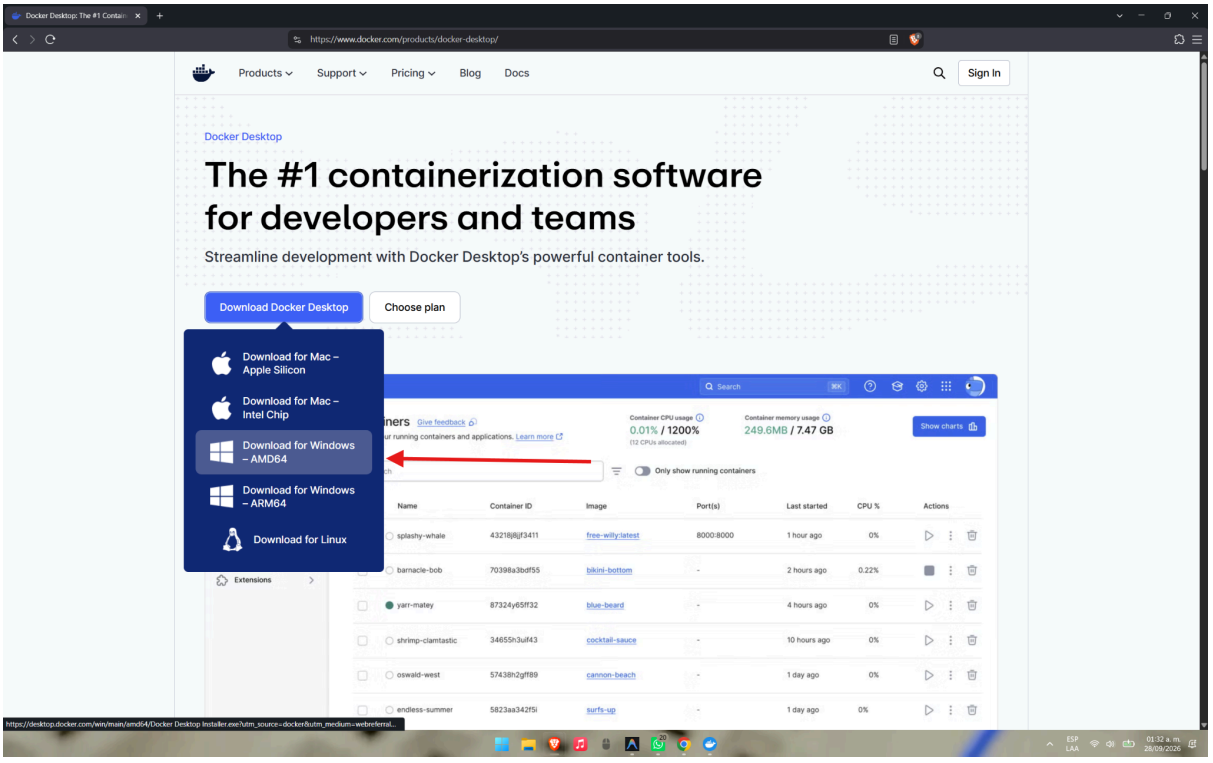
27.2 Conceptos clave

- **Imagen**: Es una plantilla de solo lectura. Contiene el código fuente, el entorno de ejecución, bibliotecas y herramientas necesarias para crear un contenedor.
- **Contenedor**: Es una instancia en ejecución de una imagen.
- **Volumen**: Permite que la información de bases de datos (como PostgreSQL o MongoDB) sobreviva incluso si el contenedor se destruye o reinicia.
- **Red** : Canal virtual que permite a múltiples contenedores aislarse del exterior y comunicarse entre sí de manera segura

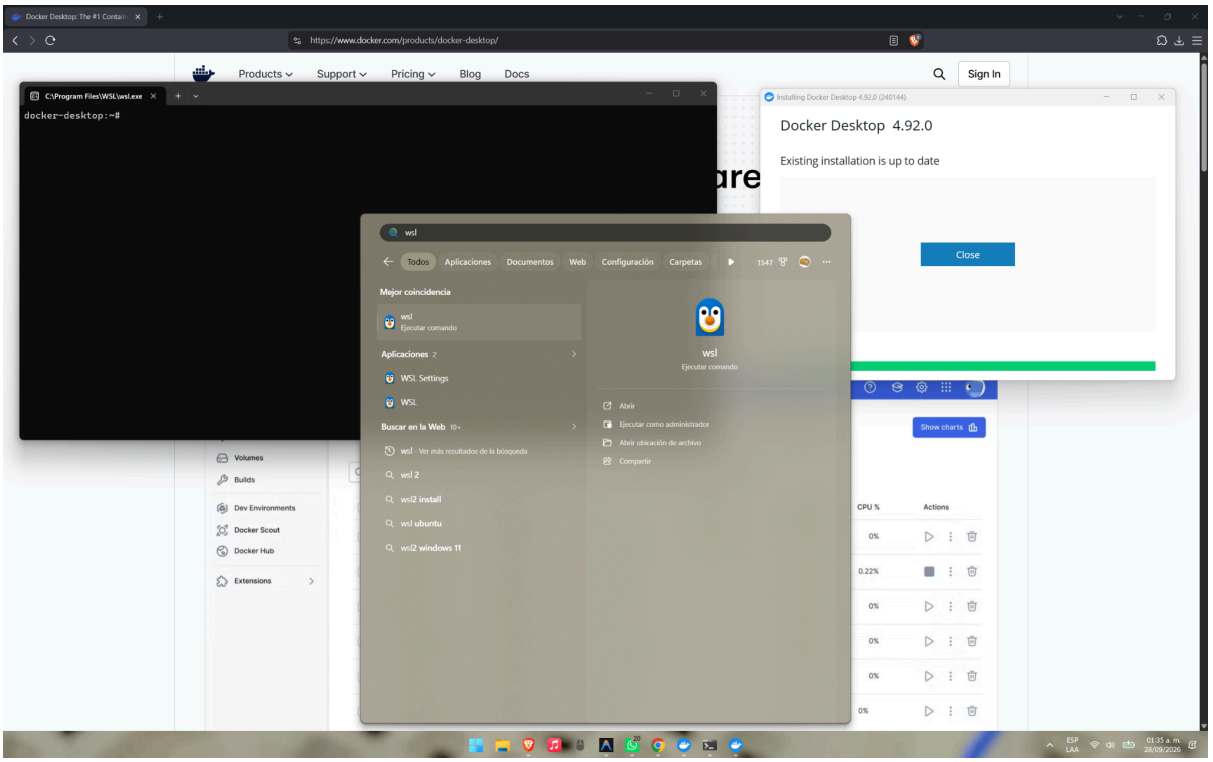
27.3 Instalación de Docker Desktop

Para entornos Windows y macOS, Docker Desktop es la herramienta gráfica recomendada:

1. Navega a docker.com/products/docker-desktop y descarga el instalador.

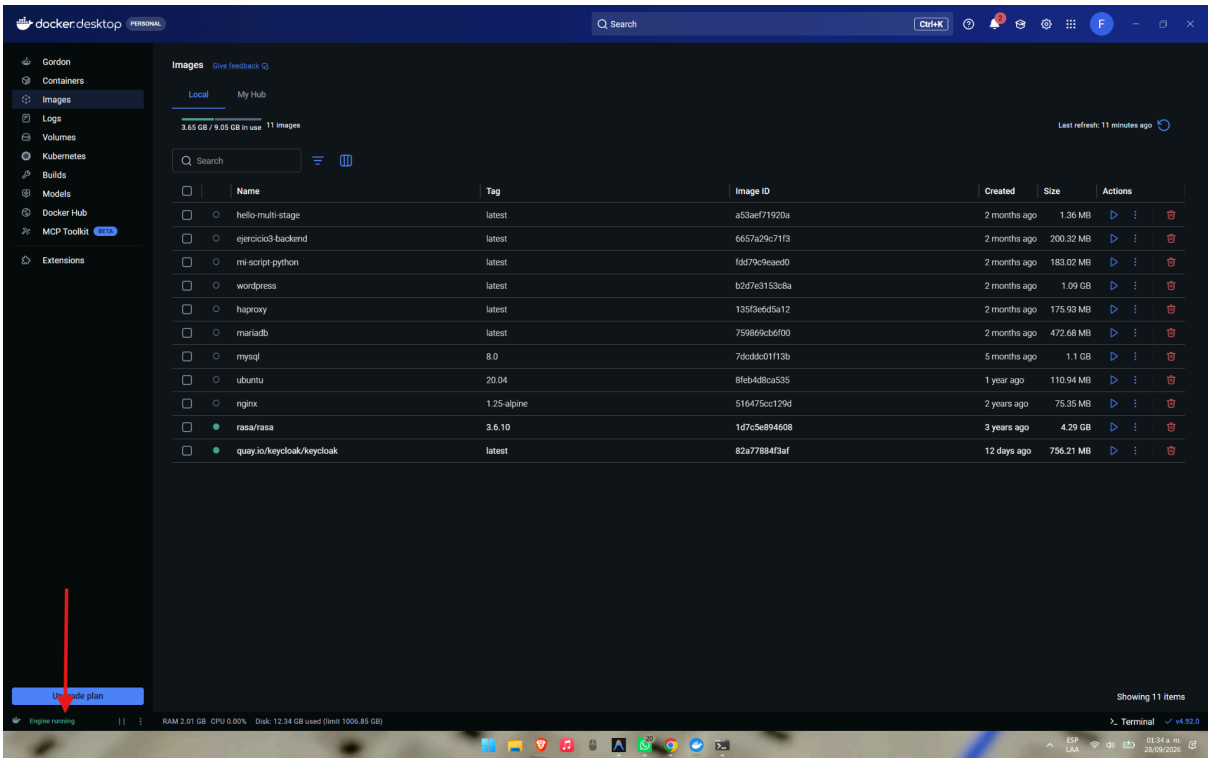


2. Sigue el asistente de instalación, asegurándote de usar WSL 2 (Windows Subsystem for Linux) si estás en Windows para tener un mejor rendimiento.



3. Reinicia tu computadora si el instalador lo solicita.

4. Abre la aplicación de Docker Desktop y acepta los términos. Deberás ver un ícono verde en la parte inferior izquierda indicando que el motor ("Engine") está corriendo.

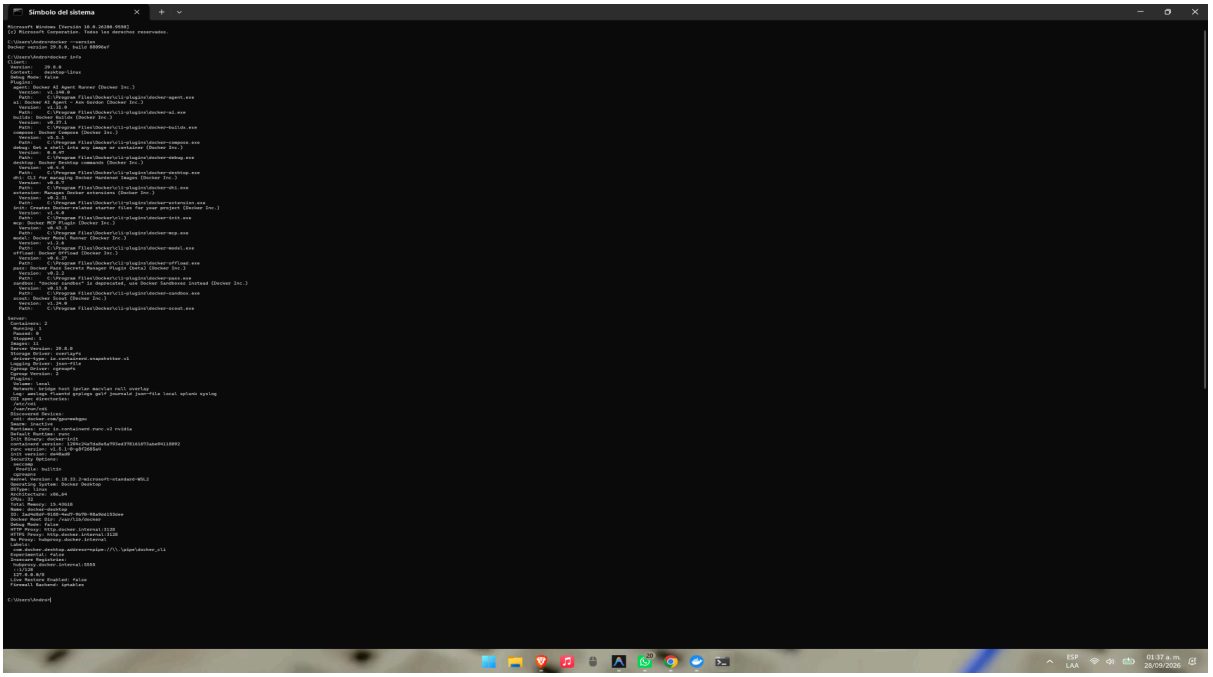


27.4 Verificación

Abre una terminal y ejecuta los siguientes comandos:

docker --version

docker info

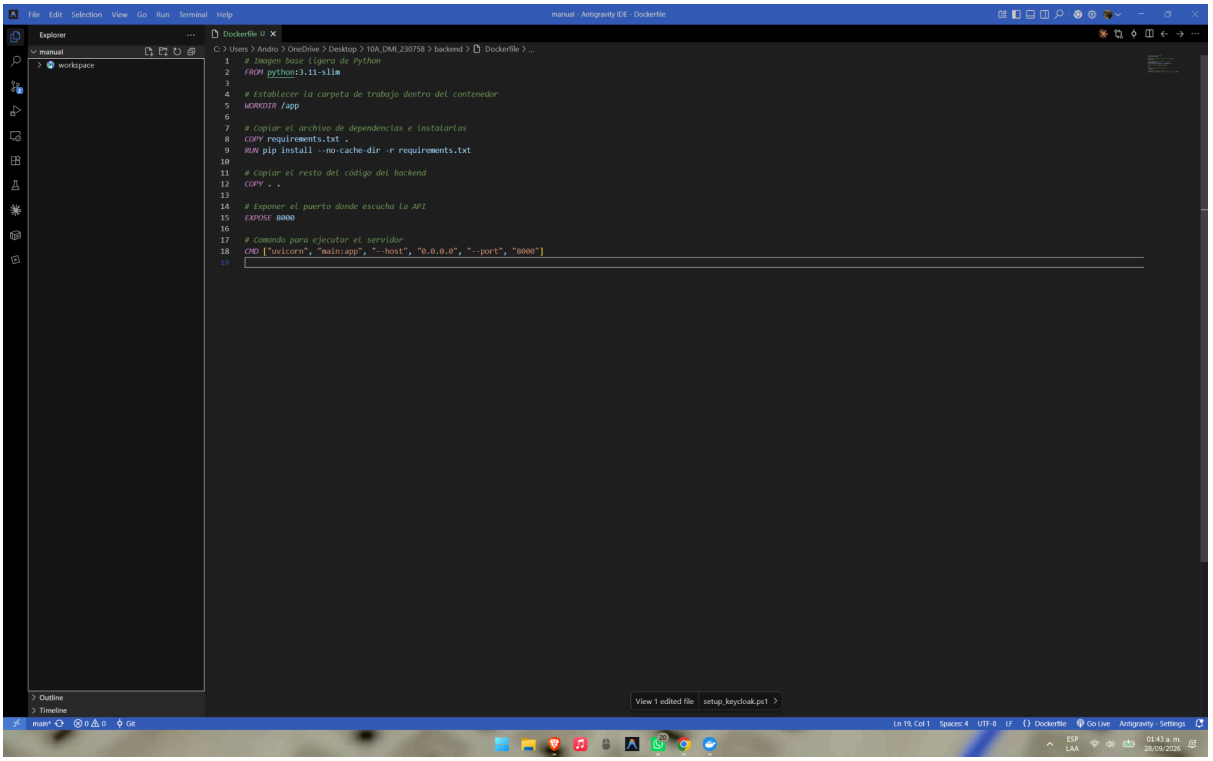



Símbolo del sistema
Microsoft Windows [Versión 10.0.26200.9550]
(c) Microsoft Corporation. Todos los derechos reservados.
C:\Users\Andro>docker --version
Docker version 29.8.0, build 88096ef
C:\Users\Andro>docker info
Client:
Version: 29.8.0
Context: desktop-linux
Debug Mode: false
Plugins:
agent: Docker AI Agent Runner (Docker Inc.)
Version: v1.140.0
Path: C:\Program Files\Docker\cli-plugins\docker-agent.exe
ai: Docker AI Agent - Ask Gordon (Docker Inc.)
Version: v1.31.0
Path: C:\Program Files\Docker\cli-plugins\docker-ai.exe
buildx: Docker Buildx (Docker Inc.)
Version: v0.37.1
Path: C:\Program Files\Docker\cli-plugins\docker-buildx.exe
compose: Docker Compose (Docker Inc.)
Version: v5.5.1
Path: C:\Program Files\Docker\cli-plugins\docker-compose.exe
debug: Get a shell into any image or container (Docker Inc.)
Version: 0.0.47
Path: C:\Program Files\Docker\cli-plugins\docker-debug.exe
desktop: Docker Desktop commands (Docker Inc.)
Version: v0.4.4
Path: C:\Program Files\Docker\cli-plugins\docker-desktop.exe
dhi: CLI for managing Docker Hardened Images (Docker Inc.)
Version: v0.0.7
Path: C:\Program Files\Docker\cli-plugins\docker-dhi.exe
extension: Manages Docker extensions (Docker Inc.)
Version: v0.2.31
Path: C:\Program Files\Docker\cli-plugins\docker-extension.exe
init: Creates Docker-related starter files for your project (Docker Inc.)
Version: v1.4.0
Path: C:\Program Files\Docker\cli-plugins\docker-init.exe
mcp: Docker MCP Plugin (Docker Inc.)
Version: v0.43.3
Path: C:\Program Files\Docker\cli-plugins\docker-mcp.exe
model: Docker Model Runner (Docker Inc.)
Version: v1.2.6
Path: C:\Program Files\Docker\cli-plugins\docker-model.exe
offload: Docker Offload (Docker Inc.)
Version: v0.6.27
Path: C:\Program Files\Docker\cli-plugins\docker-offload.exe
pass: Docker Pass Secrets Manager Plugin (beta) (Docker Inc.)
Version: v0.2.2
Path: C:\Program Files\Docker\cli-plugins\docker-pass.exe
sandbox: "docker sandbox" is deprecated, use Docker Sandboxes instead (Docker Inc.)
Version: v0.13.0
Path: C:\Program Files\Docker\cli-plugins\docker-sandbox.exe
scout: Docker Scout (Docker Inc.)
Version: v1.24.0
Path: C:\Program Files\Docker\cli-plugins\docker-scout.exe
Server:
Containers: 2
Running: 1
Paused: 0
Stopped: 1
Images: 11
Server Version: 29.8.0
Storage Driver: overlayfs
driver-type: io.containerd.snapshotter.v1
Logging Driver: json-file
Cgroup Driver: cgroupfs
Cgroup Version: 2
Plugins:
Volume: local
Network: bridge host ipvlan macvlan null overlay
Log: awslogs fluentd gcplogs gelf journald json-file local splunk syslog
CDI spec directories:
/etc/cdi
/var/run/cdi
Discovered Devices:
cdi: docker.com/gpu=webgpu
Swarm: inactive
Runtimes: runc io.containerd.runc.v2 nvidia
Default Runtime: runc
Init Binary: docker-init
containerd version: 1294c24a7da8e5a793ed378161673abe94118892
runc version: v1.5.1-0-g8f2685a4
init version: de40ad0
Security Options:
seccomp
Profile: builtin
cgroupns
Kernel Version: 6.18.33.2-microsoft-standard-WSL2
Operating System: Docker Desktop
OSType: linux
Architecture: x86_64
CPUs: 32
Total Memory: 15.436iB
Name: docker-desktop
ID: 2ad4d8df-9188-4ed7-9b70-98a9dd153dee
Docker Root Dir: /var/lib/docker
Debug Mode: false
HTTP Proxy: http.docker.internal:3128
HTTPS Proxy: http.docker.internal:3128
No Proxy: hubproxy.docker.internal
Labels:
com.docker.desktop.address=npipe://\\.\pipe\docker_cli
Experimental: false
Insecure Registries:
hubproxy.docker.internal:5555
::1/128
127.0.0.0/8
Live Restore Enabled: false
Firewall Backend: iptables
C:\Users\Andro>

Si la terminal te muestra los detalles de la versión y la configuración del motor de Docker sin errores, significa que la herramienta está instalada y corriendo correctamente.

28. Dockerfile

El archivo Dockerfile es un script de texto (sin extensión) que contiene las instrucciones paso a paso para construir una Imagen de Docker personalizada. En la arquitectura de este proyecto, servirá principalmente para empaquetar y aislar el entorno de nuestro Backend (la API REST).



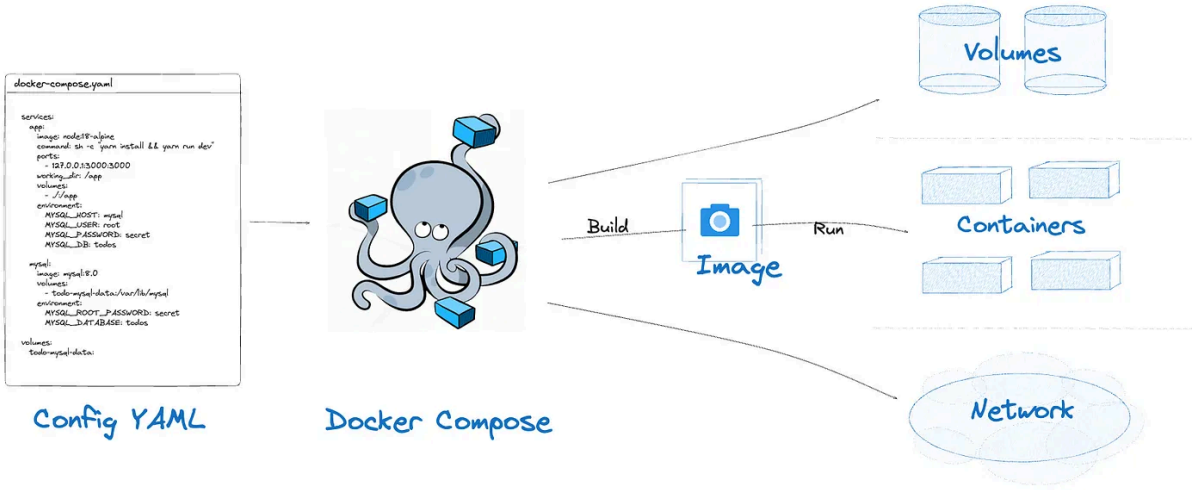
Instrucciones base más utilizadas para construir la imagen:

- **FROM**: Define la imagen base desde la cual partimos.
- **WORKDIR**: Establece la carpeta de trabajo principal dentro del contenedor.
- **COPY**: Copia los archivos del código fuente local desde nuestra computadora física hacia el interior del contenedor.
- **RUN**: Ejecuta comandos durante la fase de construcción de la imagen, útil para instalar dependencias.
- **EXPOSE**: Declara el puerto en el que el contenedor estará escuchando peticiones de red hacia el exterior.
- **CMD**: Es la instrucción final y define el comando por defecto que mantiene viva a la aplicación al momento de arrancar el contenedor.

29. Docker Compose

29.1 Concepto

Aunque Docker sirve de forma nativa para levantar un solo contenedor a la vez, los proyectos de software necesitan muchos servicios corriendo al mismo tiempo. Docker Compose es una herramienta que nos permite definir entornos de varios contenedores mediante un único archivo de configuración YAML (docker-compose.yml).



29.2 Arquitectura

Para nuestro entorno, Compose se encargará de orquestar 4 servicios simultáneos corriendo dentro de una misma red virtual aislada:

docker-compose.yml

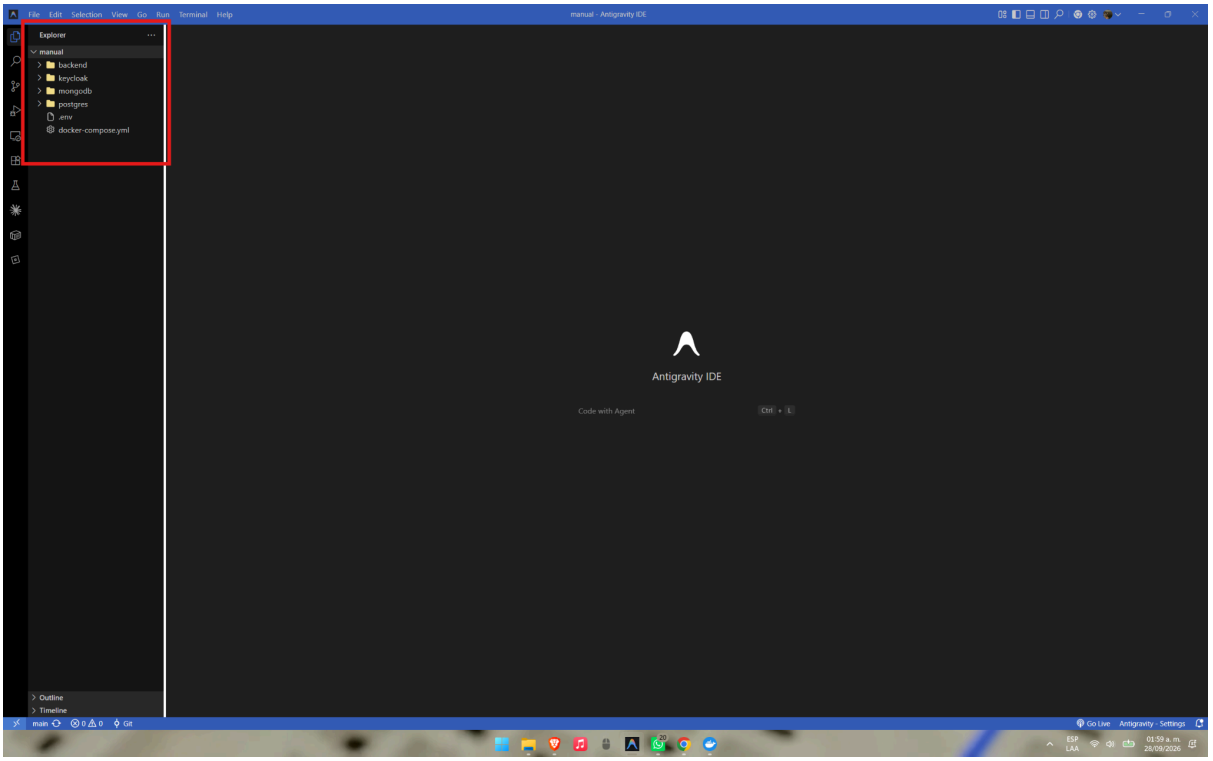
│

├── backend

├── postgres

├── mongodb

└── keycloak

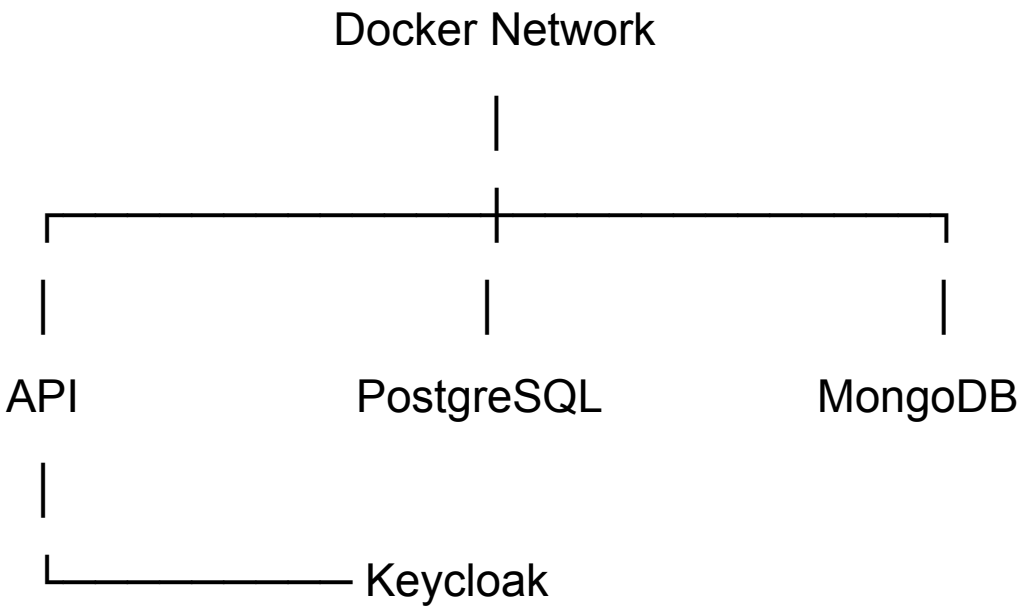


29.3 Componentes principales de un archivo docker-compose.yml

Al construir nuestro archivo YAML de orquestación, se definen los siguientes nodos principales:

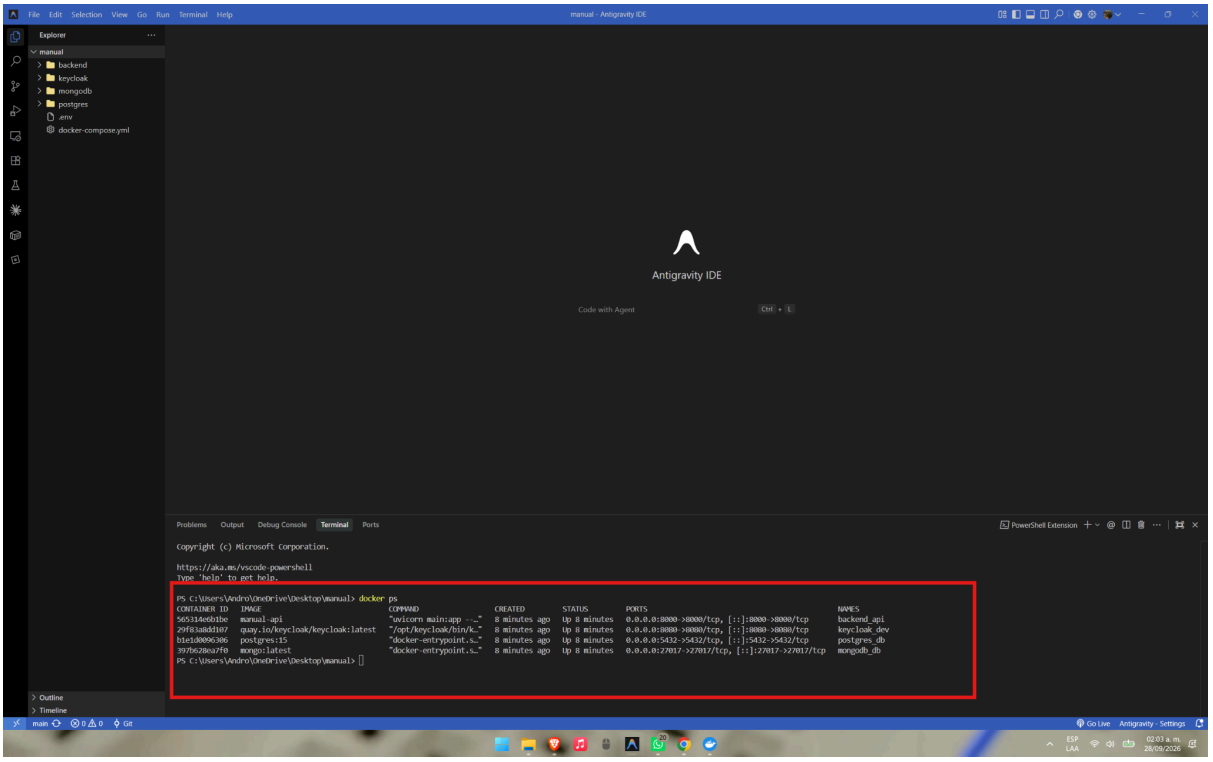
- **version**: Define la especificación del archivo de compose.
- **services**: Declara los bloques individuales para cada contenedor que deseamos levantar.
- **image**: Indica de dónde descargar la imagen pública o, en su defecto, la ruta del Dockerfile para construir la imagen localmente.
- **ports**: Mapea los puertos del contenedor aislado hacia los puertos de nuestra máquina física.
- **environment**: Inyecta las variables de entorno necesarias para iniciar un contenedor.
- **volumes**: Declara la persistencia de datos. Los volúmenes evitan que perdamos las tablas de PostgreSQL o colecciones de MongoDB al apagar y eliminar los contenedores.
- **depends_on**: Dicta el orden lógico de encendido.

30. Infraestructura completa con Docker Compose



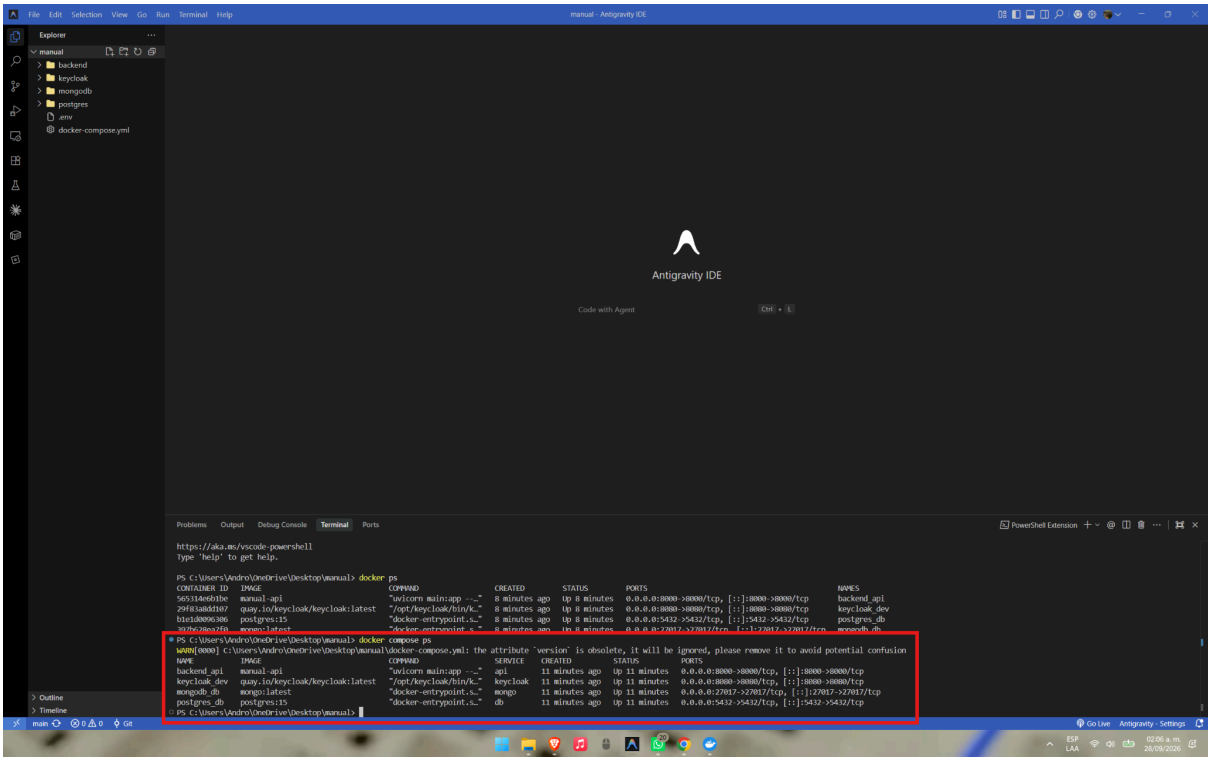
Comando:

docker compose up -d



Validación:

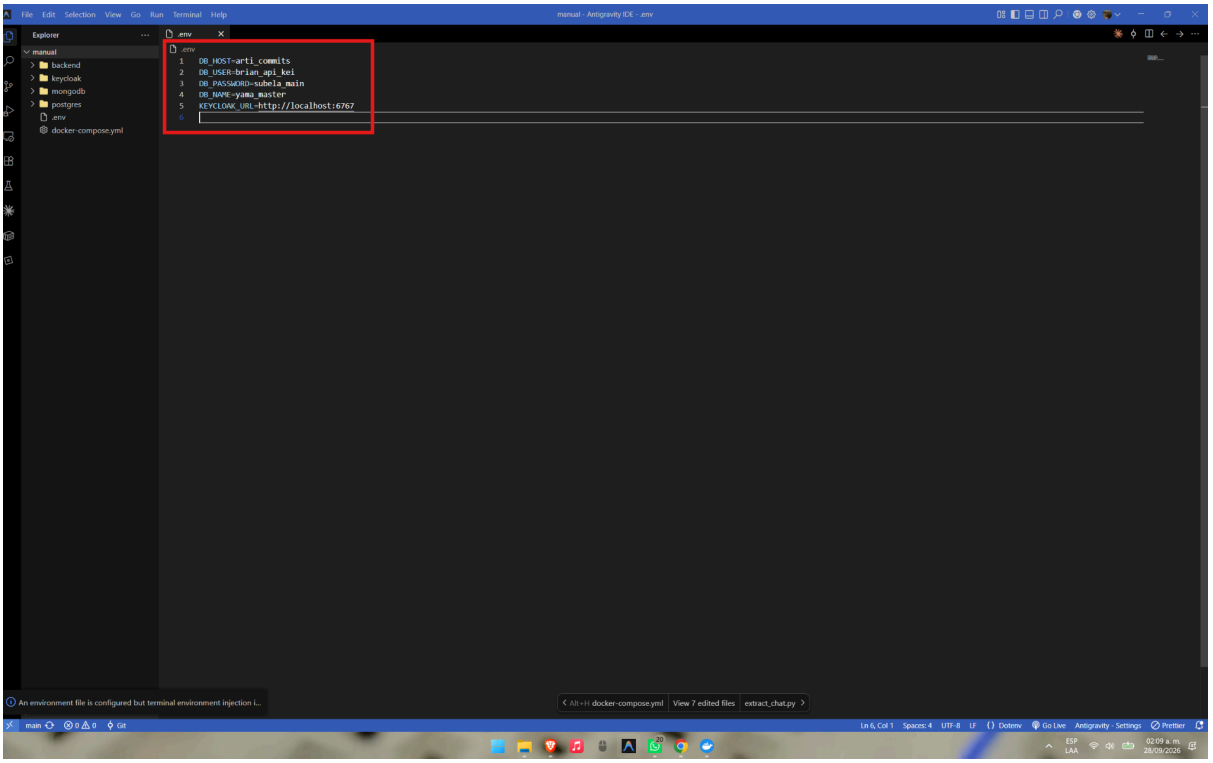
docker compose ps



31. Variables de entorno

Muy importante para buenas prácticas.

Las variables de entorno son de gran importancia por dos razones: evitan que las contraseñas se queden "quemadas" (hardcodedadas) en el código fuente, y permiten configurar el entorno sin tener que modificar el código de la aplicación, ejemplo:



Archivo .env

DB_HOST=arti_commits

DB_USER=brian_api_kei

DB_PASSWORD=subela_main

DB_NAME=yama_master

KEYCLOAK_URL=http://localhost:6767

IMPORTANTE: El archivo .env nunca debe subirse al repositorio público de código. Para evitar filtraciones, siempre se debe asegurar que el archivo .gitignore contenga la línea .env.

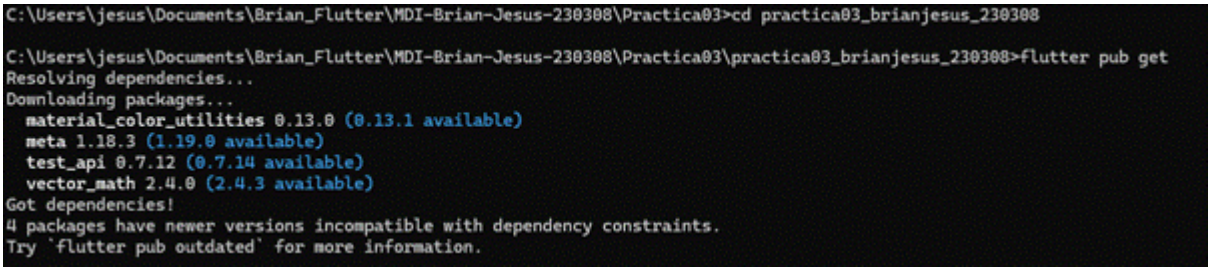
32 Integración Flutter y API

32.1 Preparación y arquitectura

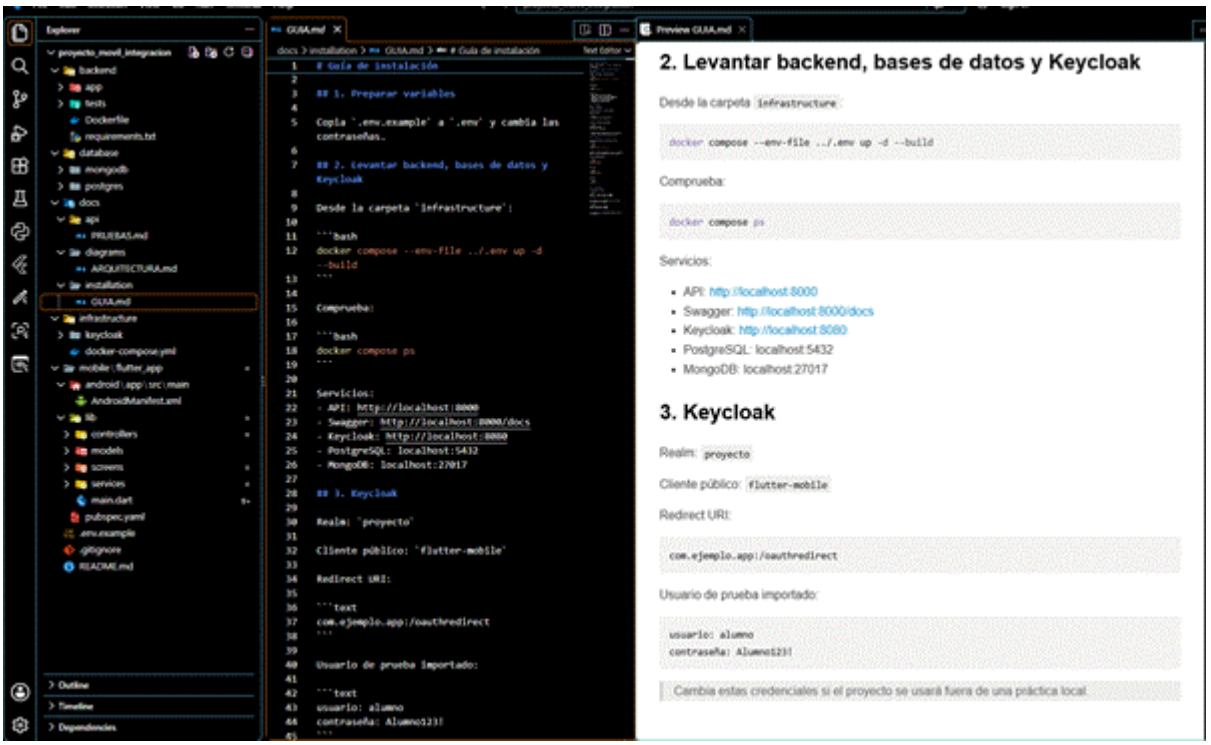
Desde la carpeta mobile/flutter_app, verifica la instalación y agrega los paquetes. El paquete http realiza solicitudes; provider comunica el estado de la pantalla con el servicio.

flutter doctor
flutter pub get
flutter pub add http provider flutter_appauth









La UI muestra datos y acciones; el Controller mantiene loading, datos y mensajes de error; el Service construye solicitudes y transforma respuestas; el cliente HTTP se conecta a la API.

32.2 GET y 32.3 POST

Configura una dirección accesible desde el dispositivo. En el emulador Android, 10.0.2.2 apunta al equipo anfitrión; desde un teléfono físico usa la IP local del equipo. Ajusta el puerto y las rutas a tu backend.

```
import 'dart:convert';
import 'package:http/http.dart' as http;

class ProductoService {
  ProductoService(this.client, this.baseUrl);
  final http.Client client;
  final String baseUrl;

  Future<List<dynamic>> obtener(String token) async {
            final r = await client.get(
            Uri.parse('$baseUrl/productos'),
            headers: {'Authorization': 'Bearer $token'},
            ).timeout(const Duration(seconds: 10));
            if (r.statusCode != 200) throw Exception('GET: ${r.statusCode}');
            return jsonDecode(utf8.decode(r.bodyBytes)) as List<dynamic>;
  }

  Future<Map<String, dynamic>> crear(
            String nombre, String token) async {
            final r = await client.post(
            Uri.parse('$baseUrl/productos'),
            headers: {
            'Content-Type': 'application/json',
            'Authorization': 'Bearer $token',
            },
            body: jsonEncode({'nombre': nombre}),
```

```
            ).timeout(const Duration(seconds: 10));
            if (r.statusCode != 201 && r.statusCode != 200) {
            throw Exception('POST: ${r.statusCode}');
            }
            return jsonDecode(utf8.decode(r.bodyBytes)) as Map<String, dynamic>;
  }
}
```

32.4 JSON 32.5 Errores y 32.6 Carga

jsonEncode convierte los datos de Dart en el cuerpo JSON del POST; jsonDecode convierte la respuesta en objetos de Dart. Para un proyecto mayor, crea modelos tipados con fromJson y toJson. Ajusta la forma de respuesta si la API devuelve un objeto con una lista interna.
En el Controller, activa loading antes de llamar al servicio y desactívalo en finally. Muestra un mensaje comprensible si ocurre timeout, falla la conexión o llega un código 4xx/5xx. Trata 401 como sesión no válida y 403 como permiso insuficiente.

```
bool loading = false;
String? error;
List<dynamic> productos = [];

Future<void> cargar(ProductoService service, String token) async {
  loading = true; error = null; notifyListeners();
  try { productos = await service.obtener(token); }
  catch (_) { error = 'No fue posible cargar los productos'; }
  finally { loading = false; notifyListeners(); }
}
```

La vista observa al Controller: muestra un indicador si loading es true, un mensaje si error tiene texto y la lista cuando la solicitud termina correctamente.

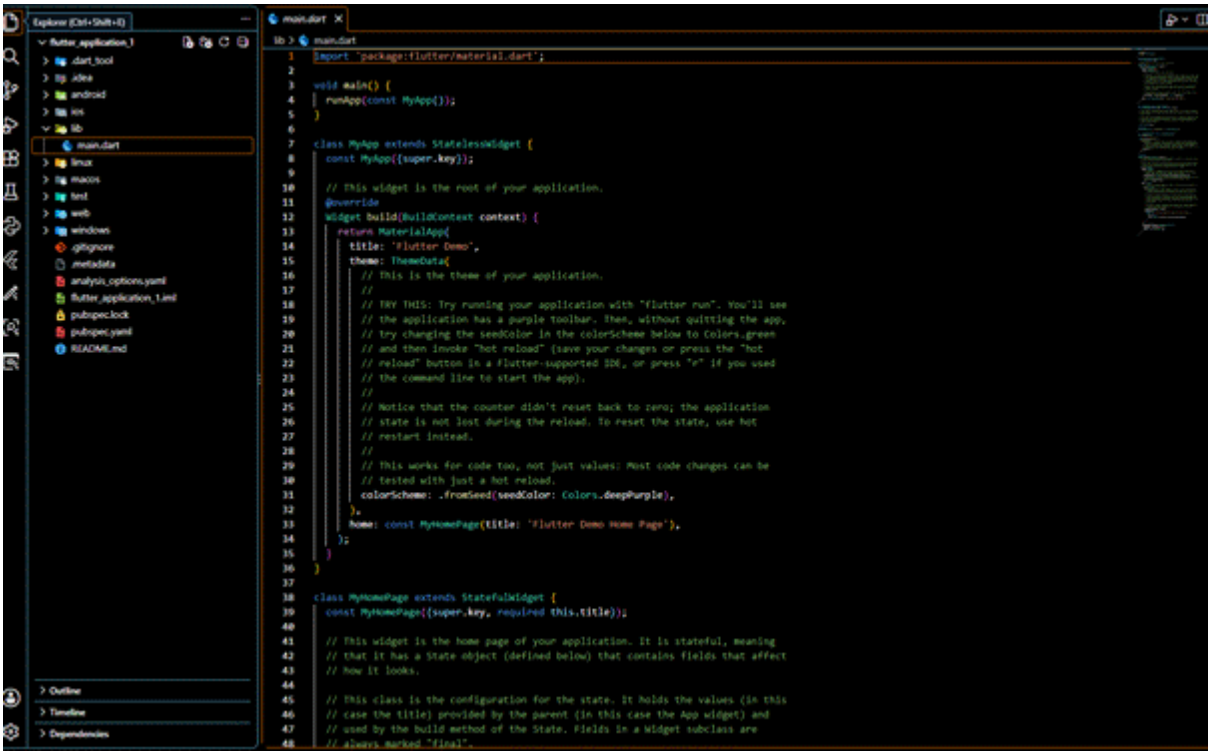
32.4 JSON 32.5 Errores y 32.6 Carga

jsonEncode convierte los datos de Dart en el cuerpo JSON del POST; jsonDecode convierte la respuesta en objetos de Dart. Para un proyecto mayor, crea modelos tipados con fromJson y toJson. Ajusta la forma de respuesta si la API devuelve un objeto con una lista interna.
En el Controller, activa loading antes de llamar al servicio y desactívalo en finally. Muestra un mensaje comprensible si ocurre timeout, falla la conexión o llega un código 4xx/5xx. Trata 401 como sesión no válida y 403 como permiso insuficiente.

```
bool loading = false;
String? error;
List<dynamic> productos = [];

Future<void> cargar(ProductoService service, String token) async {
  loading = true; error = null; notifyListeners();
  try { productos = await service.obtener(token); }
  catch (_) { error = 'No fue posible cargar los productos'; }
  finally { loading = false; notifyListeners(); }
}
```

La vista observa al Controller: muestra un indicador si loading es true, un mensaje si error tiene texto y la lista cuando la solicitud termina correctamente



33 Integración Flutter y Keycloak



Imagen 2. Ejemplo ilustrativo de inicio de sesión con Keycloak.

Configura un cliente público en Keycloak para la aplicación móvil, habilita Authorization Code con PKCE y registra una URI de retorno que también reconozca Flutter. Usa el navegador del sistema mediante flutter_appauth; evita enviar usuario y contraseña directamente desde el código de la aplicación.

```
final auth = FlutterAppAuth();
final resultado = await auth.authorizeAndExchangeCode(
  AuthorizationTokenRequest(
            'flutter-mobile',
            'com.ejemplo.app:/oauthredirect',
            issuer: 'https://servidor/realms/proyecto',
            scopes: ['openid', 'profile'],
  ),
);
final token = resultado?.accessToken;
```

Keycloak valida las credenciales y entrega un access token. Flutter lo mantiene durante la sesión y lo envía en cada solicitud protegida. La API verifica firma, emisor, vencimiento y permisos antes de responder. Para conservar una sesión al cerrar la app, usa almacenamiento seguro del sistema y una política de renovación; nunca incluyas secretos de cliente en el código móvil.

```
Authorization: Bearer TOKEN
```

El encabezado anterior pertenece a HTTP. El token real sustituye TOKEN y no debe aparecer en capturas, registros ni archivos compartidos.

34 pruebas de integración

Ejecuta cada prueba en el entorno instalado y registra evidencia, fecha y resultado. Los estados esperados son criterios de aceptación, no pruebas ya realizadas.

Prueba	Herramienta	Resultado esperado
Flutter instalado	flutter doctor	Sin errores críticos
API activa	Swagger /docs	Respuesta HTTP 200
PostgreSQL	API	Consulta realizada
MongoDB	API	Documento creado
Keycloak	Postman o AppAuth	Token obtenido
API protegida	Postman	HTTP 200 con token válido
Sin token	Postman	HTTP 401
Docker	docker compose ps	Servicios activos
Flutter → API	Aplicación Flutter	Datos visualizados

Imagen 3. Ejemplo ilustrativo de una prueba de acceso a la API.

Registra también un caso de token vencido, permisos insuficientes y pérdida de red. La API puede devolver 403 cuando el usuario está autenticado, pero carece de permisos.

35 Flujo Git recomendado

Arquitectura

```
Flutter UI
   ↓
Provider / ProductoController
   ↓
ProductoService
   ↓
HTTP Client
   ↓
FastAPI
   ├── PostgreSQL
   ├── MongoDB
   └── Keycloak (validación de token)
```

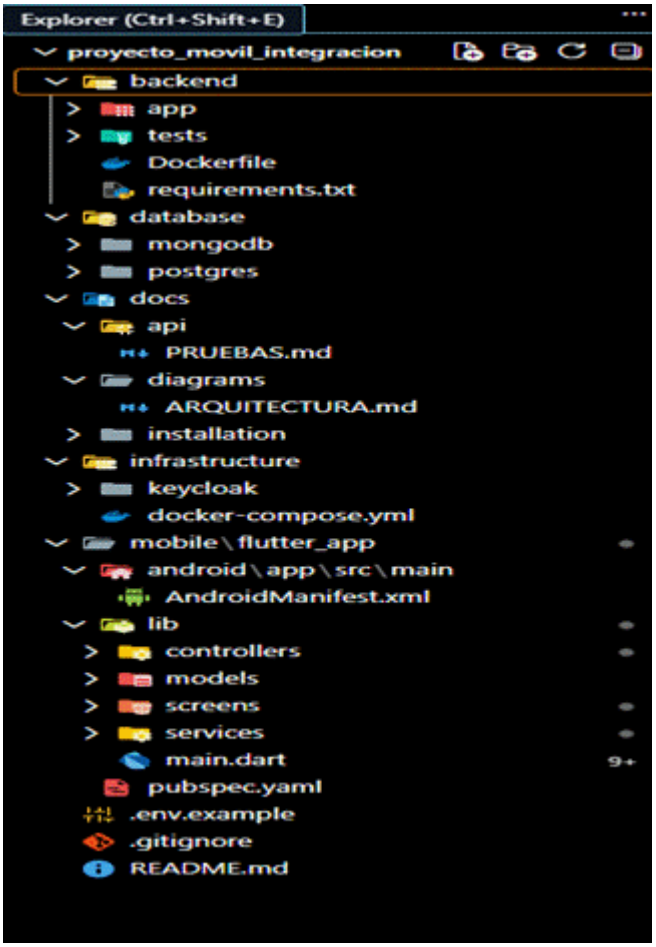
Flujo de autenticación

```
Usuario
  ↓
Flutter
  ↓
Navegador del sistema
  ↓
Keycloak
  ↓
Authorization Code + PKCE
  ↓
Access Token
  ↓
Flutter
  ↓
Authorization: Bearer TOKEN
  ↓
```

Imagen 4. Ejemplo ilustrativo del repositorio y su rama de trabajo.

main conserva las entregas estables; develop reúne cambios para la siguiente entrega. Cada integrante crea una rama feature desde develop, realiza commits pequeños con mensajes claros y hace push al repositorio remoto. Abre un Pull Request hacia develop, solicita revisión, corrige observaciones y hace merge tras aprobar. Cuando se valida la entrega, se integra develop en main.

36 estructura recomendada del repositorio



mobile contiene Flutter; backend guarda la API y sus pruebas; database reúne archivos de inicialización cuando sean necesarios; infrastructure agrupa Docker y Keycloak; docs conserva instrucciones y diagramas. En .env.example coloca nombres de variables sin credenciales reales. README.md explica cómo levantar y comprobar el proyecto.

37. Verificación final del entorno (Checklist de cierre)

Herramientas

- ☑ Git, GitHub, VS Code instalados y funcionales.
- ☑ Flutter, Android SDK, Android Emulator, Postman y Docker operativos.

Backend

- ☑ FastAPI o Express funcionando, Swagger UI accesible.
- ☑ Endpoints GET y POST probados con éxito.

Bases de datos

- ☑ PostgreSQL y MongoDB en ejecución vía Docker y accesibles.

Seguridad

- ☑ Keycloak en ejecución, Realm creado, Client configurado, Usuario de prueba creado, Token JWT obtenido.

Aplicación móvil

- ☑ Aplicación Flutter ejecuta en el emulador, consume la API backend, procesa JSON y envía token JWT en las peticiones.

38. Problemas frecuentes y solución

Problema	Posible causa	Solución
`flutter` no reconocido	PATH incorrecto en variables del sistema.	Revisar y corregir la variable de entorno PATH.
Emulador no inicia	Virtualización deshabilitada.	Habilitar la virtualización (VT-x/AMD-V) en la BIOS.

API no responde	Puerto cerrado o proceso no iniciado.	Verificar que el servidor backend se encuentre en ejecución.
Docker no inicia	Backend WSL2 / virtualización inactiva.	Revisar la configuración de Docker Desktop y componentes de Windows.
PostgreSQL no conecta	Credenciales o puerto incorrecto.	Revisar el archivo de configuración `.env`.
Mongo no conecta	URI de conexión mal formada.	Revisar la cadena URI de conexión `mongodb://...`
401 Unauthorized	Token expirado o inválido.	Revisar la configuración de autenticación y Realm en Keycloak.
CORS	Backend restringe origen cruzado.	Configurar el middleware de CORS en el framework backend.
Flutter no accede a localhost	Dirección IP local en emulador Android.	Utilizar `10.0.2.2` para referenciar la máquina host desde Android.

39. Buenas prácticas de seguridad

- No guardar contraseñas ni claves secretas directamente en el código fuente.
- Evitar incluir archivos de variables de entorno `.env` dentro del repositorio de GitHub.
- Utilizar protocolo HTTPS para las comunicaciones en entornos de producción.
- Utilizar Tokens JWT estructurados con un tiempo razonable de expiración.
- Aplicar control de acceso basado en roles (RBAC) y validar todos los datos de entrada.

- Separar claramente los entornos de desarrollo, pruebas y producción.
- Mantener todas las dependencias del proyecto actualizadas.

40. Buenas prácticas de desarrollo

- Aplicar principios de Clean Code y refactorización continua.
- Garantizar la separación estricta de responsabilidades.
- Diseñar la arquitectura por capas (Presentación, Lógica, Datos).
- Mantener la modularidad en los componentes.
- Utilizar control de versiones mediante flujos de trabajo con Git (GitFlow).
- Diseñar e implementar APIs RESTful siguiendo los estándares de la industria.
- Documentar los endpoints y componentes principales del sistema.
- Implementar pruebas unitarias e integrales.
- Utilizar contenedores para aislar los entornos de ejecución.

41. Actividad integradora del manual

Aplicación móvil de gestión de tareas

- **Flutter:** Flujo de Login → Dashboard → Lista de Tareas → Perfil de Usuario.
- **Backend:** Endpoints implementados:
  - `POST /auth` (Autenticación)
  - `GET /tasks` (Listado de tareas)
  - `POST /tasks` (Creación de tarea)
  - `PUT /tasks/{id}` (Actualización)
  - `DELETE /tasks/{id}` (Eliminación)
- **Persistencia PostgreSQL:** Tablas `users`, `tasks`, `categories`.
- **Persistencia MongoDB:** Colecciones `logs`, `activity`, `notifications`.
- **Keycloak:** Configuración de roles `usuario` y `administrador`.
- **Ejecución:** Despliegue de servicios integrados mediante `docker compose up -d`.

42. Evidencias de instalación

- **E01 — Flutter:** Captura de pantalla con el comando `flutter doctor` sin errores.
- **E02 — Git:** Salida en consola de `git --version`.

```
Last login: Mon Sep 28 14:52:26 on console
[farias@fariaslymc ~ % git --version
git version 2.54.0 (Apple Git-157)
[farias@fariaslymc ~ % flutter doctor
Doctor summary (to see all details, run flutter doctor -v):
[✓] Flutter (Channel stable, 3.47.2, on macOS 27.0 26A428 darwin-arm64, locale es-MX)
[✓] Android toolchain - develop for Android devices (Android SDK version 36.1.0)
[!] Xcode - develop for iOS and macOS (Xcode 27.0)
    ! CocoaPods not installed.
        CocoaPods is a package manager for iOS or macOS platform code.
        Without CocoaPods, plugins will not work on iOS or macOS.
        For more info, see https://flutter.dev/to/platform-plugins
      For installation instructions, see https://guides.cocoapods.org/using/getting-started.html#installation
[✓] Chrome - develop for the web
[✓] Connected device (3 available)
[✓] Network resources
```

- **E03 — Docker:** Salida de `docker --version` y `docker compose version`.

```
[farias@fariaslymc ~ % docker --version
 Docker version 29.7.2, build a7dcaa6
[farias@fariaslymc ~ % docker compose version
 Docker Compose version v5.5.1
```

- **E04 — PostgreSQL:** Ejecución de la consulta `SELECT version();` desde el cliente de base de datos.
- **E05 — MongoDB:** Consulta exitosa del estado y versión de la instancia.
- **E06 — Backend:** Captura de la interfaz interactiva Swagger UI cargada en el navegador.
- **E07 — Keycloak:** Captura de la consola de administración mostrando Realm, Client y Usuario creados.
- **E08 — Aplicación:** Captura del emulador ejecutando la aplicación Flutter consumiendo los datos de la API.

43. Glosario

- **API:** Interfaz de Programación de Aplicaciones.
- **REST:** Transferencia de Estado Representacional.
- **Endpoint:** URL específica donde se accede a un servicio o recurso dentro de una API.
- **Backend:** Capa de procesamiento y datos de una aplicación.
- **Frontend:** Capa de presentación e interfaz gráfica con la que interactúa el usuario.
- **SDK:** Kit de Desarrollo de Software.
- **Framework:** Entorno o estructura de trabajo estandarizada para el desarrollo de software.
- **Runtime:** Entorno de ejecución de programas.
- **ORM:** Mapeo Objeto-Relacional.
- **SQL:** Lenguaje de Consulta Estructurado.
- **NoSQL:** Base de datos no relacional o no estructurada.

- **JSON:** Notación de Objetos de JavaScript.
- **JWT:** JSON Web Token.
- **OAuth:** Protocolo abierto de autorización.
- **OpenID Connect:** Capa de identidad sobre el protocolo OAuth 2.0.
- **Contenedor:** Unidad estándar de software que empaqueta código y dependencias.
- **Imagen:** Plantilla de solo lectura utilizada para crear contenedores.
- **Volumen:** Mecanismo para persistir datos generados por contenedores.
- **Repositorio:** Almacenamiento digital donde se guarda el código fuente y su historial.
- **Branch:** Línea independiente de desarrollo en un repositorio de Git.
- **Commit:** Registro de cambios guardados en el historial del repositorio.
- **Pull Request:** Solicitud para integrar cambios de una rama a otra en una plataforma de Git.

44. Anexos

Anexo A: Comandos Git de uso frecuente

```
git clone <url_repositorio>
git status
git add .
git commit -m "Mensaje descriptivo"
git push origin <rama>
git pull origin <rama>
git branch
git switch <nombre_rama>
```

Anexo B: Comandos Flutter de uso frecuente

```
flutter doctor
flutter devices
flutter create <nombre_proyecto>
flutter run
flutter clean
flutter pub get
```

Anexo C: Comandos Docker de uso frecuente

```
docker ps
docker images
docker compose up -d
```

```
docker compose down
docker compose logs -f
```

Anexo D: Comandos y fragmentos adicionales (PostgreSQL, MongoDB, FastAPI, Node/Express)

Ejemplo de endpoint básico en FastAPI

Python

```python
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def read_root():
    return {"status": "API activa", "version": "1.0.0"}
```